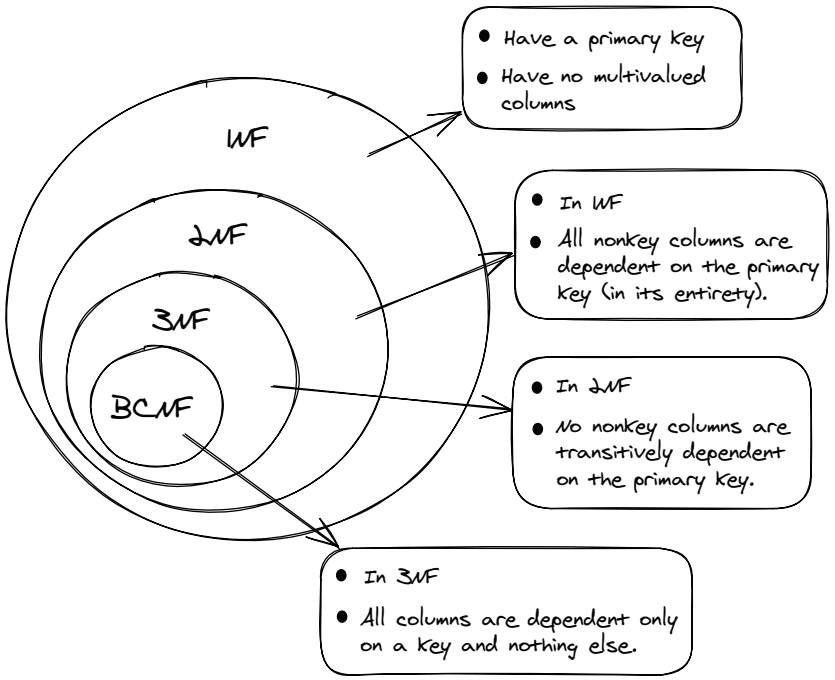


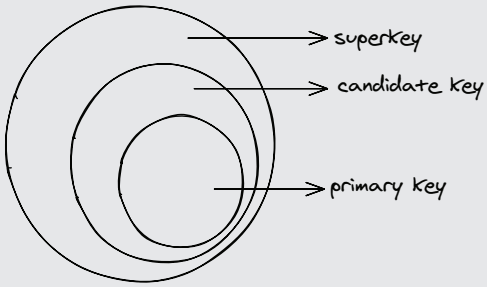
You will use normal forms to guide the normalization process. There are multiple normal forms, including the First Normal Form (1NF), Second Formal Form (2NF), Third Normal Form (3NF), and Boyce-Codd Normal Form (BCNF). The relationship among these normal forms is hierarchical and sequential in database normalization. Each form builds on the preceding one. Being the smallest nesting doll of these four normal forms, BCNF has all the characteristics of the other three.



Superkeys, candidate keys, and primary keys

Do you remember what they are? The coverage of different types of keys goes back to chapter 4.

A *superkey* is a set of one or more columns of a table that can uniquely identify a row in the table, but it may contain columns that are not required to uniquely identify a row. A *candidate key* is a minimal superkey, which means that it is a superkey with no unnecessary columns. Of all the candidate keys in a table, one is chosen as the primary key. The following figure summarizes the relationships among the three types of key:



In practice, when all your tables are in BCNF, you can consider your database fully normalized. A table in BCNF is already in 1NF to 3NF and has the following characteristics:

- It has a primary key.
- It has no multivalued columns.
- All columns are dependent on a key but nothing else.
- It contains no transitive dependency.

You will find detailed definitions of these characteristics in the following sections and use them as the guidelines to normalize the database design of The Sci-Fi Collective. A table that doesn't meet one, two, or any guidelines may cause problems such as update and deletion anomalies. Remember attempting to combine product and customer data into one table in chapter 1?

product_id	name	price	manufacturer	customer_id	customer_name	customer_email	quantity
1	Atomic Nose ...	19.99	Mad Inventors Inc.	a1	Bob	bob@gmail.com	5
2	Selfie Toaster	24.99	Goofy Gadgets Corp.	b2	Dave	dave@outlook.com	15
3	Cat-Poop Coffee	29.99	Absurd Accessories	a1	Bob	bob@gmail.com	2
...	...	...	...	...	...	...	...
9	The Infinite ...	9.99	Silly Supplies Co.	j8	John	john@123.net	1
10	The Neuralyzer	33.55	Silly Supplies Co.	p9	Katy	katy@123.net	2

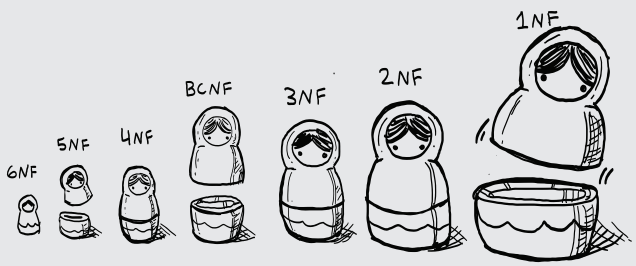
Normal forms: Crazy nesting dolls

You may wonder whether normal forms can go beyond BCNF. Yes! Fourth Normal Form (4NF), Fifth Normal Form (5NF), and even Sixth Normal Form (6NF) have been proposed and discussed in the theoretical framework of database normalization.

4NF, 5NF, and 6NF build on previous normal forms, and each form targets increasingly specific, less common design problems. 4NF addresses multivalued dependencies, for example, and 5NF eliminates redundancy caused by join dependencies that are not covered by 4NF. 6NF is largely theoretical.

4NF, 5NF, and 6NF are beyond the scope of this book. If you want a quick introduction to them, grab a textbook on databases (such as *Database System Concepts*, 7th ed., by A. Silberschatz, H. F. Korth, and S. Sudarshan [McGraw-Hill Education, 2019]) or make the following request of ChatGPT:

> Give me an introduction to 4NF, 5NF, and 6NF, using plain language and examples.



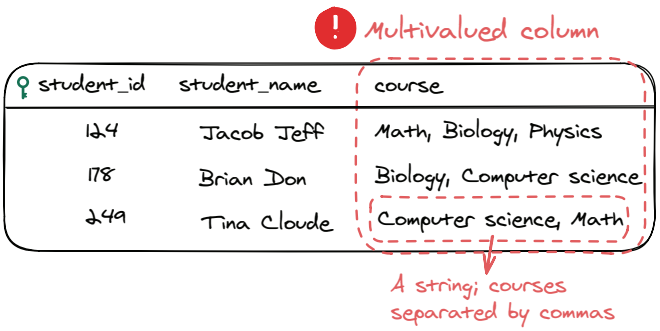
There is always a primary key

A table in BCNF should always have a primary key. Do you see primary keys everywhere in the entity-relationship (E-R) diagram you developed in chapter 5? Great! That means each table follows the first simple guideline of BCNF.

What if you have a table that doesn't have a primary key? Well, you need to stop and identify the primary key of that table before going any further.

There are no multivalued columns

A table in BCNF should have no multivalued columns. What is a multivalued column? Think about a table named `course_registration` that represents students taking a variety of courses. The `course` column is a multivalued column. Each course record holds multiple values.



The multivalued columns lead to problems such as difficult querying, data redundancy, inconsistency, and anomalies. You can easily spot a course repeated in different rows in `course_registration`, for example.

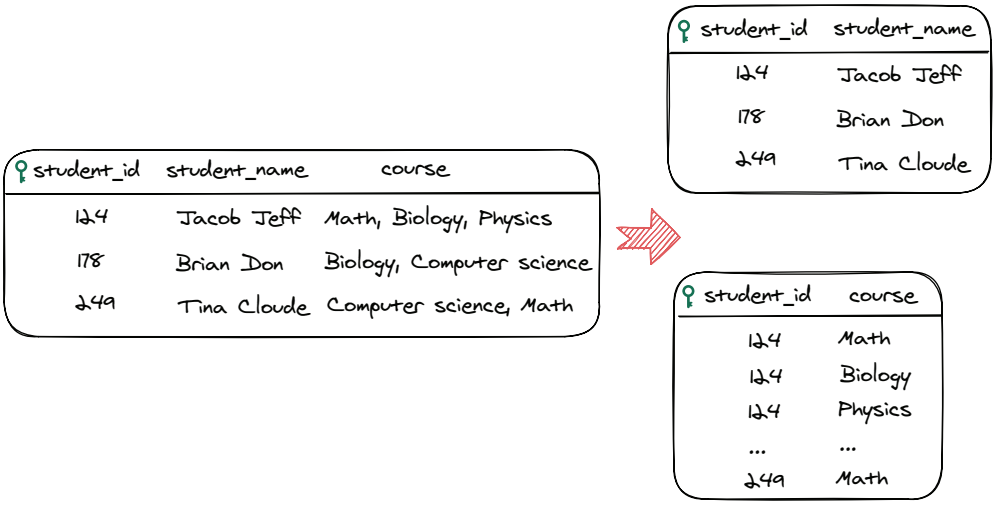
Who decides whether something is multivalued?

In a relational database, a table represents a single entity or a concept about which information is stored. A column in a table represents a specific attribute of the entity or concept that the table describes. Each column has a distinct name and a data type that defines the kind of data it stores. You may wonder who decides whether something should be considered a single concept, entity, or attribute. The answer is the users of the application supported by the database.

Determining whether a column is multivalued is based on the standards of users. In a table of an e-commerce database, for example, you would call out a column that represents phone numbers and stores multiple phone numbers per row as a multivalued column because each phone number represents a value that's undividable to users. Dividing a phone number into an area number and subscriber number makes no sense to any users of the database. As another example, you won't consider a column in the same database that stores single email addresses per row a multivalued column because users don't care about dividing an email address into a username and domain name.

Overall, user requirements decide the purposes of databases, which in turn decide whether a column is multivalued.

To fix such a problem, you typically need to redesign the multivalued column so that it holds only a single value in each row. Often, you need to move that multivalued column to a new table to prevent redundancy and other problems. To fix `course_registration`, for example, you limit the `course` column to hold a single course per row. But you can't repeat the same `student_id` values in different rows of the original table because it's the primary key. You need to break `course_registration` into two tables, one holding student information and the other holding course registration information:



Does your database design have you have any tables containing multivalued columns? Nope. When you mapped each entity to a table, you didn't attempt to use a column to hold more than one value. If you do, you will fix such problems in a similar manner.

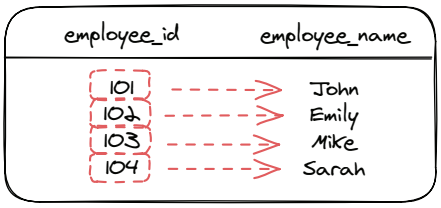
All columns are dependent on a key but nothing else

All columns of a table in BCNF should be functionally dependent on a key. To understand this requirement, you need to understand functional dependency.

Think about how functions work in any programming language. Suppose that you have a function, `power(x)`, that takes `x` as the only input and returns its power as the output. Given the same input `x`, the function `power(x)` always returns the same output. The input functionally determines the output.

Now let's switch our attention to a table. In a table, given a value of column A, if there is always a unique corresponding value of column B, column A functionally determines column B—that is, column B is functionally dependent on column A. In the following table, which represents employees, `employee_name` is functionally dependent on `employee_id`. In other words, knowing the value of `employee_id` can help you determine the value of `employee_name`. This functional dependency can be expressed as follows:

```
employee_id → employee_name
```



Functional dependency is directional. In the preceding figure, `employee_id` functionally determines `employee_name`. It doesn't mean, however, that `employee_name` functionally determines `employee_id`. The value of `employee_name` can't be used to determine the value of `employee_id`. Think what would happen if two different employees had the same name.

Now that you know what functional dependency is in the context of relational databases, look at this BCNF requirement again:

> A table in BCNF should have all its columns functionally dependent on a key but nothing else.

You may wonder why we say "dependent on a key" without specifying "primary key." There is a fine-grained difference, and edge cases exist. In most cases that you'll deal with in practice, however, you don't need to worry about it and can relax this requirement to the following:

> A table in BCNF should have all its columns functionally dependent on the primary key but nothing else.

Toward a deeper understanding of BCNF

A deep understanding of BCNF and all other normal forms is helpful in normalization (before they drive you crazy). Edge cases, which are good opportunities to deepen your understanding, are beyond the scope of this book, but it doesn't hurt to list them:

1. Table R has three columns, a, b, and c. The primary key of R is (a, b). If c is functionally dependent on a, R is not in 2NF, let alone 3NF or BCNF.
2. Table R has five columns, a, b, c, d, and e. The primary key of R is (a, b, c). d and e are functionally dependent on (a, b, c). If c is functionally dependent on d, R is in 3NF but not in BCNF.
3. Table R has five columns, a, b, c, d, and e. R has two candidate keys, (a, b) and (c, d). The primary key is (a, b). If e is functionally dependent only on (c, d), R is still in BCNF.

Before your head explodes, rest assured that you'll rarely need to deal with case 2 or 3.

In other words, if you find a table with one or more columns that are not functionally dependent on the primary key, the table must be normalized. How? You break the table into two or more tables, each table containing only the columns that are dependent on the primary. Depending on the relationship between the two new tables, you may link them via a foreign key.

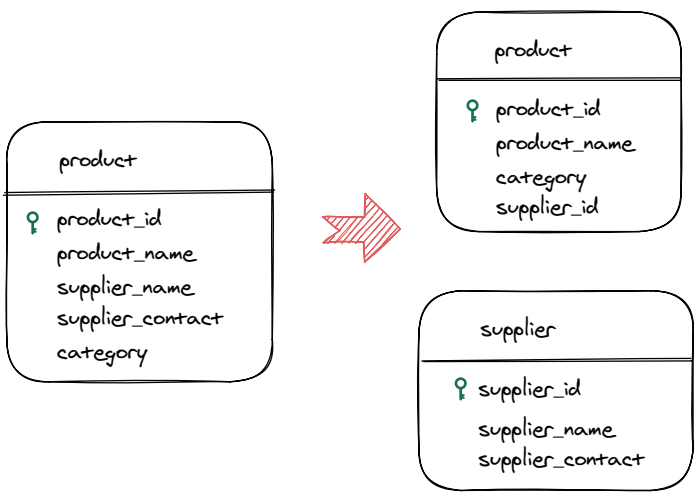
We'll demonstrate this process with an example. Imagine a `product` table designed by a novice designer:

product_id	product_name	supplier_name	supplier_contact	category
1224	Stardust	Brenlux	555-0123	kitchen
3378	Time Machine	TimeCo	555-0456	utility
2549	Magic Wand	Magico	555-0789	utility

It may seem logical to include supplier contact information alongside product details for convenience. But the column `supplier_contact` is functionally dependent only on `supplier_name`. In other words, the `product` table contains two functional dependencies:

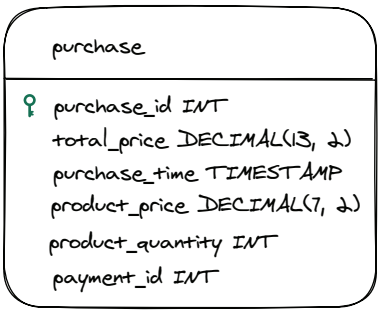
```
product_id → product_name, supplier_name, category
supplier_name → supplier_contact
```

To fix this problem, you need to remove the columns that are not determined by the primary key of the `product` table. Where do they fit in? A new table. The `supplier_contact` column is functionally dependent on `supplier_name`. If the two columns are in one table that represents suppliers, they fully meet the requirement of functional dependency. If each supplier can be identified by a unique ID, you can even use this piece of information as the primary key of this new table, and it will naturally become the foreign key in the `product` table that links `product` and `supplier`:

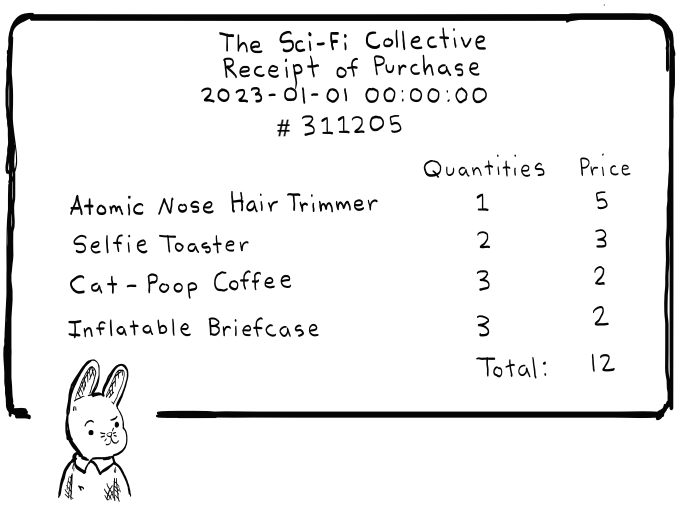


Normalize your database design

When you check the design of The Sci-Fi Collective's database against the requirement that all columns be functionally dependent on the primary key, you need to examine every table, including `purchase`:



Undoubtedly, many columns are functionally dependent on the primary key, including `total_price`, `purchase_time`, `payment_id`, and `email`. If you think about what goes into a receipt, you see that knowing the value of `purchase_id` is equal to having a receipt, which determines the values of `total_price` and `purchase_time`:

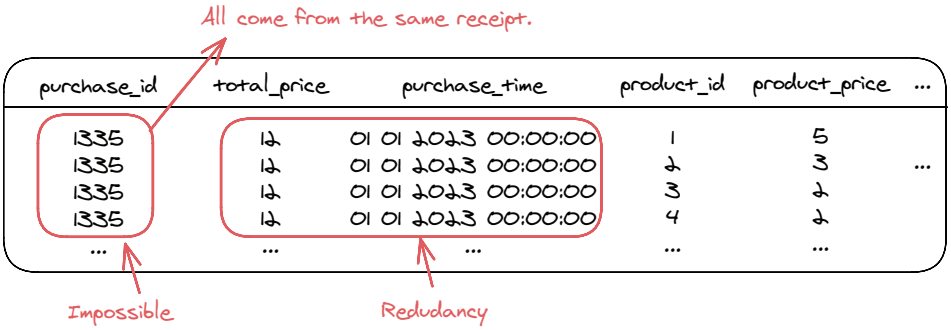


From this example receipt, you can tell that each purchase involves multiple products with different quantities and prices. Product quantities (`product_quantity`) and prices (`product_price`) are functionally determined by a combination of purchase ID (`purchase_id`) and product code (`code`). The product code (`code`) isn't in the `purchase` table. That said, you have two functional dependencies in this table:

```
product_id → total_price, purchase_time, payment_id, email
purchase_id, code → product_quantity, product_price
```

Because of the two functional dependencies, this table violates BCNF.

When you start adding data to the `purchase` table, you see how tricky the problem is. Adding the data of a receipt to the `purchase` table, for example, is a mission impossible:



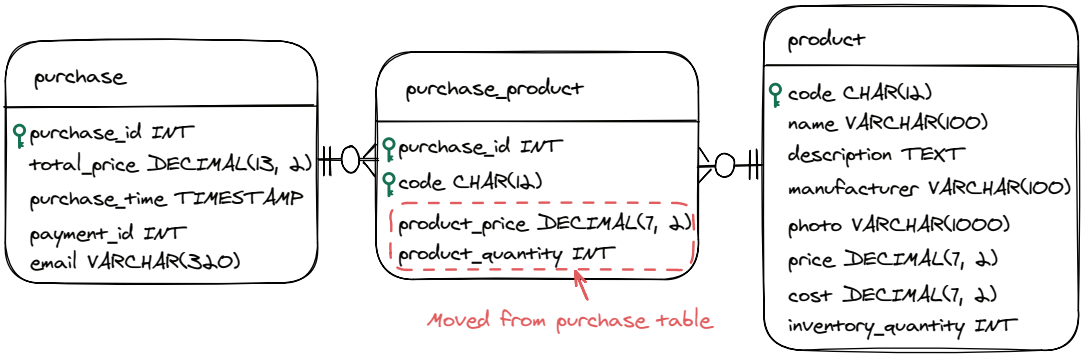
You can't repeat the values of the primary key. Even if you could, you would still face redundancy problems in other columns, such as `total_price` and `purchase_time`.

To fix this problem, move the columns that are not functionally dependent on the primary key to a new table. In this case, `product_quantity` and `product_price` are functionally dependent on the combination of `purchase_id` and `code`. But neither `purchase_id` nor `code` can functionally determine `product_quantity` or `product_price` alone:

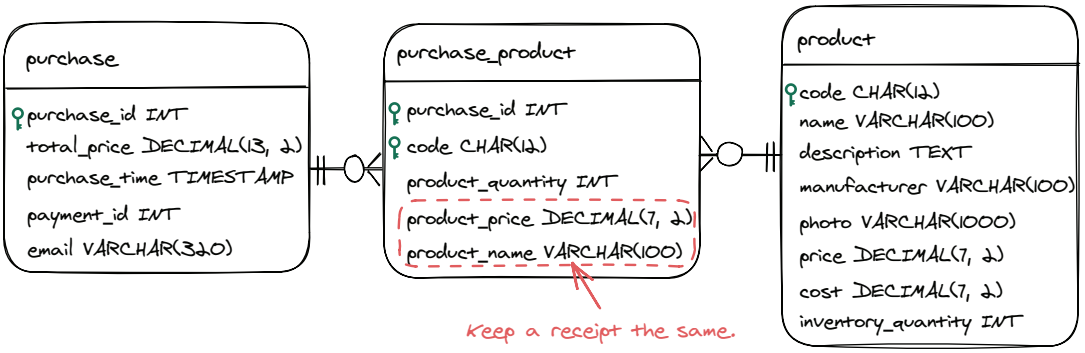
- `purchase_id` does not functionally determine `product_quantity` or `product_price`. Each purchase can include multiple products with different quantities and prices.

- `code` does not functionally determine `product_quantity` or `product_price` in the context of a purchase. The same product can be sold in different quantities and at different prices in various purchases.

Luckily, you already have a junction table, `purchase_product`, that connects `purchase` and `product`. The junction table uses the combo of `purchase_id` and `code` as its primary key. Moving `product_quantity` and `product_price` to the `purchase_product` table is like killing two birds with one stone. With this change, both `purchase` and `purchase_product` meet the requirement of functional dependency:

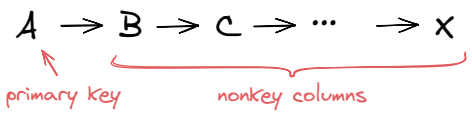


You may be tempted to eliminate `product_price` from the `purchase_product` table because you can always retrieve a product's price from the `product` table. But it's best not to do that. Over time, the price of a product may change due to factors such as inflation or market competition. If you rely on only the `product` table to retrieve product prices, you may lose the data required to put together a receipt from several months ago. For the same reason, you should add a product name column to the `purchase_product` table. After all, nothing should lead to changes in a receipt—not even product name changes.

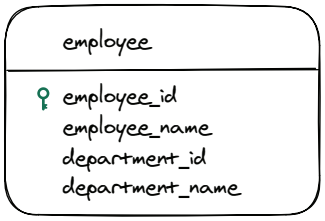


There is no transitive dependency

A table in 3NF should have no transitive dependency. *Transitive dependency* occurs when something depends on something else that depends on yet another thing; it's like a chain of things that need one another to work. In the context of database design, transitive dependency means that a nonkey column is functionally dependent on another nonkey column, which in turn is functionally dependent on a key of the table. The core concept of a transitive dependency is that one nonkey column's value depends on another through a chain of dependencies, which is ultimately dependent on a key.



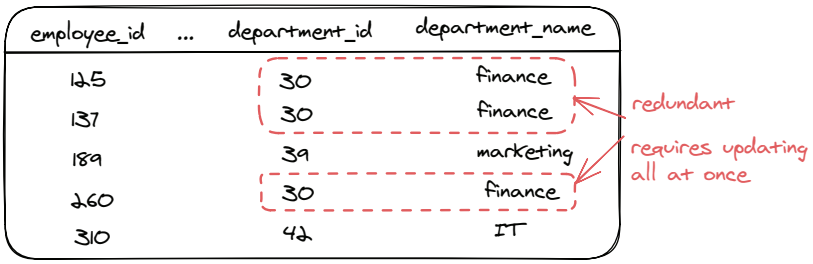
A table representing employees has the following columns and functional dependencies:



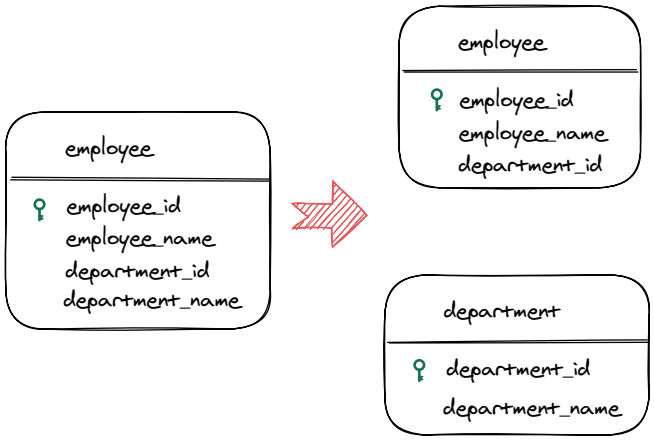
employee_id → department_id → department_name

The primary key of the `employee` table is `employee_id`. In this table, `department_name` is dependent on `department_id`, and `department_id` is dependent on `employee_id`. This chained functional dependency is transitive dependency.

Transitive dependency is problematic. If a table contains transitive dependency, it leads to all kinds of problems, such as data redundancy and insertion/update/deletion anomalies. Updating a department name in the preceding `employee table` properly means updating all the `department_name` values in every row:



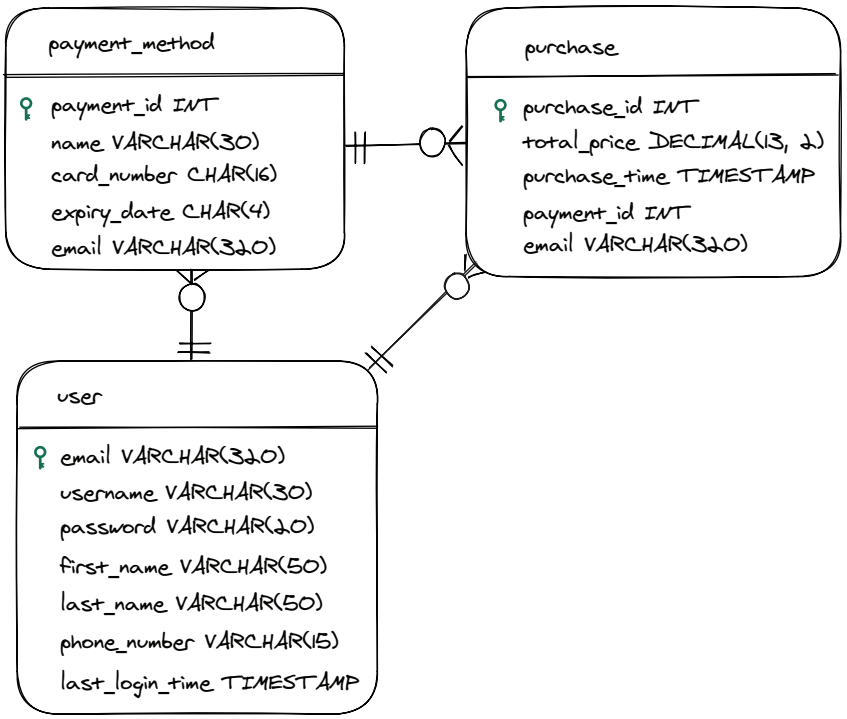
What should you do with a table containing transitive dependency? Break the table into two or more tables, each table containing columns that are directly dependent on a key and dependent only on that key. Depending on the relationship between the two new tables, you may link them via a foreign key. If you stick to this principle, break the `employee` table into two tables as follows:



The `department` table contains the nonkey columns that are involved in the transitive dependency, with the `department_id` as its primary key. The new `employee` table contains only the columns that are directly dependent on `employee_id` and dependent only on it, including the foreign key `department_id`. A foreign key is always directly dependent on the primary key because each row has a unique combination of the primary key and foreign key.

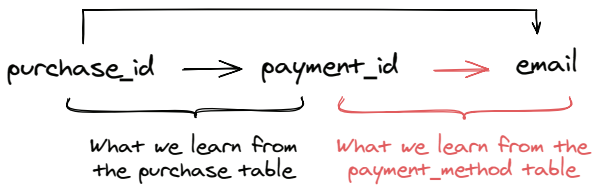
Normalize your database design: A cycle involving three tables

A transitive dependency may be hard to spot without a deep understanding of the data, requirement analysis, and some sample data. Your `purchase` table, for example, uses two foreign keys, `email` and `payment_id`, to maintain its relationships with the `payment_method` and `user` tables:



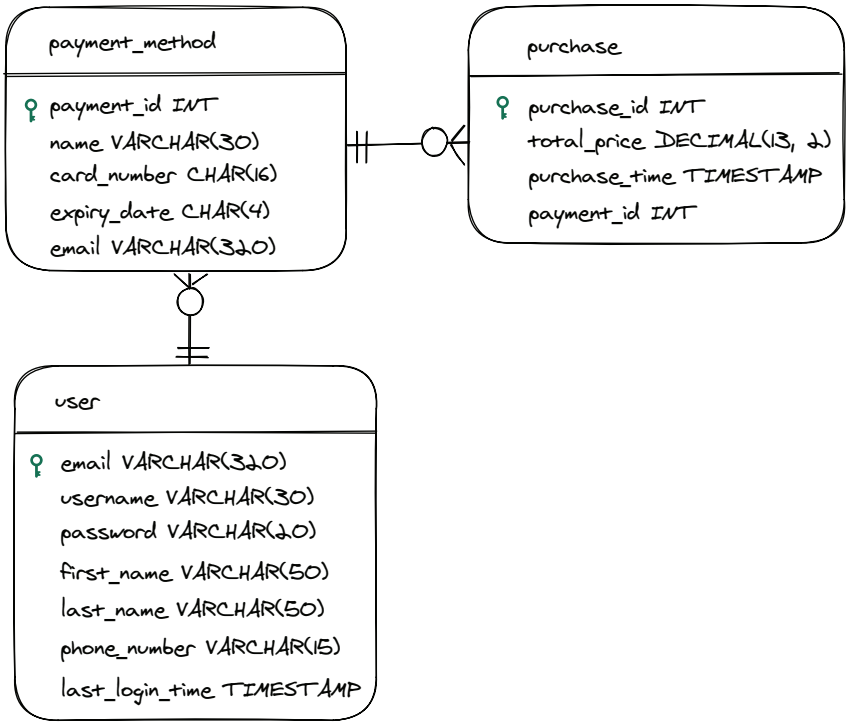
Without question, in the `purchase` table, all other non-primary-key columns are dependent on the primary key, including the foreign keys.

If you look beyond the `purchase` table, however, you notice something unexpected: the two foreign keys in the `purchase` table, `payment_id` and `email`, have a dependency relationship. From the `payment_method` table, you see that `email` is a foreign key that helps maintain the relationship between `payment_method` and `user`. As in the `purchase` table, the foreign key (`email`) is dependent on the primary key (`payment_method`). When you consider this new piece of information, you see a transitive dependency in `purchase`:



In theory, you need only two relationships to connect three tables. If you notice that you are using three relationships and that the relationships are starting to look like a cycle in your E-R diagram, you may have a transitive dependency somewhere.

Now that you have identified the transitive dependency, how do you fix it? The principle is the same a table should contain only columns that are directly dependent on the primary key and dependent only on it. You can remove the `email` column from the `purchase` table to break the direct relationship between the `user` and `purchase` tables, but `user` is still related to `purchase` via `payment_method`, as shown in the following figure.

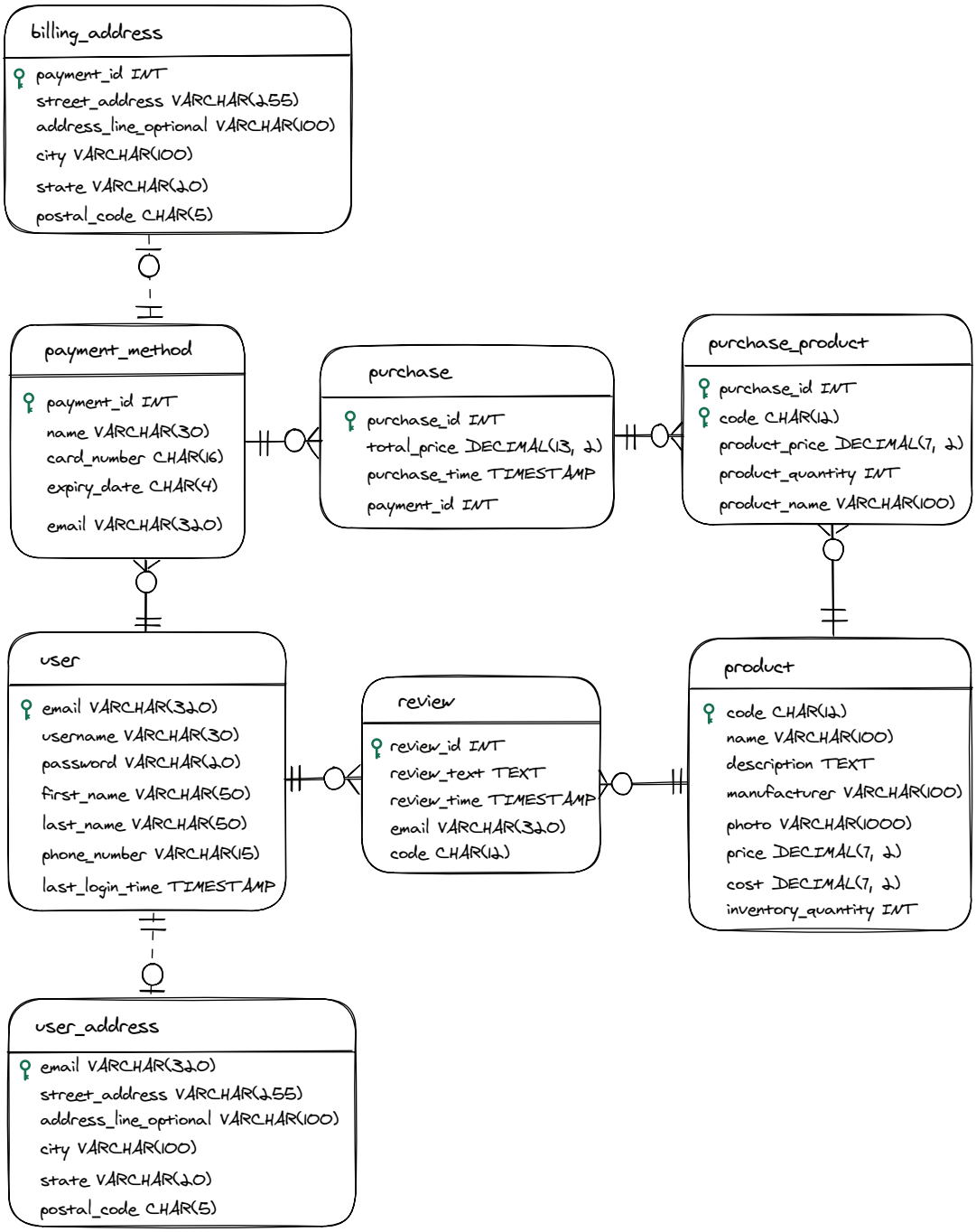


If normalization is your only concern, removing the link between the `user` and `purchase` tables is a brilliant move. But it may not be the best move if you are also concerned with other problems, such as query speed and cost. We will revisit the relationship between the `user` and `purchase` tables in chapter 7.

When you go through all other tables in your database design, you won't see other transitive dependency problems. That said, you have completed all the steps to check each table against the 3NF requirements:

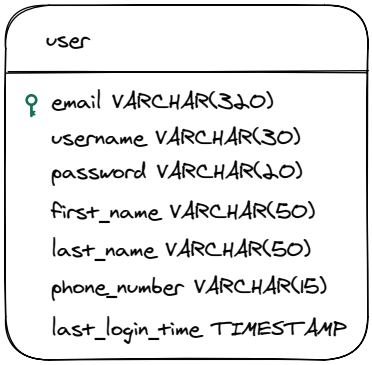
- It has a primary key.
- It has no multivalued columns.
- All columns are dependent on a key but nothing else.
- It contains no transitive dependency. (All its nonkey columns are directly dependent on a key.)

Your updated E-R diagram looks like this:



Implementation

When you finish normalization, you are ready to implement your database design. You learned how to create tables via SQL in chapters 1 and 2. With this knowledge, you may feel that you can translate your database design into SQL with little to no effort. Your `user` table looks like this in your E-R diagram:



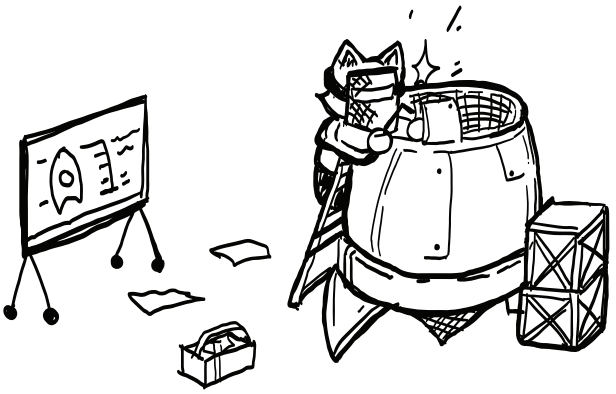
Translating this design directly to SQL leads to the following code snippet:

```
-- comment: works for MySQL and MariaDB
-- comment: see the code repo for other RDBMS
CREATE TABLE user (
  email VARCHAR(320) PRIMARY KEY,
  username VARCHAR(30),
  password VARCHAR(20),
  first_name VARCHAR(50),
  last_name VARCHAR(50),
  phone_number VARCHAR(15),
  last_login_time TIMESTAMP
);
```

Is the job done? Nope. To implement a database design successfully, you need to learn more about constraints and use that knowledge to make decisions beyond the E-R diagram.

You saw the use of constraints in chapter 2 but haven't been formally introduced to them. *Constraints* are SQL rules applied to columns in a table to ensure data accuracy and reliability. Constraints are used

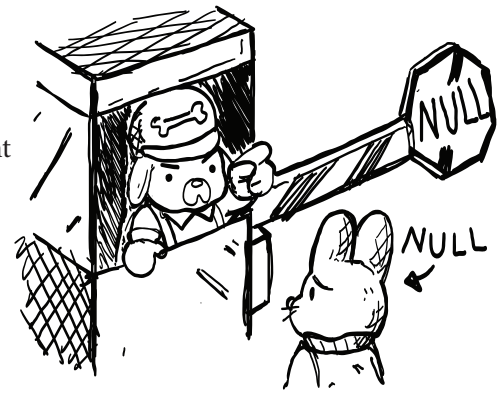
to enforce data integrity and can be established during or after table creation. Are you worried that software developers may not follow your database design when they develop the APIs? When you apply constraints to your tables, they have to listen! Without constraints, a database is like a toddler on a sugar high: chaotic and prone to causing havoc.



In the following sections, you learn about common SQL constraints and other design decisions that you need to make. We don't strive to cover all the constraints or SQL code required to implement the database. If you want to see the full SQL script, refer to the GitHub repository (https://mng.bz/4ao5).

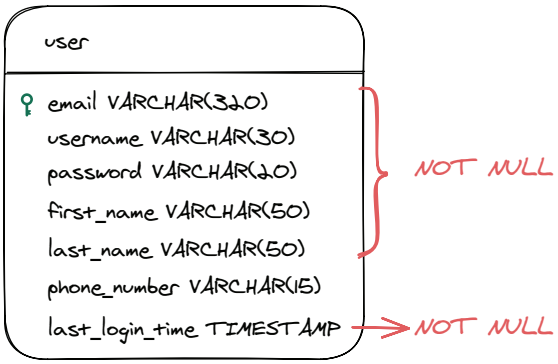
NOT NULL: Can't have null-thing to say

The `NOT NULL` constraint ensures that a column does not accept `NULL` values in SQL. In other words, when the `NOT NULL` constraint is applied to a column, you have to provide a value for that column. The `NOT NULL` constraint can be handy for preventing problems in data storage and analytics. As we stated in chapter 1, a `NULL` value in SQL represents an unknown value. Allowing `NULL` values for columns may lead to unexpected behaviors in SQL.



When do you need help from NOT NULL constraints? In most cases. Although their use depends on the requirements and the data, NOT NULL constraints are commonly applied in a wide range of scenarios. Columns that contain critical information for business operations (such as usernames, email addresses, or passwords) often have NOT NULL constraints. As another example, legal or compliance reasons force some columns to use NOT NULL constraints, such as date of birth in an option-trading application. Also, when two tables are related, the foreign keys may need NOT NULL constraints, depending on the nature of the relationship. You learn more about how NOT NULL constraints are used with foreign keys later in this chapter.

Do you have any columns that require NOT NULL constraints in the database design of The Sci-Fi Collective? Yes. Many columns in your database design contain critical data necessary for business operations. Six columns in the user table fall into this basket: username, email, password, first_name, last_name, and last_login_time.



As a result, you'll add NOT NULL constrains to these columns. You can do so by adding NOT NULL in the same code lines that define columns:

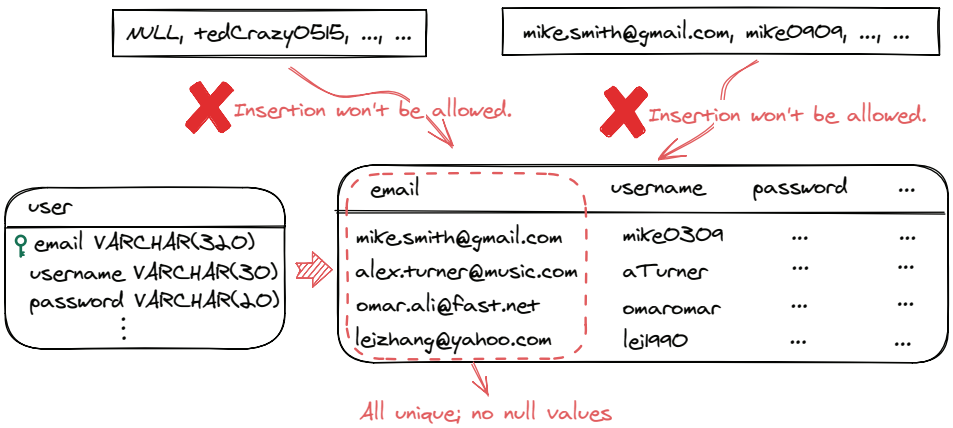
```
-- comment: works for MySQL, MariaDB, and SQLite
-- comment: see the code repo for other RDBMS
CREATE TABLE user (
  email VARCHAR(320) PRIMARY KEY,
  username VARCHAR(30) NOT NULL,
  password VARCHAR(20) NOT NULL,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  phone_number VARCHAR(15),
  last_login_time TIMESTAMP NOT NULL
);
```

You may notice that the primary key, `email`, doesn't have the `NOT NULL` constraint. The reason is that it already has the `PRIMARY KEY` constraint, which implies a `NOT NULL` constraint and enforces it automatically. You learn more about the `PRIMARY KEY` constraint in the following section.

Primary key: The one and only

As you already know, in a solid database design, every table has a primary key. When such a design is translated to SQL, it is expressed as the primary key constraint.

The primary key constraint ensures that no duplicate rows with the same primary key exist in the same table. In addition, the primary key constraint guarantees that no row with a `NULL` value in the primary key column can be inserted into the table.



The syntax of a primary key constraint is simple. You can add `PRIMARY KEY` to the code line that defines the column:

```
-- comment: works for MySQL, MariaDB and PostgreSQL
-- comment: see the code repo for other RDBMS
CREATE TABLE user (
  email VARCHAR(320) PRIMARY KEY,
  username VARCHAR(30) NOT NULL,
  password VARCHAR(20) NOT NULL,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  phone_number VARCHAR(15),
  last_login_time TIMESTAMP NOT NULL
);
```

If your database design is ongoing and there's a chance that the primary key may change, however, you want to name the primary key constraint explicitly:

```
-- comment: works for MySQL, MariaDB and PostgreSQL
-- comment: see the code repo for other RDBMS
CREATE TABLE user (
  email VARCHAR(320),
  username VARCHAR(30) NOT NULL,
  password VARCHAR(20) NOT NULL,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  phone_number VARCHAR(15),
  last_login_time TIMESTAMP NOT NULL,
  CONSTRAINT pk_user PRIMARY KEY (email)
);
```

The `CONSTRAINT...` clause allows you to name this constraint. The `PRIMARY KEY...` statement creates the primary key constraint.

Naming constraints: A best practice

Naming constraints in SQL is not strictly required but is considered a best practice for the following reasons:

- Named constraints communicate their purpose clearly, which improves the readability of your database schema and makes it easier for anyone who interacts with the database to understand the role of each constraint.
- When a constraint violation occurs, error messages include the name of the constraint, which makes the messages more informative and allows quicker identification of problems.
- If you don't name a constraint, the SQL system will generate a name automatically. Autogenerated names are often meaningless and vary from one RDBMS to another, which may lead to confusion, especially when you're porting the database schema to a different system.

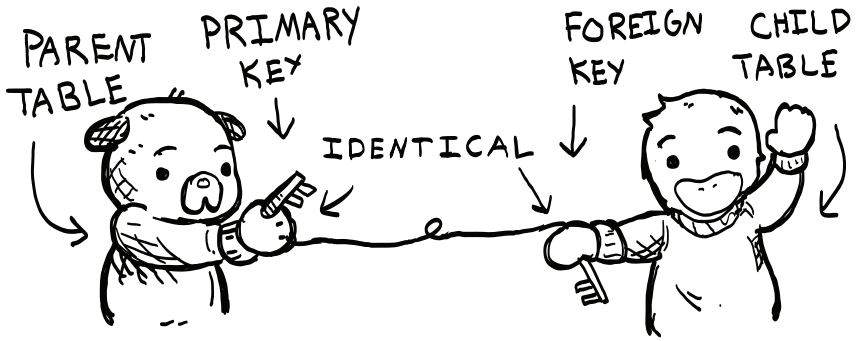
Certain constraints, such as `NOT NULL` and `DEFAULT`, are not always named. These constraints usually are integral parts of column definitions and could rarely be referenced separately; keeping them as part of column definition may make SQL scripts easier to write and simpler to read. In the case of other constraints, such as primary key and foreign key, if there is even a small chance that you'll need to restructure or migrate the database, you had better keep them named.

What if you have a composite primary key that is composed of more than one column? You can define the primary key separately from any individual column definition. In the E-R diagram of The Sci-Fi Collective, the `purchase_product` table uses a composite primary key composed of two columns, `purchase_id` and `code`. You can define this primary key separately from individual column definitions as follows:

```
-- comment: works for MySQL, MariaDB, and PostgreSQL
-- comment: see the code repo for other RDBMS
CREATE TABLE purchase_product (
  purchase_id INT NOT NULL,
  code CHAR(12) NOT NULL,
  product_price DECIMAL(7,2) NOT NULL,
  product_quantity INT NOT NULL,
  product_name VARCHAR(100) NOT NULL,
  CONSTRAINT pk_purchase_product
     PRIMARY KEY (purchase_id, code)
);
```

Foreign key: Playing Cupid

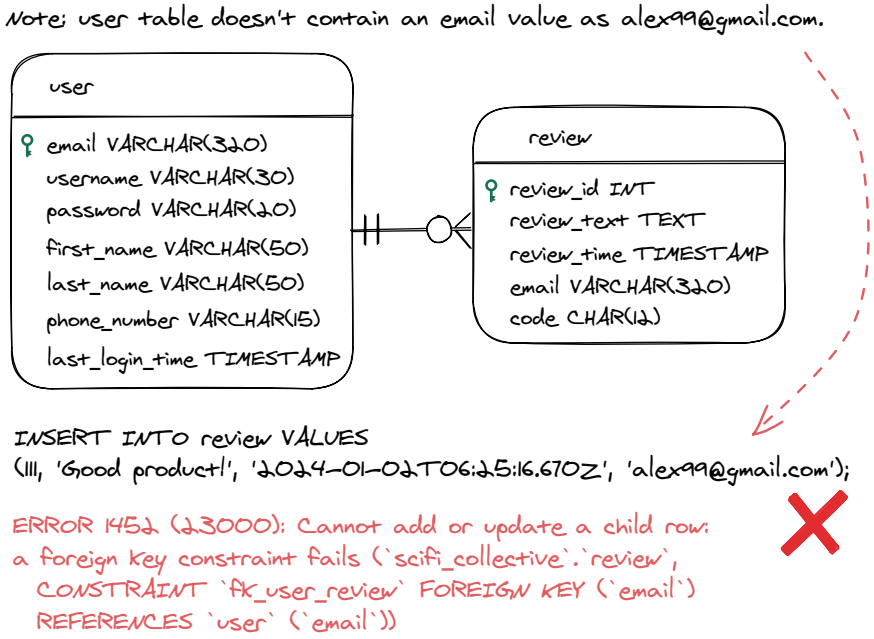
When two tables have a relationship, you use a foreign key to link them. As you know, a *foreign key* is at least one column in a table that refers to the primary key in another table. When a foreign key is translated to SQL, it is typically expressed as the foreign key constraint.



The foreign key constraint enforces referential integrity. What is referential integrity? Given a relationship between two tables, one table contains the foreign key that references the primary key of the other table. From chapter 2, you know that the table containing the foreign

key is called a *child table*, and the other table is called a *parent table*. If you attempt to add a row to the child table but your foreign key value doesn't exist yet in the parent table, SQL would stop you and complain.

You have a one-to-many relationship between the `user` and `review` tables in the E-R diagram of The Sci-Fi Collective, for example. In the `review` table, `email` is the foreign key that links `review` to `user`. If the foreign key constraint is implemented properly, SQL should stop you if you try to add a `review` record with an `email` value that doesn't exist in `user` yet.



Implementing a foreign key constraint is similar to implementing a primary key constraint. Your `review` table, for example, contains two foreign keys that reference the `user` and `product` tables. You can define and name the foreign key constraints when creating the `review` table:

```
-- comment: works for MySQL and MariaDB
-- comment: see the code repo for other RDBMS
CREATE TABLE review (
  review_id INT PRIMARY KEY,
  review_text TEXT NOT NULL,
```

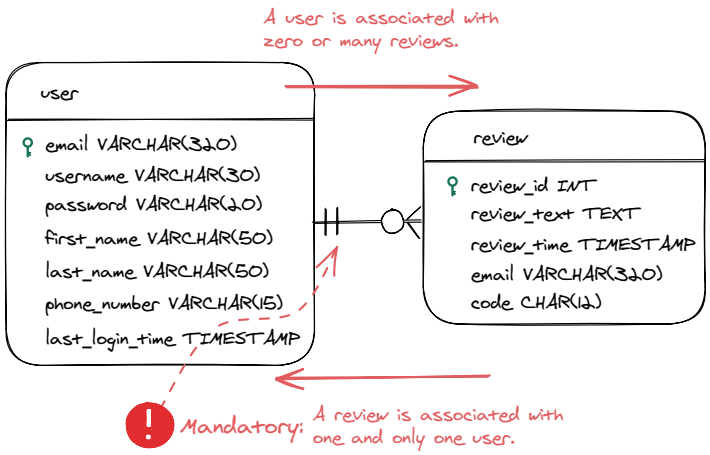
```
    review_time TIMESTAMP NOT NULL,
    email VARCHAR(320) NOT NULL,
    code CHAR(12) NOT NULL,
    CONSTRAINT fk_user_review
      FOREIGN KEY (email) REFERENCES user(email),
    CONSTRAINT fk_product_review
      FOREIGN KEY (code) REFERENCES product(code)
  );
```

Or you can define the foreign key constraints after the `review` table is created:

```
-- comment: works for MySQL, MariaDB and PostgreSQL
-- comment: SQLite doesn't support
-- comment: ALTER TABLE ADD CONSTRAINT
-- step 1: create the review table first
CREATE TABLE review (
  review_id INT PRIMARY KEY,
  review_text TEXT NOT NULL,
  review_time TIMESTAMP NOT NULL,
  email VARCHAR(320) NOT NULL,
  code CHAR(12) NOT NULL
);
-- step 2: add foreign key constraints afterward
ALTER TABLE review
  ADD CONSTRAINT fk_user_review
    FOREIGN KEY (email) REFERENCES user(email),
  ADD CONSTRAINT fk_product_review
    FOREIGN KEY (code) REFERENCES product(code)
```

In the same way that a primary key constraint is defined and named, the `CONSTRAINT…` clause names a constraint, and the `FOREIGN KEY… REFERENCES…` statement creates a foreign key constraint that enforces referential integrity.

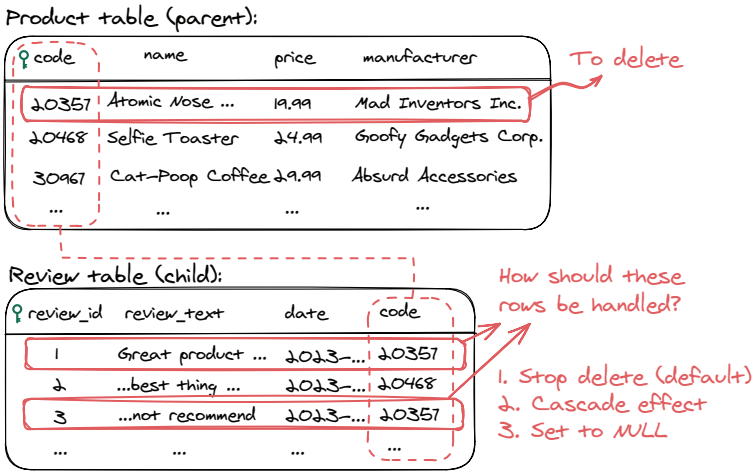
It is worth noting that when a relationship is mandatory, the foreign key typically requires help from the `NOT NULL` constraint. When the minimum cardinality of a directional relationship is one, that directional relationship is considered mandatory. The relationship from `review` to `user` in your database design, for example, is mandatory: the min cardinality is one, which means that a review is associated with at least one user. If you have a `review` record that doesn't reference any users in the `user` table, this row can't be added to the `review` table. That said, allowing `email` in `review` to accept `NULL` values makes no sense because a `NULL` value of `email` doesn't reference any users.



Referential actions

In the preceding section, you learned that you can use foreign key constraints to enforce one aspect of referential integrity: data insertion. When someone tries to insert a row into a child table, but the foreign key value in that row doesn't reference anything in the parent table, the foreign key constraint will stop the operation and raise a complaint.

Beyond data insertion, referential integrity cares about data deletion and updating. When a row is removed from the parent table, how should the corresponding rows in the child be handled? When a row's primary key value is updated in the parent table, what should happen to the corresponding rows in the child table?



You can handle such problems by respecting the default action, which prevents you from updating or deleting a row in the parent table if there are matching rows in the child table. To respect the default actions, either do nothing or expand the foreign key constraint definitions with the following statements:

```
ON DELETE RESTRICT
ON UPDATE RESTRICT
```

The `ON DELETE` line defines how to handle the corresponding rows in the child table when a row is deleted in the parent table. The `ON UPDATE` line defines how to handle the corresponding rows in the child table when a row's primary key is updated in the parent table. You can also swap `RESTRICT` with `NO ACTION`, which has the same effect. These clauses are known as *referential actions*, which are triggered automatically when you update primary key values or delete rows in the parent table.

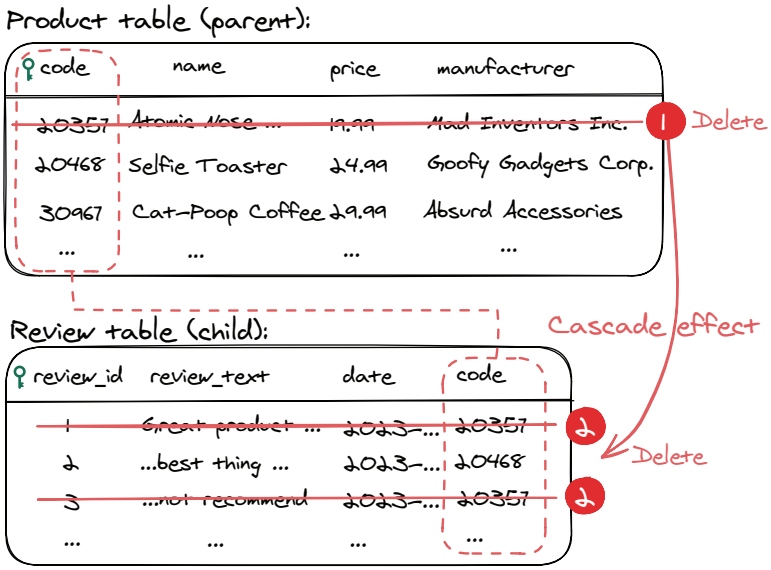
The default action, `RESTRICT` (or `NO ACTION`) maintains the integrity and consistency of the database by preventing you from deleting or updating a parent table. But what if you need to delete some data or update a primary key value in a parent table? To maintain the integrity and consistency of the database, use a different type of referential action, called `CASCADE`:

- `CASCADE` *delete*—If a row in the parent table is deleted, the corresponding rows in the child table are automatically be deleted if possible. This action ensures that no rows in the child table lack corresponding rows in the parent table.
- `CASCADE` *update*—If a primary key in the parent table changes, the corresponding foreign keys in the child table automatically update with the new key value. This action ensures that the link between the two tables is maintained without manual update operations.

The implementation of a `CASCADE` action is simple. You expand your foreign key constraint definitions a bit to include two different statements:

```
ON DELETE CASCADE
ON UPDATE CASCADE
```

You see the `CASCADE` effect in the child table whenever a row is deleted or a row's primary key is updated in the parent table.

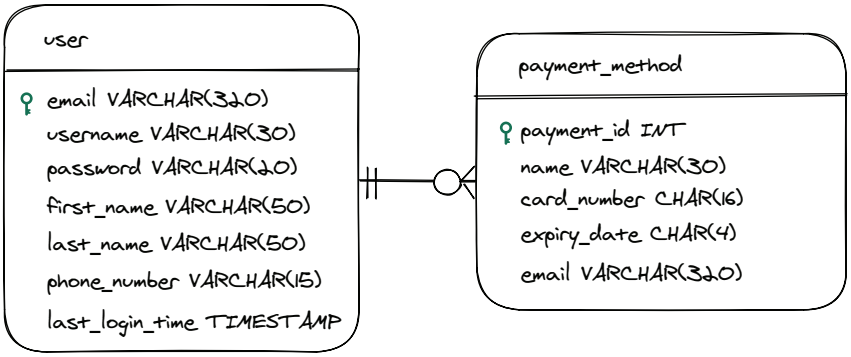


In a less common scenario, you may want to support the delete operation in the parent table but preserve the corresponding rows in the child table. A database that tracks company assets might have two related tables, `device` and `employee`. If an employee leaves the company and is removed from `employee`, it may be beneficial to retain the corresponding rows in `device` because those devices still exist in the company's inventory. In such a scenario, you can use the help of another type of referential action, `SET NULL`:

```
ON DELETE SET NULL
ON UPDATE CASCADE
```

With the help of `SET NULL`, all corresponding foreign keys in the child table are converted to `NULL`, and data in other columns is retained when a row is deleted in the parent table. The `SET NULL` action can be handy when a relationship between two tables is nonessential or when you need to retain the child rows for recordkeeping. But this action may introduce problems, such as creating orphan rows or complicating queries that expect non-null foreign keys down the road.

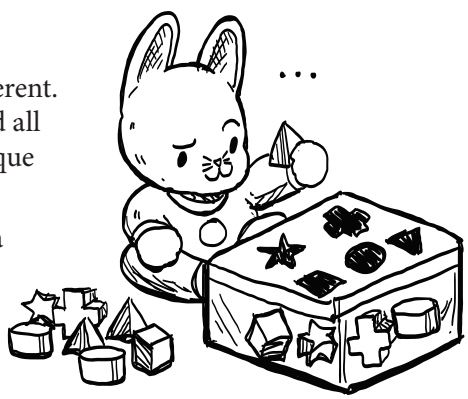
Do you have any relationships that need help from explicit referential actions in the database design of The Sci-Fi Collective? Yes. The `user` table, for example, is a parent table of other tables, including `payment_method`. The `email` column is the primary key of the `user` table.



If The Sci-Fi Collective ever allows users to change their email addresses or delete their accounts, many possible updates and deletions may occur in the parent table. To maintain the integrity and consistency of the database, you can opt for the `CASCADE` action, which is part of the foreign key constraint in the `payment_method` table:

```
-- comment: works for MySQL, MariaDB and PostgreSQL
-- comment: see the code repo for other RDBMS
CREATE TABLE payment_method (
  payment_id INT PRIMARY KEY,
  name VARCHAR(30) NOT NULL,
  card_number CHAR(16) NOT NULL,
  expiry_date CHAR(4) NOT NULL,
  email VARCHAR(320) NOT NULL,
  CONSTRAINT fk_payment_method_user
    FOREIGN KEY (email) REFERENCES user (email)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);
```

Unique: Sorry, I'm taken

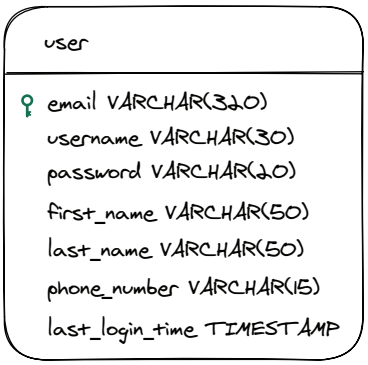


Unique constraints ensure that all values in a column or a combination of columns are different. If you have some nonkey columns that should all contain unique values, you need to apply unique constraints.

You need unique constraints when you have a single column or a combination of nonkey columns for which duplicate values make no sense in the real world. Consider a table containing a column that represents Social

Security numbers (SSN), which are unique nine-digit identifiers issued by the U.S. government to track citizens' earnings, taxes, and eligibility for benefits. This column is a nonkey column. Each row in this table represents a person, and it makes no sense for different people to have the same SSN. In a scenario like this one, you need to use unique constraints.

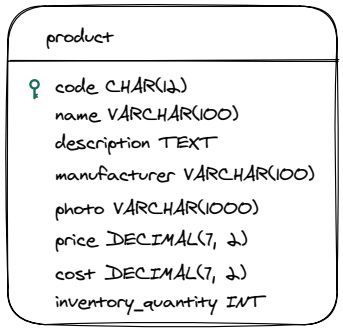
Your database design contains some nonkey columns that require unique constraints. To identify these columns, rely on your understanding of the requirements analysis. Your `user` table looks like this:



The requirements analysis tells you that both the `username` and `phone_number` columns need to hold unique values. Both columns are be nonkey columns. As a result, you should add unique constraints to these two columns and name them accordingly:

```
-- comment: works for MySQL, MariaDB and PostgreSQL
-- comment: see the code repo for other RDBMS
CREATE TABLE user (
  email VARCHAR(320) PRIMARY KEY,
  username VARCHAR(30) NOT NULL,
  password VARCHAR(20) NOT NULL,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  phone_number VARCHAR(15),
  last_login_name TIMESTAMP NOT NULL,
  CONSTRAINT unq_username UNIQUE(username),
  CONSTRAINT unq_phone_number UNIQUE(phone_number)
);
```

Sometimes, you need to apply the unique constraint to a combination of columns. Consider your `product` table:



If your requirements analysis says you need to ensure the uniqueness of the combination of two pieces of information, the name and manufacturer of a product, you need to apply the unique constraint to a combination of the two columns:

```
-- comment: works for MySQL, MariaDB and PostgreSQL
-- comment: see the code repo for other RDBMS
CREATE TABLE product (
  code CHAR(12) PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  description TEXT NOT NULL,
  manufacturer VARCHAR(100) NOT NULL,
  photo VARCHAR(1000) NOT NULL,
  price DECIMAL(7,2) NOT NULL,
  cost DECIMAL(7,2) NOT NULL,
  inventory_quantity INT,
  CONSTRAINT unq_name_manufacturer
    UNIQUE(name, manufacturer)
);
```

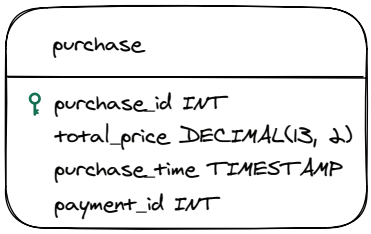
Default to awesome

You use the default constraint to set a default value for a column when a new row is inserted into the table and no value is provided for that column. The default constraint ensures that a column always has a value, specified by either the user or the constraint.

You typically use the default constraint in the following situations:

- You have a column that stores timestamps, such as a column representing when a row is created or modified.
- You have a numeric column on which you might perform aggregation, such as a column representing the quantity or price.
- You have a column that is optional and can use the help of a placeholder value.

Your database design has columns that could use default constraints. Your `purchase` table, for example, has a `purchase_time` column that stores timestamps:



When a purchase is made online, a row of data is supposed to be inserted into the `purchase` table. Instead of relying solely on the software developers to do the right thing, you can lessen their burden by setting the default value of `purchase_time` to the time when a row of purchase data is added. The syntax of the default constraint is simple. Add the `DEFAULT` value to the same line of your column definition, and make sure to provide the proper target value:

```
CREATE TABLE purchase (
  purchase_id INT PRIMARY KEY,
  total_price DECIMAL(13,2) NOT NULL,
  purchase_time TIMESTAMP NOT NULL
  DEFAULT CURRENT_TIMESTAMP,
  payment_id INT NOT NULL,
  CONSTRAINT fk_payment_method_purchase
    FOREIGN KEY (payment_id)
    REFERENCES payment_method(payment_id)
);
```

Two things in the preceding SQL snippet require explanation:

- Both `NOT NULL` and default constraints were added to the `purchase_time` column. You may wonder whether the `NOT NULL` constraint is still necessary, considering that the default value is already set. Well, it is useful, especially if `NULL` values are problematic for that column. The default constraint, after all, doesn't prevent users from adding `NULL` values.
- The default value was set to `CURRENT_TIMESTAMP`, which is a widely supported function in most RDBMSs, including MySQL, MariaDB, SQLite, and PostgreSQL. `CURRENT_TIMESTAMP` is commonly used to obtain the current date and time.

Clocking databases: Navigating UTC across RDBMSs

In chapter 4, we recommended storing date and time values in Coordinated Universal Time (UTC) because UTC ensures consistency and prevents problems such as Daylight Saving Time changes and different time zones.

Does `CURRENT_TIMESTAMP` adopt UTC automatically? The answer depends on the RDBMS you use. `CURRENT_TIMESTAMP` achieves what you want in MySQL, for example, because the temporal data in MySQL is stored in UTC by default. By contrast, `TIMESTAMP` comes in two flavors in PostgreSQL. To get what you want in PostgreSQL, you need to specify the data type of `purchase_time` as `TIMESTAMP WITH TIME ZONE`. When you do, `CURRENT_TIMESTAMP` automatically stores temporal data in UTC:

```
-- PostgreSQL
CREATE TABLE purchase (
  purchase_id INT PRIMARY KEY,
  total_price DECIMAL(13,2) NOT NULL,
  purchase_time TIMESTAMP
      WITH TIME ZONE NOT NULL
      DEFAULT CURRENT_TIMESTAMP,
  payment_id INT NOT NULL,
  CONSTRAINT fk_payment_method_purchase
      FOREIGN KEY (payment_id)
         REFERENCES payment_method(payment_id)
);
```

The `TIMESTAMP` data types also come in two flavors in PostgreSQL:

- `TIMESTAMP WITHOUT TIME ZONE` (`TIMESTAMP` is the same)
- `TIMESTAMP WITH TIME ZONE`

To enable time zones, you declare the data type of the target column as `TIMESTAMP WITH TIME ZONE`. Otherwise, the data that you store in that column will carry no time-zone information. This is one thing you want to double-check in the manual of the RDBMS that you use. Or ask ChatGPT when you're about to use some RDBMS that's new to you:

- How do I create a column representing the current time and store data in that column in UTC in SQLite?
- How do I create a column representing the current time and store data in that column in UTC in SQL Server?

Check: Enforce data decorum

A *check constraint* is a rule that specifies a condition each row must meet for the data to be considered valid in a column. You can use a check constraint to enforce data integrity by restricting the values that can be inserted into columns.

You need a check constraint when you want to enforce specific rules on the data, such as enforcing data ranges or validating data formats. If you have a column representing the ages of people, for example, you can use a check constraint to limit the table to accept only values between 0 and 120:

```
-- comment: works for MySQL, MariaDB and PostgreSQL
CREATE TABLE person (
  person_id int NOT NULL,
  last_name varchar(255) NOT NULL,
  first_name varchar(255),
  age int,
  CONSTRAINT age_check CHECK (age >= 0 AND age <= 120)
);
```

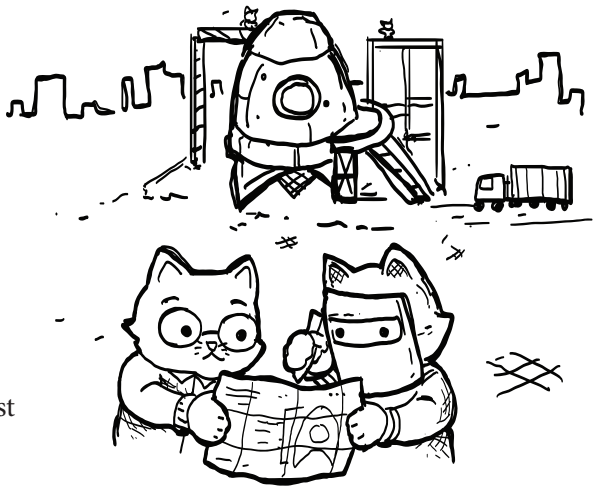
In the preceding example, `Age >= 0 AND Age <= 120` is the condition of the check constraint.

Do you have any columns that can use check constraints in your database design? You have one case that fits the preceding scenarios perfectly. Both your `user_address` and `billing_address` tables contain a `state column` that stores the names of states in the United States. The valid values of this column are limited to 50 known names. Instead of relying on your frontend or backend developers to do the right thing, you can use the check constraint to take the burden off their shoulders. Given that 50 is not a small number, you can opt to add the constraint after you create the table, as follows:

```
-- comment: works for MySQL, MariaDB, PostgreSQL, and
-- comment: Oracle; see the code repo for other RDBMS
CREATE TABLE user_address (
  email VARCHAR(320) PRIMARY KEY,
  street_address VARCHAR(255),
  address_line_optional VARCHAR(100),
  city CHAR(100) NOT NULL,
  state VARCHAR(20) NOT NULL,
  postal_code CHAR(5) NOT NULL
);
```

```
ALTER TABLE user_address
 ADD CONSTRAINT chk_state
 CHECK (
  state IN (
   'Alabama', 'Alaska', 'Arizona',
   'Arkansas', 'California', ...,
   'West Virginia', 'Wisconsin', 'Wyoming'
  )
);
```

`('Alabama', 'Alaska', … 'Wyoming')` defines a list of the allowed values. The `IN` keyword requires all the values in that column to be one of the values in the list.



Recap

- Normalization is the process of breaking the database into smaller, more manageable tables, each table representing a single entity or concept.
- Typically, when all your tables are in BCNF, you can consider your database fully normalized. A table in BCNF must have the following features:
  - It has a primary key.
  - It has no multivalued columns.
  - All columns are dependent on a key but nothing else.
  - It contains no transitive dependency. (All its nonkey columns are directly dependent on a key.)

  In a sense, normalization involves checking your tables against the guidelines of BCNF, identifying violations, and revising your database design.
- `NOT NULL` and primary key/foreign key constraints play critical roles in your database design.
- If you may need to restructure your database design or migrate your database, name all constraints that can be named.
- To avoid using `NULL` values in a column, use the `NOT NULL` constraint.

- The `UNIQUE` constraint is handy if you have some nonkey columns that should all contain unique values.
- The `DEFAULT` constraint is handy for dealing with timestamps and placeholder values.
- The `CHECK` constraint can validate data format, ranges, and other complex business logic when other constraints fail.

Security and optimization 7



In this chapter

- You evaluate and strengthen the security of your database.
- You further improve the storage efficiency of your database design beyond normalization.
- You learn about indexing and how to apply it when implementing your database design to improve query performance.
- You learn about denormalization and apply it to your database design to further improve query performance.

What you need to know

You can find the database design covered in this chapter (it's finally complete) represented using tools commonly used by practitioners, such as dbdiagram.io and MySQL Workbench. in the GitHub repository (https://github.com/Neo-Hao/grokking-relational-database-design). You can navigate to the `chapter_07` folder and follow the instructions in the `README.md` file to load the database design into corresponding tools.

(continued)

You can also find the SQL scripts corresponding to the finalized database design for different relational database management systems (RDBMSs), including MySQL, MariaDB, PostgreSQL, SQLite, SQL Server, and Oracle.

Overview

In this chapter, you will explore database security and optimization. You will see how to evaluate the integrity and confidentiality of your database design, enhance storage efficiency beyond normalization, understand when to use denormalization, and apply indexing to improve query performance.

Security



Security is an important aspect of database design. To help safeguard the data against leaks, you need to evaluate the security of your database design and ensure that it follows at least a few key security principles. In this section, you will learn about two of those principles—integrity and confidentiality—and see how to apply them to your database design.

Integrity

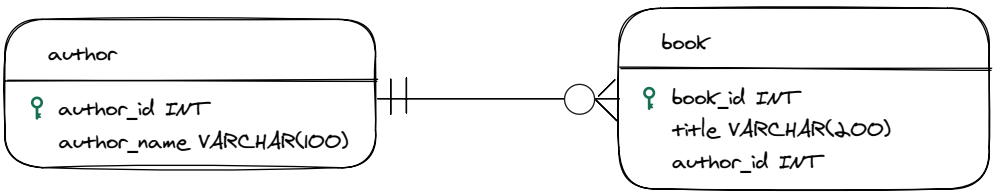
Integrity involves maintaining the accuracy, completeness, and trustworthiness of data and systems. In database design, using the integrity lens to review your database design typically involves checking whether all columns, tables, and relationships are designed and implemented properly to preserve data integrity. By "designed and implemented properly," we mean that they follow the established

principles of database design and implementation covered in chapters 4 to 6, such as the following:

- Every column has the best data type.
- Every table has a primary key.
- A foreign key maintains the relationship between tables.
- Constraints are based on requirement analysis.

Database design typically involves multiple iterations. It is easy to make some small mistakes or neglect something. The foreign key constraint may easily slip your mind, for example.

Imagine a database dedicated to storing authors and their books for a publishing company's web application. This database has two related tables, `author` and `book`, with the following design:



Because entity-relationship (E-R) diagrams don't represent foreign keys or other constraints, it is easy to neglect them and map the two tables to the following SQL command:

```
CREATE TABLE author (
  author_id INT PRIMARY KEY,
  author_name VARCHAR(100) NOT NULL
);
-- comment: the book table misses the
-- comment: foreign key constraints
CREATE TABLE book (
  book_id INT PRIMARY KEY,
  title VARCHAR(200) NOT NULL,
  author_id INT NOT NULL
);
```

Without a foreign key constraint, nothing prevents an orphan record in the child table from referring to a nonexistent record in the parent table. Such problems can make the database more complex to manage

and secure, in turn creating gaps that attackers can exploit. To fix the preceding SQL command, you need to add the foreign key constraint to the `book` table:

```
CREATE TABLE book (
  book_id INT PRIMARY KEY,
  title VARCHAR(200) NOT NULL,
  author_id INT NOT NULL,
  CONSTRAINT FK_author_id
    FOREIGN KEY (author_id)
    REFERENCES author(author_id)
);
```

Confidentiality

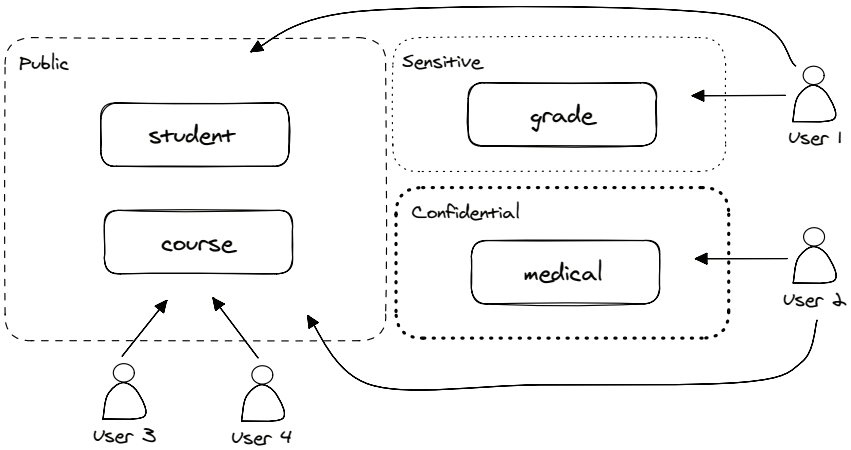
Confidentiality involves protecting information from unauthorized access and disclosure, ensuring that data is accessed only by authorized people and processes. You can take two approaches to confidentiality: *access control* and *encryption*.

Access control

Using access control in databases is like posting a security guard at a club entrance. This guard decides who gets in and what areas they can access. In a database, access control determines who can view, modify, or delete data. It's essential to ensure that authorized users are given the right amount of authority to access and modify the data they should be able to control.

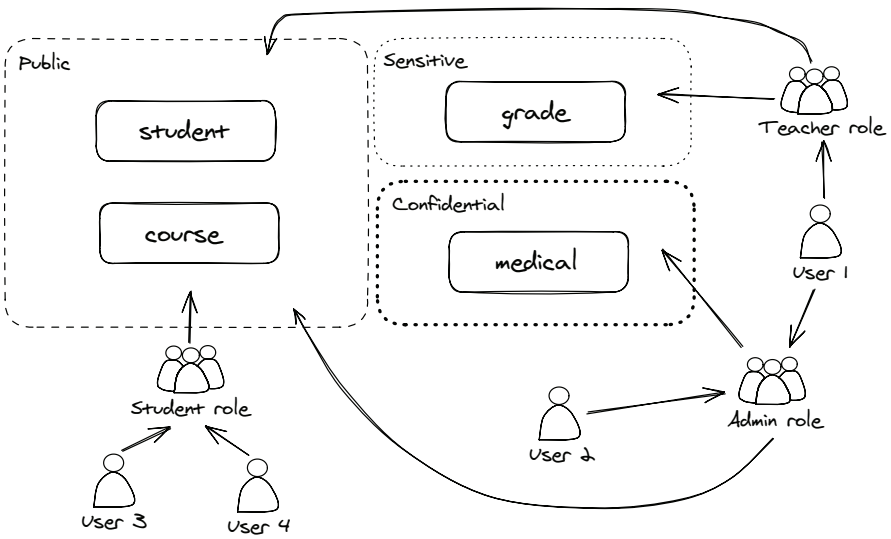
The two main types of access control are *mandatory access control* (MAC) and *role-based access control* (RBAC). MAC is like a strict school with rigid rules. In this system, a central authority controls who can access what data, and the rules are very tight. Every piece of data, such as a table or row, has a classification level (such as sensitive or confidential), and users are given access permissions for different classification levels. If your permission doesn't match the data's classification, you can't access it. There are no exceptions.

Think about a high school database that manages student records. In this database, tables are labeled by level of sensitivity:



Only users with the right clearance can access tables labeled as sensitive or confidential. In MAC, sensitivity classification needs to be configured manually.

RBAC is more flexible and user-friendly. Instead of assigning permissions to users individually, RBAC assigns permissions to roles, and users are assigned to these roles. The system is like different types of membership in a club: some members can access the VIP lounge, some can access only the bar, and others can only access the general area. If RBAC is used in the high school's database, it adds an extra layer of roles:



Now you know the two main types of access control, let's walk through implementing RBAC on a few tables of The Sci-Fi Collective's database. To keep this example simple, we'll have you work with only four tables: `product`, `purchase`, `review`, and `purchase_review`. Assume that you've done some requirements analysis of the user roles for the four tables:

- `USER`—View and buy products, and submit reviews for different products.
- `ANALYST`—View records from all tables to perform statistical analysis.
- `ADMIN`—View, modify, or delete records from all four tables.

Based on these requirements, you can create the role, grant permissions to the roles, and assign the roles to users. You can conduct this process by using the graphical interface software of a RDBMS or SQL commands. You can start by tackling the `USER` role:

```
-- comment: works for MySQL and MariaDB
-- comment: consult ChatGPT on adaptations
-- comment: for other RDBMS
-- step 1: create a role
CREATE ROLE standard_user;
-- step 2: grant permissions to a role
GRANT SELECT ON database_name.product TO standard_user;
GRANT INSERT ON database_name.purchase TO standard_user;
GRANT INSERT ON database_name.purchase_product
   TO standard_user;
GRANT INSERT ON database_name.review TO standard_user;
-- step 3: assign the role to a new user
CREATE USER 'morpheus'@'%' IDENTIFIED BY 'password';
GRANT standard_user TO 'morpheus'@'%';
```

A few things about the preceding code snippet deserve explanation:

- The role's name is `standard_user` instead of `user` because `user` is a reserved SQL keyword in most RDBMSs.
- A `SELECT` clause (such as `SELECT ON database_name.product`) corresponds to viewing a table, and an `INSERT` clause (such as `INSERT ON database_name.purchase`) corresponds to adding data to a table. The `USER` role allows users to view and buy products as well as submit reviews. Viewing products translates to the `GRANT SELECT ON … TO …` command; buying products and submitting reviews translate to the `GRANT INSERT ON … TO …` command that is applied to multiple tables.

- Whenever you create a new user d via the `CREATE` command, the `standard_user` role is immediately granted to that user.
- In MySQL and MariaDB, user accounts always require a host specification in the format `'username'@'host'`. The `'host'` portion determines where the user can connect from. `'username'@'%'` allows connections from any host.

The `ANALYST` role can be put together as follows:

```
-- comment: works for MySQL and MariaDB
-- comment: consult ChatGPT on adaptations
-- comment: for other RDBMS
-- step 1: create a role
CREATE ROLE analyst;
-- step 2: grant permissions to a role
GRANT SELECT ON database_name.product TO analyst;
GRANT SELECT ON database_name.purchase TO analyst;
GRANT SELECT ON database_name.purchase_product
   TO analyst;
GRANT SELECT ON database_name.review TO analyst;
-- step 3: assign the role to a new user
CREATE USER 'smith'@'%' IDENTIFIED BY 'password';
GRANT analyst TO 'smith'@'%';
-- step 4: enable role by default (MySQL 8.0+)
ALTER USER 'smith'@'%' DEFAULT ROLE analyst;
```

Because the `ANALYST` role requires only view access to four tables, only the `GRANT SELECT` clause is needed here.

The `ADMIN` role is addressed as follows:

```
-- comment: works for MySQL and MariaDB
-- comment: consult ChatGPT on adaptations
-- comment: for other RDBMS
-- step 1: create a role
CREATE ROLE admin;
-- step 2: grant permissions to a role
GRANT SELECT, INSERT, UPDATE, DELETE
  ON database_name.product TO admin;
GRANT SELECT, INSERT, UPDATE, DELETE
  ON database_name.purchase TO admin;
GRANT SELECT, INSERT, UPDATE, DELETE
  ON database_name.purchase_product TO admin;
GRANT SELECT, INSERT, UPDATE, DELETE
  ON database_name.review TO admin;
-- step 3: assign the role to a new user
CREATE USER 'david'@'%' IDENTIFIED BY 'password';
GRANT admin TO 'david'@'%';
-- step 4: enable role by default (MySQL 8.0+)
ALTER USER 'david'@'%' DEFAULT ROLE admin;
```

Access control is an important task frequently performed by both database designers and administrators to various degrees throughout the software development process. This topic is worthy of its own chapter or a whole book that goes beyond the focus of this book (database design).

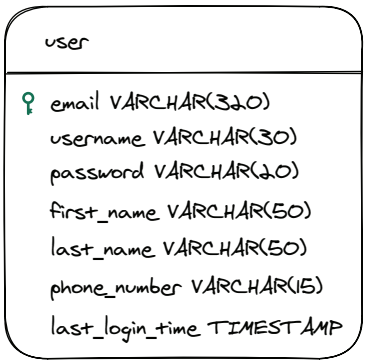
Encryption

Two types of encryption are commonly used in databases: one-way and symmetric.

One-way encryption turns data into a fixed-size, seemingly random string. This process, typically known as *hashing*, refers to a function that takes an input and returns a fixed-size string of bytes. One-way encryption is practically impossible to reverse, especially for longer values. In other words, given a hashed value derived from one-way encryption, you can't obtain its original input data.

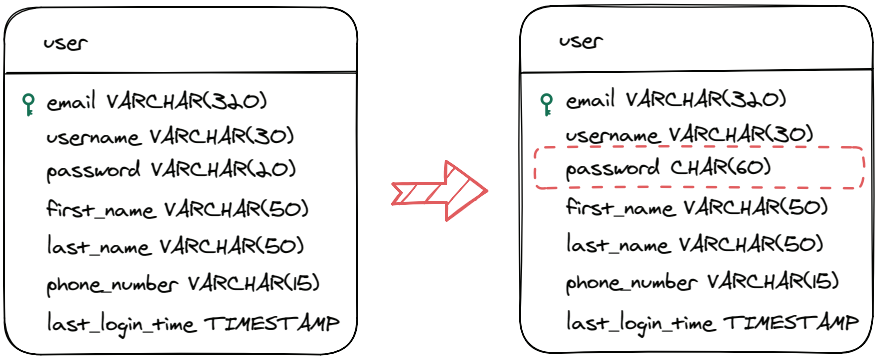
One-way encryption is frequently used to hash passwords in databases because people typically reuse passwords everywhere. If an unauthorized person gains access to the database, they won't know what the original passwords were, so the damage won't be greatly amplified. Common one-way encryption algorithms include bcrypt, PBKDF2, and Sha512.

Your own database design includes columns that can benefit from one-way encryption. The `user` table has a column (`password`) that's dedicated to storing users' passwords. Up to now, this column has been designed to store users' passwords in plain text:



When you apply a one-way encryption algorithm to hash all the passwords, however, you need to revise the database design accordingly. You could use the bcrypt algorithm to hash all the passwords, for example. This algorithm results in sequences of 40-byte binary data. Considering that some binary data can't be printed easily, it is common practice to further encode the binary data into a string of ASCII characters via some encoding mechanism (such as Base64) so you can store hashed passwords with the `CHAR` data type.

The encoded string from the 40-byte binary data will be 60 bytes long. That said, `VARCHAR(20)` is too small for the hashed passwords, so you have to update the data type of your `password` column to `CHAR(60)`:



If you have already implemented your database design, you can use the following SQL command to update your `user` table:

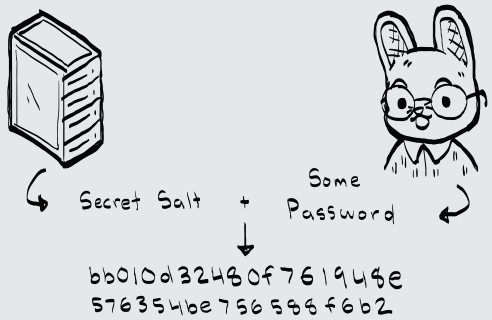
```
-- comment: works for MySQL and MariaDB
-- comment: consult ChatGPT on adaptations
-- comment: for other RDBMS
ALTER TABLE user MODIFY COLUMN password CHAR(60);
```

Unhashing the mystery

A *hash function* is a mathematical function that takes a variable-length input (such as a password) and produces a fixed-length output: the hash. The function is one-way, so you won't see the word *unhashing* in any dictionary! Hash functions have many uses, but the hash functions we cover here are used specifically for encryption.

(*continued*)

Plain passwords in databases that are compromised can be stolen and reused elsewhere, which is why good database design dictates that passwords be stored in hash form. Many hash functions are well known and standardized (such as Sha512 and bcrypt). Even hashed passwords can be vulnerable, though, because an attacker may try a brute-force attack (try random passwords through the hash function) or use a rainbow table (a list of known passwords and their precalculated hashes). ost applications would concatenate a password with a *salt* (random string) that is stored or produced by the application. An attacker has no means of deducing what the password is without knowing the salt.



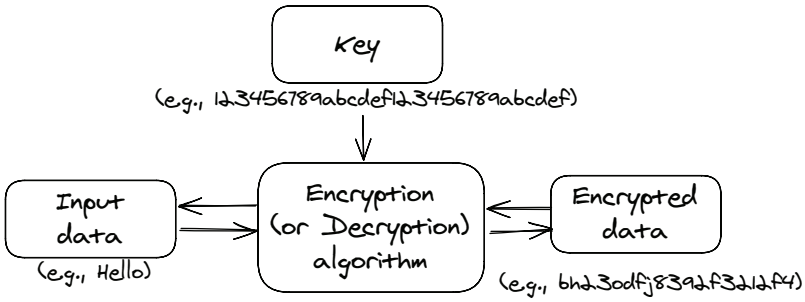
In case if you wonder how user authentication works, here is a breakdown of the process:

1. The application takes the password that the user provides during the login process and hashes it in the same approach.
2. The application compares the hashed result with what is already stored in the database. If the hashed result matches the hash stored in the database, the user is authenticated.

In contrast to one-way encryption, *symmetric encryption* is reversible. In other words, when a symmetric encryption algorithm is used for encryption, the encrypted value can be decrypted to retrieve the original input data.

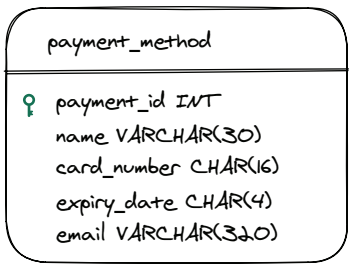
Symmetric encryption is commonly used to protect sensitive information that needs to be used repeatedly and/or periodically in its original form. Online stores, for example, want to protect users' data, so they don't users' credit card information in plain text. Online stores also want to make users' lives easier by allowing them to save and reuse

credit cards for the next order. As a result, symmetric encryption is used to encrypt credit card information to keep it safe in the database. When a user wants to reuse a credit card for a transaction, the application can always decrypt the stored value. Symmetric encryption achieves encryption and decryption by using the same key and same algorithm in the process of encryption and decryption. The key is typically a string of characters or numbers, and the security of symmetric encryption relies on the secrecy and proper handling of this key.



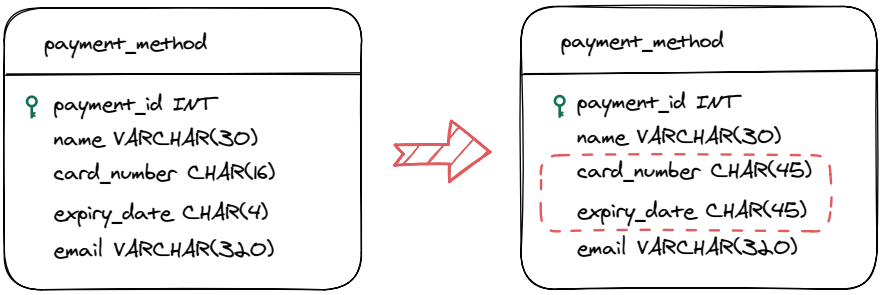
Common symmetric algorithms include Advanced Encryption Standard (AES), Triple DES (3DES), and Blowfish. Each algorithm has a set of variants. Depending on the key length (such as 128 bits, 192 bits, or 256 bits), AES has three variants: AES-128, AES-192, and AES-256. The key length determines the level of security and the computational requirements for encryption and decryption. The longer the key is, the higher the security and computational requirements are.

Can any columns in your own database design can benefit from symmetric encryption? Yes. The `payment_method` table stores some columns of sensitive information that can be reused repeatedly, such as `card_number` and `expiry_date`. The two columns represent the credit card information required to place an order. Up to now, they've been set to store everything in plain text:



If you want to store encrypted data instead of plain text, you must revise the database design. You could use AES-256 to encrypt `card_number` first. AES-256 operates on blocks of 16 bytes. The `card_number` is 16 bytes, which is one block in ES-256. AES algorithms commonly use *initialization vectors* (IVs) to further enhance security. You can think of an IV as a nonsecret 16-byte binary sequence used as an additional input. When IV is used, the input data is 32-byte. As in one-way encryption, the encrypted data is binary, and it is common to further encode the binary data into a string of ASCII characters. Assuming that Base64 was used to encode the binary data, the encoded string is about 45-byte.

What about `expiry_date`? AES-256 operates on blocks of 16 bytes; when the plain text is shorter than 16 bytes, padding is required to make the plain text fit into a 16-byte block, so AES-256 considers the lengths of `expiry_date` and `card_number` to be the same (16 bytes). If the same encryption and encoding procedure is applied, the encoded and encrypted string will all be about 45-byte. That said, you need to update the data types of both columns to `CHAR(45)`:



If you have already implemented your database design, you can use the following SQL command to update your `payment_method` table:

```
-- comment: works for MySQL and MariaDB
-- comment: consult ChatGPT on adaptations
-- comment: for other RDBMS
ALTER TABLE payment_method
  MODIFY COLUMN card_number CHAR(45),
  MODIFY COLUMN expiry_date CHAR(45);
```

Symmetric encryption: Not the panacea

Symmetric encryption seems to be more convenient than one-way encryption because it supports both encryption and decryption. You may wonder why we don't use symmetric encryption to encrypt passwords and all other information that requires protection. Symmetric encryption should not be used to encrypt passwords. To take this discussion one step further, if the sensitive data doesn't need to be used repeatedly in its original form, you shouldn't consider symmetric encryption.

One-way encryption is typically stronger than symmetric encryption. Symmetric encryption works on a big assumption: the key used for encryption and decryption is safe. If a hacker somehow gains access to the key, all the sensitive information encrypted with symmetric encryption could be in danger because everything can be reversed and the original data exposed.

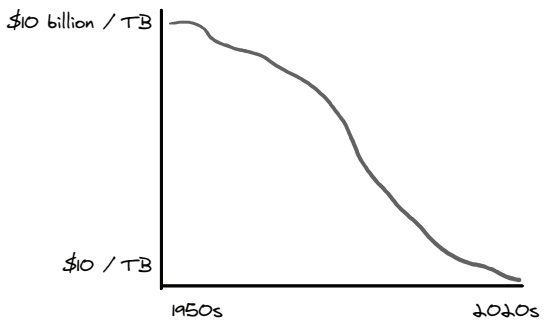
As a result, storage of the key in symmetric encryption is critical. You don't want to store this key in the same database as the encrypted data, for example. In the real world, this key is stored in various ways, such as the following:

- Via environmental variables in a file in the application server
- Via dedicated software known as a *key management service*
- Via a specialized physical device known as a *hardware security module*

Storage considerations

After a database is designed, revised, implemented, and tested, it goes into production and starts costing money. You need to think about not only read and write operations but also efficient use of storage.

Historically, storage was expensive. In the 1970s, 1 MB of storage could cost about $250. The cost of that same megabyte today is about $0.00001.



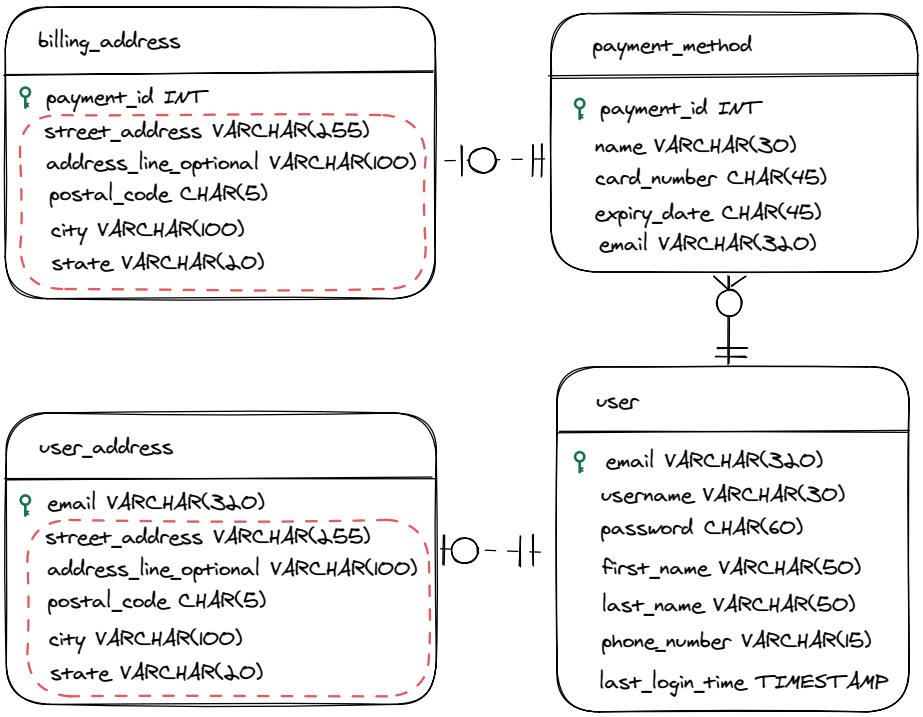
Despite this trend, improving your database design to save storage space can still be valuable, especially if you are designing the database for a scalable application that millions of people will use or that may experience exponential growth.

What do you do in database design to save storage? Reduce data redundancy. You have already done a lot in this respect. Normalization addresses anomalies as well as reduces data redundancy. Beyond normalization, you may find opportunities in your design to further reduce data redundancy in your design, as you will see in this section.

Redundant tables

Your database design for The Sci-Fi Collective is normalized. But a normalized design can still be improved to reduce redundancy.

When you review your design and check for redundancies, you can start by asking a simple question: are there any redundant (or almost redundant) tables? The answer is yes. If you focus on how addresses are stored, you see two tables that are almost identical:



The tables `billing_address` and `user_address` have nearly identical columns except for the primary key. This design is legit because an address associated with a payment method may not be the same as a user's personal address. It is highly likely, however, that a user and their payment methods share an address, so it would be nice to consolidate that data.

How do you proceed? A good start is to analyze and update the requirements. To consolidate address data, you can use a single address entity that stores a single address that both the `user` and `payment_method` tables can point to or two different addresses so that you don't have to store the same address in two different tables. Based on this change, the requirements that need to be fulfilled need further clarification:

- A user has one address.
- A payment method has one address.
- An address is associated with zero to many users or zero to many payment methods.

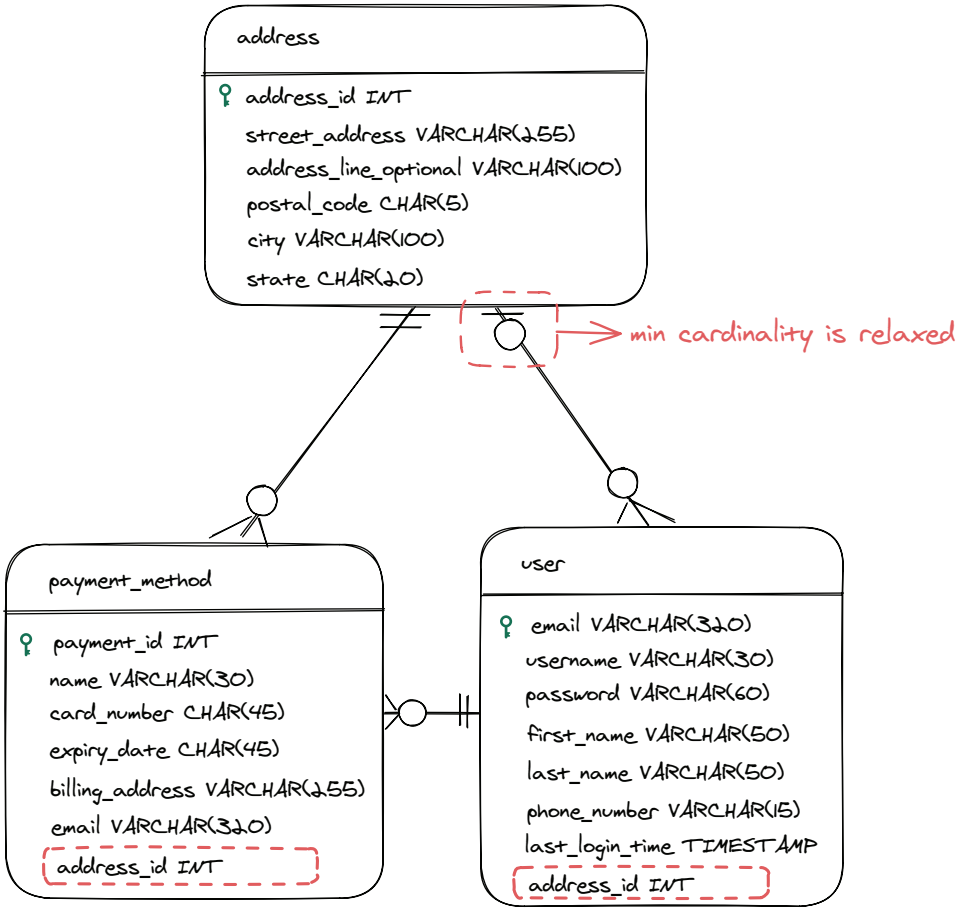
It's not hard to map the requirements to an E-R diagram. If you use a single entity (such as `address`) to represent addresses, what should be its primary key? If you remember what you learned in chapter 4, you need a real-world concept to differentiate two addresses—a combination of all columns, including `street_address`, `address_line_optional`, `postal_code`, `city`, and `state`. Using a composite primary key composed of all columns is generally considered bad design for two reasons:

- A composite key composed of every column significantly drags down performance.
- The `address` entity is in a one-to-many relationship with `user` and `payment_method`. The choice of this composite key will force you to put every column of `address` in `user` and `payment_method`.

As a result, you may choose an alternative to the composite key: surrogate key plus unique constraint. In chapter 4, you learned that a surrogate key is often implemented as an autoincrementing numeric attribute (such as `address_id`), which is simple, flexible, and easy to use as a single-column foreign key. The unique constraint will be placed

on the combination of all columns except the surrogate key (`address_id`) to ensure that no redundant address data can be recorded in the `address` table.

Now that you have nailed nearly every bit of the revised design, you are ready to map the revised requirements to an E-R diagram:



It is worth noting that the min cardinality of the relationship from `user` to `address` tables is relaxed from one to zero. This relaxation is a must, considering that users will register accounts with The Sci-Fi Collective without providing personal address information. If the min cardinality of this relationship stays as one, it will prevent user data from being recorded in the `user` table without a corresponding `address` row.

If you have already implemented your database design, you need to take three steps to update your implementation:

1. Drop two old tables, `billing_address` and `user_address`.
2. Create a new table named `address`, and add a unique constraint to the combination of all nonkey columns.
3. Update `payment_method` and `user` by adding a new column. `address_id`, and making it a foreign key in both tables.

To create the `address` table, you can use the following command:

```
-- comment: works for MySQL and MariaDB
-- comment: see the code repo for other RDBMS
CREATE TABLE IF NOT EXISTS address (
  address_id INT AUTO_INCREMENT PRIMARY KEY,
  street_address VARCHAR(255) NOT NULL,
  address_line_optional VARCHAR(100),
  city VARCHAR(100) NOT NULL,
  state VARCHAR(20) NOT NULL,
  postal_code CHAR(5) NOT NULL,
  CONSTRAINT unique_address_constraint
    UNIQUE (street_address, address_line_optional,
            postal_code, city, state)
);
```

To update the `payment_method` table, you can use the following command:

```
-- comment: works for MySQL, MariaDB, and PostgreSQL
-- comment: consult ChatGPT on adaptations
-- comment: for other RDBMS
ALTER TABLE payment_method
  ADD COLUMN address_id INT NOT NULL;

ALTER TABLE payment_method
  ADD CONSTRAINT fk_address_payment_method
     FOREIGN KEY (address_id)
     REFERENCES address(address_id);
```

By contrast, the `address_id` won't have the `NOT NULL` constraint in the `user` table because of the relaxed min cardinality of the relationship from `user` to `address`. Moreover, you need to ensure that the `address_id` column starts with a `NULL` value by default:

```
-- comment: works for MySQL, MariaDB, and PostgreSQL
-- comment: consult ChatGPT on adaptations
-- comment: for other RDBMS
ALTER TABLE user
  ADD COLUMN address_id INT NULL;
```

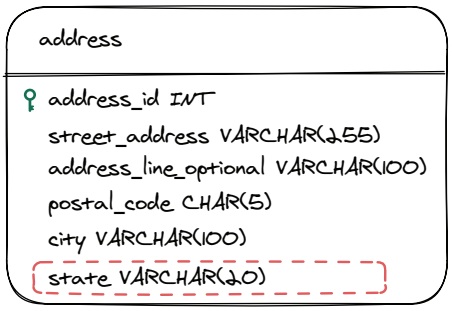
```
ALTER TABLE user
  ADD CONSTRAINT fk_address_user
    FOREIGN KEY (address_id)
    REFERENCES address(address_id);
```

NULL values in foreign key columns represent optional relationships, which is the business logic you want. In other words, not every record in the child table (user) needs to have a corresponding link in the referenced parent table (address). That logic also means, however, that you need to be extra-cautious in handling NULL values when querying the two tables.

Categorical data

After checking and dealing with redundant tables, you can go one step further by checking for columns that contain redundant data. When you review your database design, you can try a different question: Is there a column that contains a lot of redundant data? Yes.

If you focus on the address table that you just added, you see that the state column contains a lot of redundant data. After all, The Sci-Fi Collective has millions of users, and there are only 50 states plus Washington, DC, in the United States. (In chapter 4, we assumed that The Sci-Fi Collective sells only in the United States.)



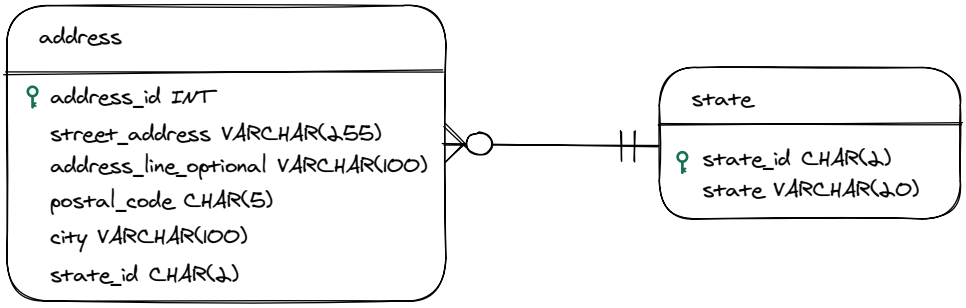
How do you address this problem? You can move the state column from the address table to a new table named state and link the two tables via a relationship. This new table, named state, will represent all 50 states plus the District of Columbia, and it will be in a one-to-many relationship with the address table.

What would be the primary key for the state table? Every state plus the District of Columbia has its own abbreviation, and the abbreviations

are different but have the same length. As a result, the abbreviation is the perfect primary key for the `state` table:

Alabama: AL
Alaska: AK
...
California: CA
Colorado: CO
...
District of Columbia: DC

Based on the preceding revision, your updated `address` table and new `state` table will look like this:



As you learned in chapter 5, you need to relax the min cardinality of the relationship from `state` to `address` a little to enable data entry.

Indexing

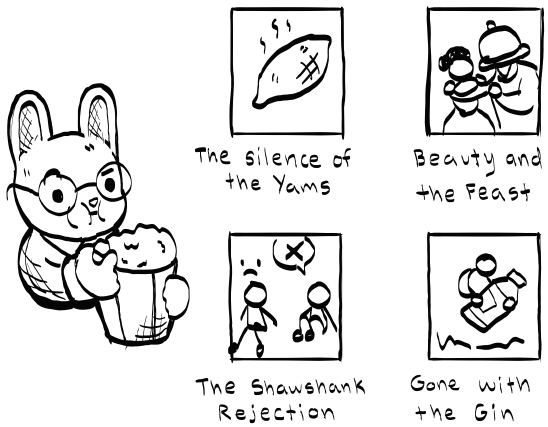
Now that you have taken action to further reduce data redundancy in your database design, it is time to work on making data retrieval efficient. *Indexing* is a critical optimization technique used to speed retrieval of data from a database. When implementing a database design, you need to index certain columns to speed tasks such as data retrieval, sorting, and searching. In this section, you will learn why and how to index; then you will decide whether you need to index certain columns in your own database design.

Why and how to index

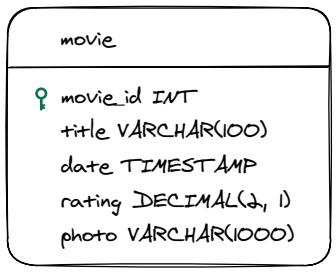
Several types of indexing are used to optimize query performance in databases, such as standard indexes, clustered indexes, spatial indexes, and full-text indexes. In this section, you will learn why and how to implement standard and full-text indexes.

Standard indexes

To index properly, you need to understand why indexes are important and what problems they address. Let's start with a simple `movie` table in a database that supports a movie application:



The design of the `movie` table looks like this:



Notice that the application frequently sorts all movies by ratings because users always want to watch the highest-rated movies. As a

result, your application frequently takes advantage of the following query:

```
-- comment: works for MySQL, MariaDB,
-- comment: PostgreSQL and SQLite
-- comment: consult ChatGPT on adaptations
-- comment: for other RDBMS
SELECT *
  FROM movie
  ORDER BY rating DESC
  LIMIT 30;
```

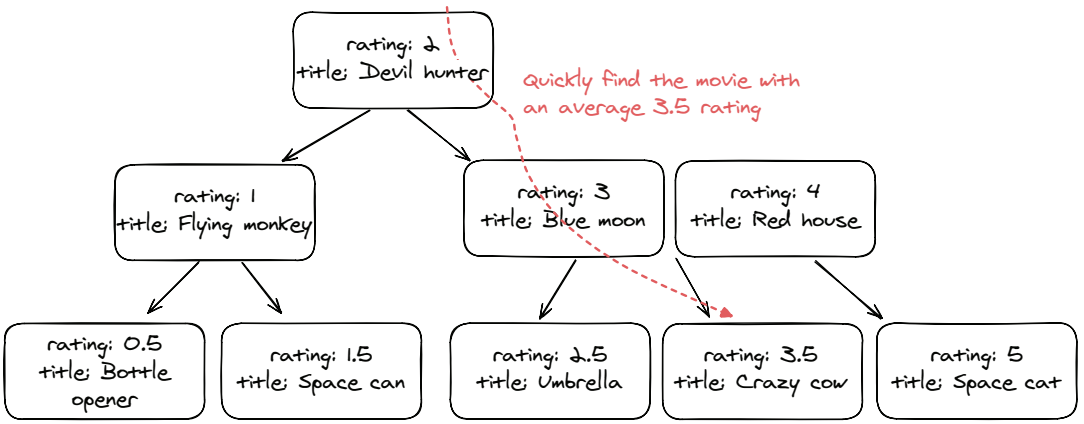
The query is executed clause by clause in the following order:

1. `FROM`—The query begins with the `FROM` clause. The database system identifies the `movie` table from which the data will be retrieved and reads data from the table.
2. `SELECT`—The `SELECT` clause selects every column for each row.
3. `ORDER BY`—`ORDER BY` sorts all the rows in the `movie` table by rating in descending order.
4. `LIMIT`—Finally, the `LIMIT 30` clause restricts the output to the first 30 rows of the ordered list.

Without the help of indexes, this query needs to sort all movies by rating from scratch every time this query is executed. If the `movie` table holds millions of records and keeps growing, you can imagine how easy it is for this query to affect the performance of the database and slow the application. To address this problem, you can index the `rating` column:

```
CREATE INDEX idx_rating ON movie (rating);
```

What does an index do? An index creates additional data structures that provide fast access to rows based on the values in one or more columns. Indexes are typically implemented with data structures such as B-trees. A *B-tree* is a self-balancing tree data structure that maintains sorted data and allows searches, sequential access, insertions, and deletions in logarithmic time. When an index is created on a column, the database engine constructs the index by scanning the entire table and recording the values of the indexed column along with pointers to the corresponding rows. In our case, when the index on the `rating` column is constructed, it will be used to speed queries that sort or search for a value in the `rating` column of the `movie` table:



It is worth mentioning that all queries that use the `WHERE` clause to filter data based on ratings perform searching tasks and are be optimized by the index you added on the `rating` column:

```
SELECT *
  FROM movie
  WHERE rating = 5;
```

Similarly, if you foresee or observe that users often sort or search for movies by release date, and the application frequently executes the query

```
-- comment: works for MySQL, MariaDB,
-- comment: PostgreSQL and SQLite
-- comment: consult ChatGPT on adaptations
-- comment: for other RDBMS
SELECT *
  FROM movie
  ORDER BY date DESC
  LIMIT 30;
```

you may want to index the `date` column too:

```
CREATE INDEX idx_rating ON movie (date);
```

If the application allows users to sort movies by both release date and rating, and the application frequently executes the query

```
-- comment: works for MySQL, MariaDB,
-- comment: PostgreSQL and SQLite
-- comment: consult ChatGPT on adaptations
-- comment: for other RDBMS
SELECT *
  FROM movie
```

```
  ORDER BY date DESC, rating DESC
  LIMIT 30;
```

you may want to index the combination of the `rating` and `date` columns too:

```
CREATE INDEX idx_combo ON movie (rating, date);
```

In all those cases, the columns that were explicitly indexed are not the primary key. What if the primary key requires indexing? Well, the primary key `movie_id` column is already indexed. All primary keys are indexed automatically.

Full-text indexes

A `CREATE INDEX` statement creates standard indexes, which are suitable for speeding many tasks, such as sorting and exact matches. Occasionally, however, you will find that standard indexes aren't sufficient for some indexing tasks. Your movie application may allow users to search for movies by titles, and the search bar needs to support keyword searching, partial word matching, and even fuzzy searching (handling typos or misspellings).

When you need to support complex search on text, you need *full-text indexes*, which make all such tasks much more efficient. In short, full-text indexes tokenize each word or term and then store each token in a lookup table. Without full-text indexing, keyword searching would look like this example:

```
SELECT *
  FROM movie
  WHERE title LIKE '%exciting%';
```

As you can imagine, even if this search is doable, it is inefficient. As the data size grows, this query takes longer to execute. With full-text indexing, the same task is much faster and looks like this:

```
-- comment: works for MySQL and MariaDB
-- comment: consult ChatGPT on adaptations
-- comment: for other RDBMS
-- comment: add a full-text index
CREATE FULLTEXT INDEX ft_idx_title
  ON movie (title);
-- comment: search
SELECT *
  FROM movie
  WHERE MATCH(title)
  AGAINST('exciting' IN NATURAL LANGUAGE MODE);
```

Although the concept of a full-text index is consistent across RDBMSs, the syntax varies significantly from one RDBMS to another. The preceding SQL command adds a full-text index for the `title` column in the `movie` table and works for MySQL and MariaDB. By contrast, to achieve the same thing in PostgreSQL, you need to use the following code:

```
-- comment: works for PostgreSQL
-- comment: consult ChatGPT on adaptations
-- comment: for other RDBMS
-- comment: add a full-text index
ALTER TABLE movie ADD COLUMN tsv_title;
UPDATE movie
  SET tsv_title = to_tsvector('english', title);
CREATE INDEX gin_idx_title
  ON movie USING gin(tsv_title);
```

If you want to figure out how to add a full-text index in a specific RDBMS, ChatGPT is a great friend.

Indexes for your ER diagram

To explore whether you can use indexes to optimize your database design, you need to understand what queries are generated by the application logic. This step may require close collaboration between you (the database designer) and application developers, especially backend engineers. In this process, backend engineers can do the following things:

- Contribute detailed knowledge of how the application interacts with the database, especially queries generated by the application logic.
- Provide insights into performance bottlenecks observed during application runtime and user interactions.
- Ensure that changes in application logic or new features align with best practices of database design.

During the collaboration, you will do the following things:

- Understand how to structure the database and design indexes to optimize these queries, and use the shared information to adjust database design, create appropriate indexes, and optimize query execution plans.
- Ensure that database schema changes do not negatively affect application performance or lead to inconsistencies.

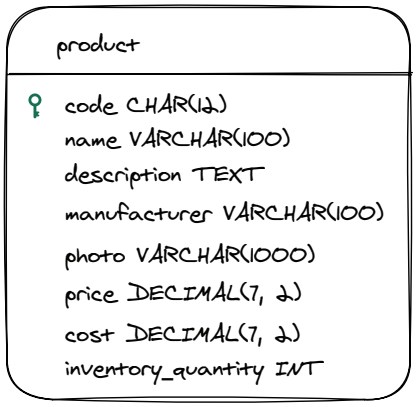
- Ensure that database-schema changes meet the customer's requirements (unless the requirements have changed).

Beyond collaborating with application developers, database designers commonly take empirical approaches to identify indexing opportunities by synthesizing data from query performance metrics and query logs when the implemented database goes into production. If you have identified opportunities to use indexes, the following three examples show what you need to do next.

The product table

For the `product` table, you've identified one interaction scenario:

> Users search for products based on their name.



This scenario corresponds to the following query:

```
-- comment: works with MySQL and MariaDB
-- comment: consult ChatGPT on adaptations
-- comment: for other RDBMS
SELECT * FROM product WHERE MATCH (name)
  AGAINST ('some text' IN NATURAL LANGUAGE MODE);
```

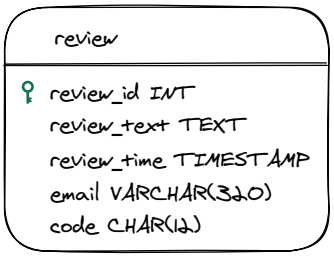
This query performs a keyword search task on the `name` column. To make the query possible and efficient, add a full-text index to this column:

```
-- comment: works for MySQL and MariaDB
-- comment: consult ChatGPT on adaptations
-- comment: for other RDBMS
CREATE FULLTEXT INDEX ft_idx_name ON product (name);
```

The review table

For the `review` table, you've identified the following interaction between users and the application:

> Users frequently check product reviews.



This interaction leads to frequent queries of the `review` table. When a user tries to check the reviews of a product with code `23356`, for example, the following query is executed:

```
SELECT * FROM review WHERE code = '23356';
```

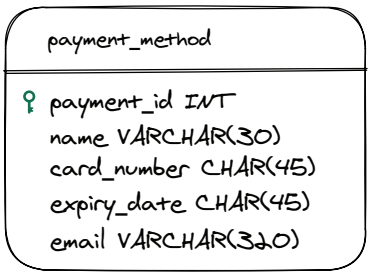
This query is a search task, but only because an exact match is involved. In other words, if this column ever needs indexing, standard indexing would be good enough. Indexing would improve the efficiency of this query. The `code` column in the `review` table is a foreign key, and foreign key columns are not indexed automatically. As a result, you can index this column as follows:

```
CREATE INDEX idx_code ON review (code);
```

The payment_method table

For the `payment_method` table, you've identified the following scenario:

> Users always need to select one of their many added payment methods frequently to complete a transaction.



This interaction leads to frequent queries of the `payment_method` table. When a user with the email address john@gmail.com tries to access his payment methods, for example, the following query executes:

```
SELECT * FROM payment_method
  WHERE email = 'john@gmail.com';
```

This query is also a search task that involves an exact match. The `email` column in the `payment_method` table is a foreign key. Standard indexing on this column would be good enough:

```
CREATE INDEX idx_email ON payment_method (email);
```

Denormalization

So far, you have put a lot of effort into optimizing a database. Now we are going to talk about edge cases in which optimization means actively going against normalization. Yes, you read that right!

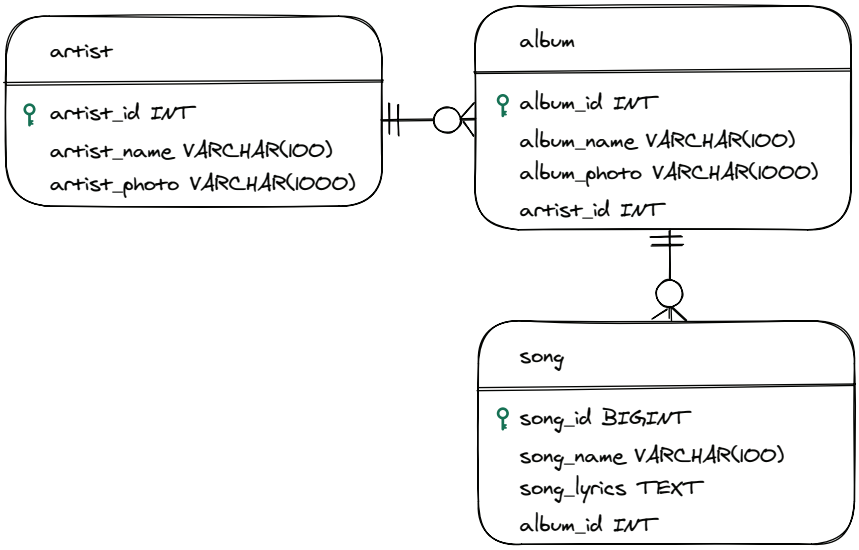
How to denormalize

Denormalization is a last-ditch database optimization technique that intentionally introduces redundancy to a database to improve query performance. Denormalization is rare, but if queries are performed frequently and require joining multiple tables, denormalization can significantly enhance the performance of your database.

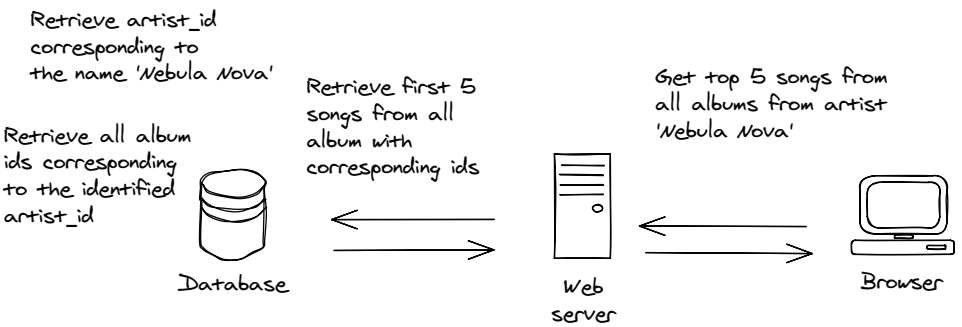
Denormalization typically involves two steps:

1. Identify queries that are used frequently and require joining more than two tables.
2. Duplicating some columns in some tables involved in the identified query to reduce the number of tables that have to be joined.

To get a better understanding of denormalization, consider an oversimplified example first. Suppose that you oversee a database supporting a music app, and you have three tables representing artists, albums, and songs in your database:



From the relationships among the tables, you can tell that songs are organized by albums and albums are organized by artists. You can also tell that the three tables are fully normalized. But the application supported by the database often needs to display the top songs for every artist:

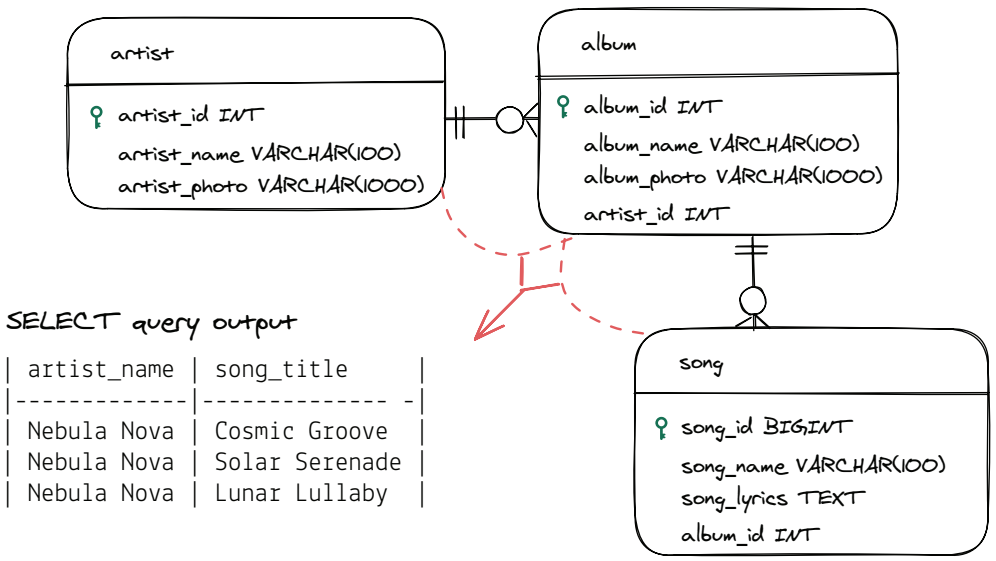


Because of the relationships among the three tables, you need to join them to map songs to artists. As a result, the following query has to be executed frequently:

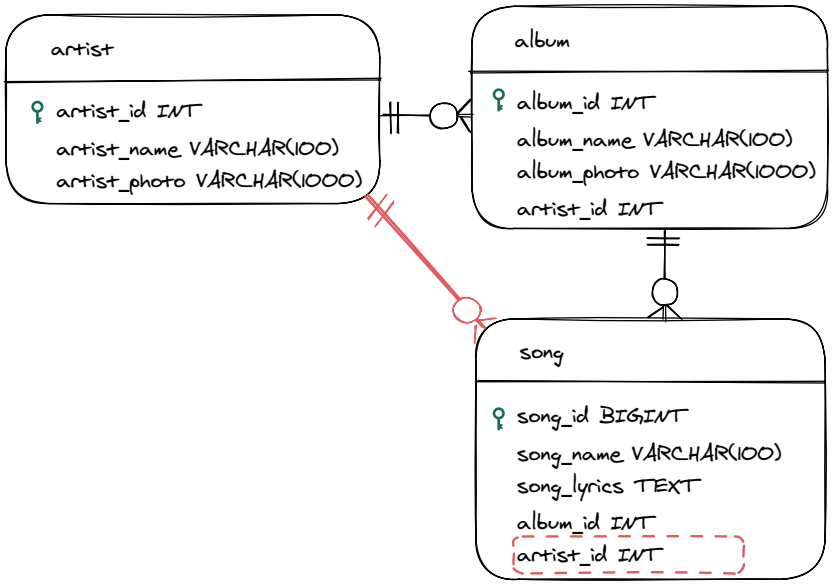
```
-- comment: works for MySQL, MariaDB,
-- comment: and PostgreSQL
-- comment: consult ChatGPT on adaptations
```

```
-- comment: for other RDBMS
SELECT artist.artist_name, song.song_title
  FROM artist
  INNER JOIN album
    ON artist.artist_id = album.artist_id
  INNER JOIN song
    ON album.album_id = song.album_id
  WHERE artist.artist_name = 'Nebula Nova'
  ORDER BY album.album_id, song.song_id
  LIMIT 5;
```

Because of the sizes of the three tables and the high frequency of the query, this query may take more time and resources to run than you expect. As a result, the pages or screens that require help from this query become less responsive in your application. Further, this query costs a lot of resources and money whenever it runs.



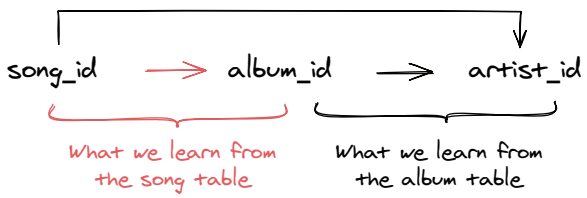
How do you address this problem? You may notice that the `album` table doesn't contribute any data to the output data. Due to how the three tables are related, you still need to join all three tables to get what you want. If a direct relationship exists between `artist` and `song`, this query will become much more efficient. Create this relationship, and update the E-R diagram of the three tables:



As you can see, this new relationship is one-to-many, and you have to add a new foreign key, `artist_i`, to the `song` table. Does this update address your problem? Yes. Now the query can be updated to a more efficient version:

```
-- comment: works for MySQL, MariaDB,
-- comment: PostgreSQL and SQLite
-- comment: consult ChatGPT on adaptations
-- comment: for other RDBMS
SELECT artist.artist_name, song.song_title
  FROM artist
  INNER JOIN song
    ON artist.artist_id = song.artist_id
  WHERE artist.artist_name = 'Nebula Nova'
  ORDER BY song.song_id
  LIMIT 5;
```

This update, however, introduces a new problem: it creates a transitive dependency in the `song` table:



You deliberately introduced redundancy to the `song` table to make a normalized design abnormal—hence, the name *denormalization*. Sometimes, you have to weigh the benefits and costs of sticking to a fully normalized design and tolerating some abnormal tables when you need to improve query efficiency and database performance.

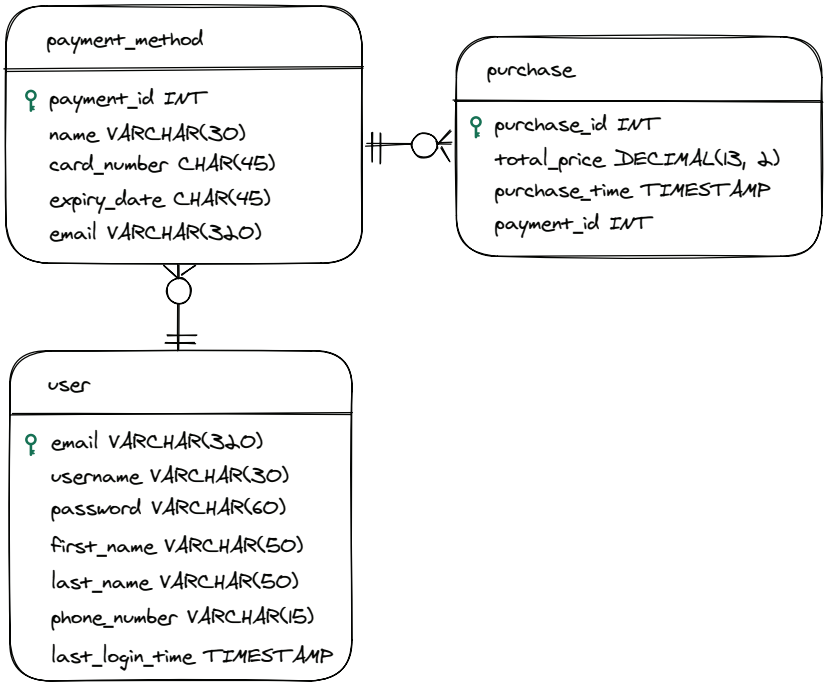
It's important to know that denormalization has the potential to compromise data integrity if it is not managed meticulously. When redundant data exists across denormalized tables, any alterations of the data may require updates of all related tables.

After denormalization, you still need to pay close attention to database performance to validate your move. If a significant volume of `INSERT`, `UPDATE`, and `DELETE` operations occur on related tables, the performance benefits of denormalization may diminish quickly, and you may need to reevaluate the situation.

Comprehensive documentation and effective communication are crucial to denormalizing a database design. After all, denormalization can be a debatable, controversial move. To get other people to buy into this change, you have to communicate the denormalization strategy to members of the development team, stakeholders, and other interested parties to establish shared understanding and ensure consistent implementation. More important, to prevent future problems or accidents, you want to thoroughly document the denormalization strategy, outlining the rationale, tradeoffs, and factors that influenced the denormalization decisions.

Denormalize your database design

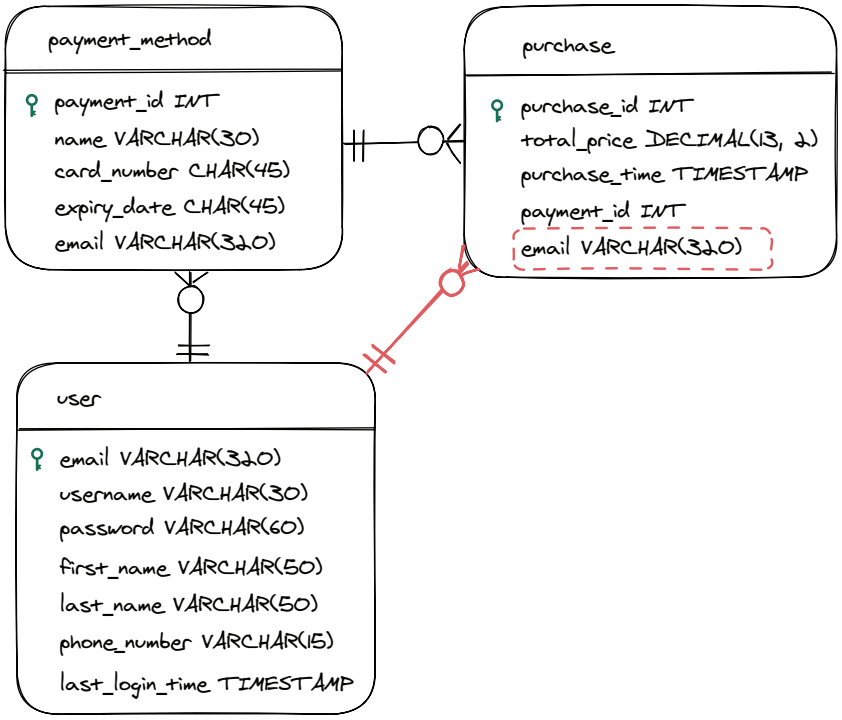
Take a look at your database design to see whether denormalization can be useful anywhere. Specifically, you want to look at the `user`, `payment_method`, and `purchase` tables, as shown in the following figure.



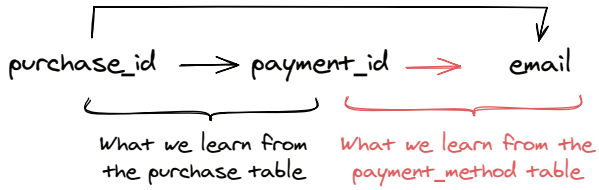
In an online store like The Sci-Fi Collective's, users commonly check their orders or receipts. Whenever that happens, the application queries the preceding three tables:

```
SELECT * FROM purchase
  INNER JOIN payment_method
    ON purchase.payment_method_id =
      payment_method.payment_method_id
  INNER JOIN user
    ON purchase.user_id = user.user_id
  WHERE user.email = 'customerCat@humor.com';
```

Apparently, the `payment_method` table barely contributes to the output data but has to be joined with the other two tables so that a purchase can be mapped to a corresponding user. If you recall from chapter 6, we removed the direct relationship between `user` and `purchase` when we focused on normalizing the database design. If you have abundant evidence showing that the preceding query is executed frequently, costing significant money and resources and slowing the application, you need to think again about whether to restore the direct relationship between `user` and `purchase` (in other words, to denormalize your database design):



Undoubtedly, this change will introduce redundancy and create a violation of Boyce-Codd Normal Form (BCNF; see chapter 6) in the `purchase` table:

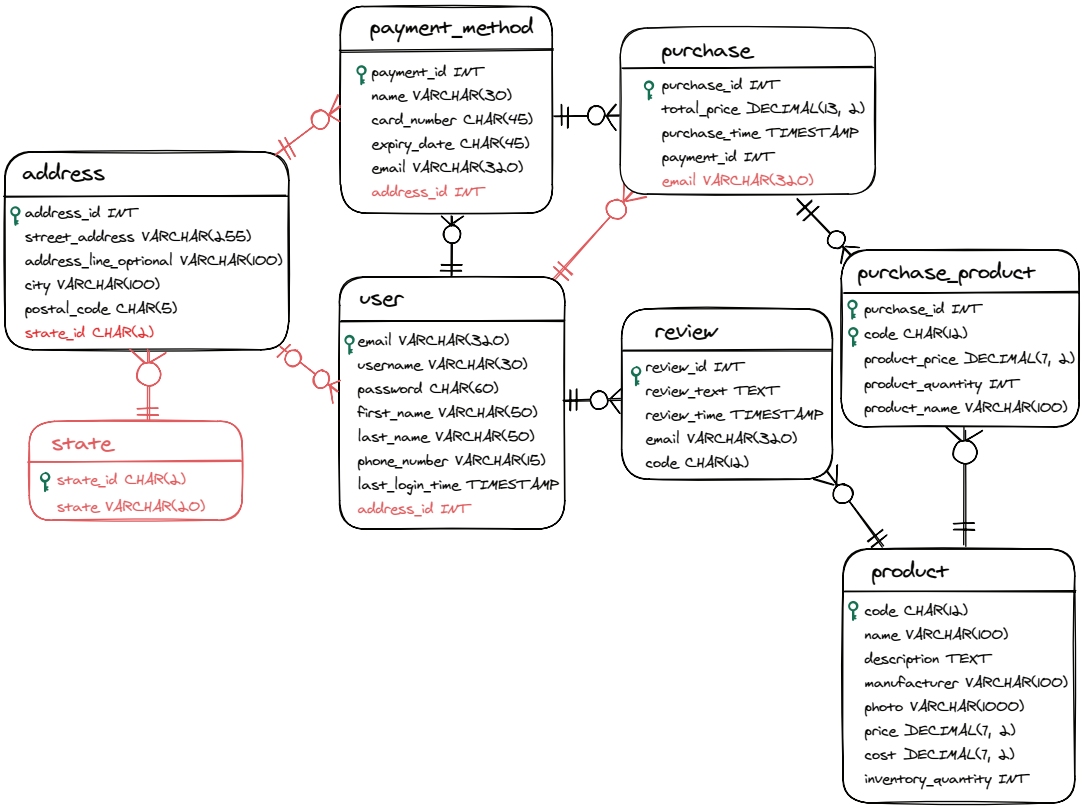


If you focus on the `purchase` table, you see a transitive dependency in which `purchase_id` determines `payment_id` and `payment_id` determines `email`.

Typically, you have to conduct a thorough cost-and-benefit analysis to evaluate whether the improved database performance is worth the trouble. A busy online store needs to generate receipts that frequently associate purchase data with a user, of course, and the direct

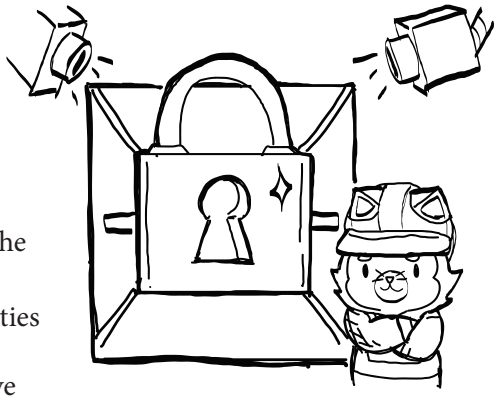
`user-purchase` relationship makes querying the two tables easier. Without it, you would have to write complex queries by joining three tables, which can be computationally expensive. Will there be any other difficult challenges, such as insertion or deletion anomalies? The Sci-Fi Collective doesn't allow users to change their email addresses; the `user` table even uses `email` as the primary key. Further, The Sci-Fi Collective doesn't allow people to make purchases without accounts. That said, no insertion or deletion anomalies will occur, and there's no chance that some purchase data will be added to the `purchase` table without being associated with an existing user. Overall, this scenario seems to be a perfect opportunity for denormalization.

Summarizing all the changes you've made so far, your E-R diagram looks like this:



If you have made it to this point in your journey through database design, you've done an amazing job! Each chapter in this book has built on the previous one, guiding you through the intricate processes that transform disconnected ideas and requirements into structured, efficient, powerful databases. Your perseverance in understanding and applying these principles of database design is a testament to your commitment and is truly commendable. Well done!

Recap



- You shouldn't save sensitive information in plain text. Instead, pick the right encryption method, encrypt the data, and ensure that the corresponding columns have enough room for the encrypted data in your database design.
- Beyond normalization, you may have opportunities to further improve the storage efficiency of your database design. These opportunities may involve redundant tables, columns, or categorical data. Be careful, though, because you don't want to negatively affect your database performance.
- Indexing can contribute to increasing the query performance of your database. You need to work closely with the application developers and use log data to understand where to apply indexes. If you aim to support queries that involve only sorting and exact-match searching, standard indexing is good enough. But if you aim to support queries that involve keyword searching, partial word matching, or fuzzy searching, you need full-text searching.
- Denormalization deliberately introduces redundancy to improve query efficiency. Denormalization is rare and requires you to be careful about handling data insertion, updates, and deletions in the same table.

Part 3
Database design and AI



Welcome to the future of database design! In this part, you'll discover how to enhance your database design process with the power of generative AI. You'll learn how to collaborate effectively with AI tools to accelerate your design workflow while maintaining control of the process.

Chapter 8 guides you through the complexities of database design in the age of generative AI. You'll explore practical ways to integrate AI into your design process, ensuring efficiency without sacrificing quality or control.

Prepare to sharpen your skills and broaden your knowledge as you create a fully functional database from the ground up.

Database design in the age of generative AI

8



In this chapter

- You explore how to use generative AI to speed database design.
- You learn how to fine-tune the prompts to get precisely what you want from generative AI tools.
- You go through the full pipeline to design and implement a database for a spaceship manufacturer with the help of generative AI tools.

What you need to know

This chapter is different from others in a major way. You are unlikely to replicate the demonstrated interactions with large language models (LLMs) due to their probabilistic nature and continuous changes. When we wrote this chapter, we used ChatGPT 4o. That said, ChatGPT may get your design exactly right or wrong in different ways even if you use the prompts demonstrated in this chapter.

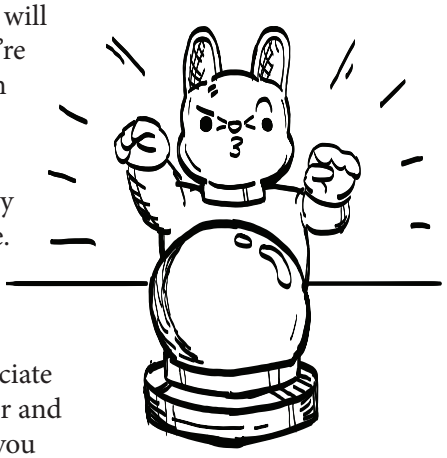
(continued)

The database design covered in this chapter (it's finally complete), using tools commonly used by practitioners, such as dbdiagram.io and MySQL Workbench, can be found in the GitHub repository (https://github.com/Neo-Hao/grokking-relational-database-design). Navigate to the `chapter_08` folder and follow the instructions in the `README.md` file to load the database design into corresponding tools.

Also, you can find the full-text prompts for ChatGPT and SQL scripts corresponding to the final database design for different relational database management systems (RDBMS), including MySQL, MariaDB, PostgreSQL, SQLite, SQL Server, and Oracle.

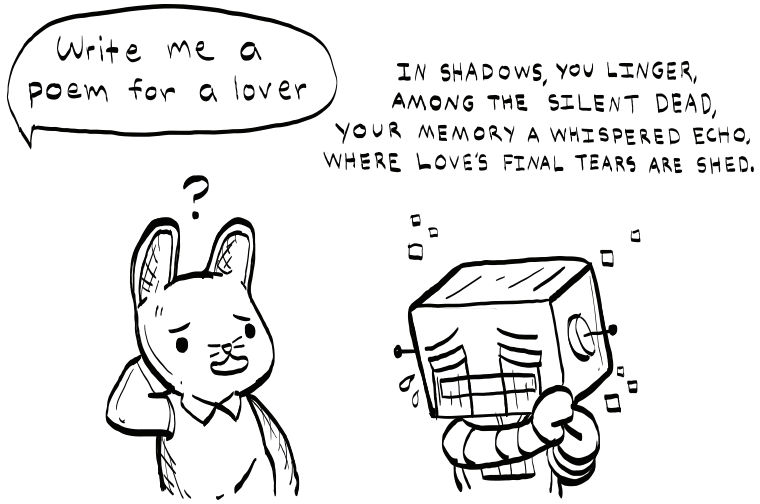
Overview

In this chapter, you will explore how to take advantage of generative AI to speed database design, and you will see what a typical workflow looks like when you're working toward a complete database design with help from generative AI.



You picked up all the essentials for designing a robust database in the previous seven chapters by working on the database of The Sci-Fi Collective. It is time for a change. In this chapter, you will design a database for a company that manufactures spaceships. By working on this new project from scratch, you will start to appreciate how generative AI tools can make your life easier and understand their limitations in delivering what you expect.

Quite a few LLMs can serve our purposes, such as ChatGPT, Claude, Google Gemini, and Llama. We will use ChatGPT (GPT-4o) to demonstrate our interactions with a generative AI tool in this chapter. Due to the way ChatGPT models were trained, specific prompt formats and practices work particularly well and lead to more useful responses.



Following are three rules of thumb for working with ChatGPT:

- *Be specific, descriptive, and as detailed as possible.* All LLMs, including ChatGPT, can't be seven steps ahead of you and know what you truly want to ask when your question or request is vague or imprecise. How ChatGPT responds to your questions and instructions is captured by the age-old adage "Garbage in, garbage out." This axiom may sound familiar if you are in the field of computer science or software engineering.
- *Separate instructions from context.* Most LLMs work better when you separate instructions from context. ChatGPT hopes that you'll put instructions at the beginning of a prompt and use `###` or `"""` to separate them from the context. When you need to summarize some long text passages in bullet points, for example, here is what ChatGPT recommends:

ChatGPT GPT-4o

Summarize the text below as a bullet-point list of the most important points.

Text: `"""`

some long texts go here

`"""`

- *Articulate the desired output format through examples.* ChatGPT responds better when you give it specific format requirements. This practice also makes it easier to programmatically parse out multiple outputs reliably. When you need to summarize some long text passages in a few target entities, here is what ChatGPT recommends:

ChatGPT GPT-4o

Extract the important entities mentioned in the text below. Extract all company names, people names, and specific topics which fit the content and the general overarching themes

Desired format:

Company names: `<comma_separated_list>`

People names: -||-

Specific topics: -||-

General themes: -||-

Text: """

some long texts go here

"""

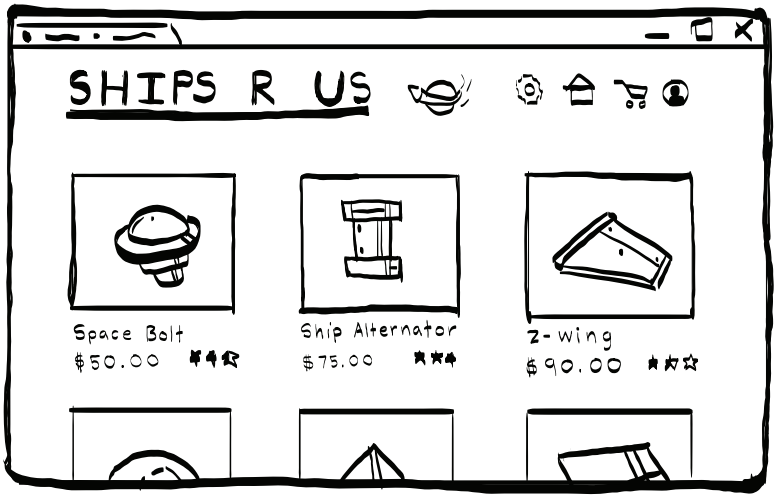
You can read more about recommended practices for using ChatGPT at https://mng.bz/QD41. In our experience, the three rules of thumb are most important and helpful for database design.

Before you start the database design, we want to emphasize that you shouldn't dwell on replicating the interactions with ChatGPT demonstrated in the chapter. The same request doesn't necessarily lead to the same answer for many reasons, including randomness, previous context, and model updates or refinements. Instead, focus on learning how to use ChatGPT efficiently, what working with ChatGPT looks like, and when to rely on your own judgment.

Requirements analysis

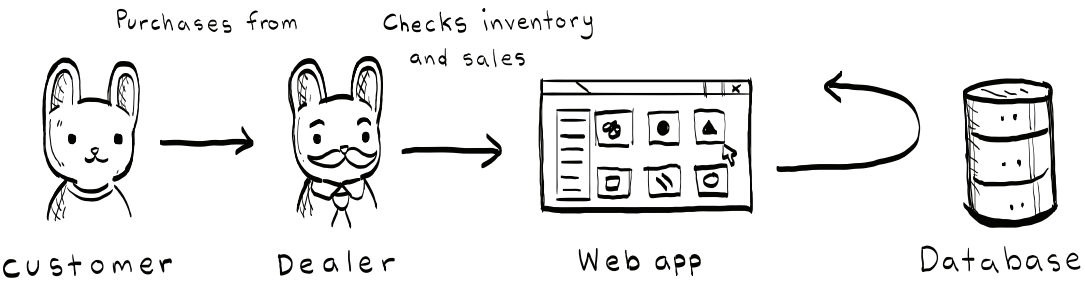
In line with the galactic theme, you will design a database for a company named SHIPS R US that specializes in manufacturing spaceships. SHIPS R US distributes spaceships in large batches

to dealers across the galaxy, who eventually sell the spaceships to customers. A good analogy is how Toyota and Ford sell cars. Like those companies, SHIPS R US requires a web application powered by a database to manage all the relevant information, such as spaceship models, production records, and dealer and customer information. Your task is to design a database for this web application.



For the sake of brevity, we'll assume that you've applied all the techniques you learned in chapter 3 to collect and analyze the requirements of the SHIPS R US database. The synthesized high-level requirements are

- The database needs to maintain records on all spaceship brands and models, manufactured spaceships, dealers, customers, and relationships between dealers and customers.
- A spaceship brand is associated with multiple models, and a model is associated with multiple spacecraft.
- One or more spaceships are distributed to a dealer, who serves one or more customers.
- A customer is served by one or more dealers and is associated with one or more spaceships that they bought from the dealer(s.)
- A spaceship is associated with one model and can be associated with no more than one owner and no more than one dealer.



These requirements focus on entities and their relationships. Beyond this summary, you have another set of requirements focusing on entities and attributes (explored further in the next section).

Entities and attributes

In this section, you design entities and attributes based on the requirement analysis with the help of ChatGPT.

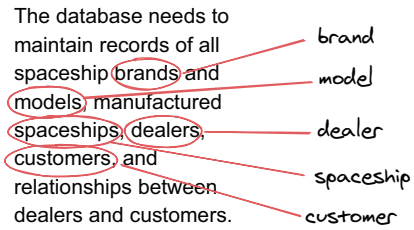
Entities

Based on the high-level requirement summary, you extracted five entities: brand, model, spaceship, dealer, and customer. If this fact is not obvious to you or you want to see how capable ChatGPT is, you can make the following request to ChatGPT:

> I am designing a database. Help me identify all the entities based on the given requirements.
>
> **Requirements:** """
>
> The database needs to maintain records of all spaceship brands and models, manufactured spaceships, dealers, customers, and relationships between dealers and customers.
>
> A spaceship brand is associated with multiple models, and a model is associated with multiple spacecrafts.
>
> A dealer is distributed one or more spaceships and serves one or more customers.
>
> A customer is served by one or more dealers and is associated with one or more spaceships that they bought from the dealers.

```
A spaceship is associated with one model and can be associated with no
more than one owner and no more than one dealer.
"""
```

As you can see from this request, we deliberately separated the instructions from the context to get better answers from ChatGPT. This question is so simple that separating instructions from context does very little, but it is a good idea to stick to the rules of thumb, and it will make a difference when you have a much more complicated question or request. Overall, the answer you get from ChatGPT will be similar to the one shown in this image:



Attributes

You worked hard on requirement analysis and have a set of synthesized requirements to help you identify the attributes of each entity:

- *A brand has a name.* The number of brands is limited and smaller than five.
- *A model has a four-digit number and name.* The number is unique per model.
- *A spaceship has a 17-digit serial number (like a vehicle identification number for a car), a recommended price, and a year number.* The serial number is unique per spaceship. The unit for the recommended price is US dollars (USD). The year number represents the year in which the spaceship was manufactured.
- *A dealer has a name, address, phone number, email address, and dealer website URL.* The dealer may be anywhere on Earth.
- *A customer has a name, address, phone number, and email address.* Multiple customers may share an address. The customer may live anywhere on Earth.

Developing the entity-relationship (E-R) diagram is a milestone in database design, requiring you to identify the attributes and their corresponding data types as well as the primary key for each entity. This step can be tedious and requires you to dig deep into the details of the gathered requirements. ChatGPT can offer help. You could use the preceding requirements for attributes to formulate the following request:

I am designing a database for a manufacturer of spaceships. I have identified five entities: brand, model, spaceship, dealer, and customer. Help me identify attributes and the primary key per entity based on the given requirements. Generate output using the desired format.

Desired format:

Entity name: `spaceship`

Attributes:

Serial number: `CHAR(17)`

Recommended price: `NUMERIC(8, 2)`

Primary key: Serial number

Requirements: `"""`

A brand has a name …… The customer may live anywhere on Earth.

(This is an excerpt. Full text can be found at https://bit.ly/grdb.)

`"""`

As you can see from this request, we added the desired format for the output, which increases the consistency of the output and makes sure that it is ready to use for future interactions with ChatGPT. Following is a sample answer from ChatGPT to this request:

ChatGPT GPT-4o

Entity name: `Brand`

Attributes:

Name: `VARCHAR(100)`

Primary key: `Name`
Entity name: `Model`
Attributes:
Model number: `CHAR(4)`
Model name: `VARCHAR(100)`
Primary key: Model number

Entity name: `Spaceship`
Attributes:
Serial number: `CHAR(17)`
Recommended price: `NUMERIC(8, 2)`
.....

As soon as you see the answer, you may realize that you missed something in formulating your request: naming the entities and attributes properly. The names of entities and attributes don't follow the naming conventions we discussed in chapter 3. To fix that problem, you can ask ChatGPT a follow-up question or reformulate your initial question. Regardless of your choice, we recommend that you give ChatGPT an example of how the output should be formatted. Here is an example follow-up request:

Regenerate your answer using a new format but the same requirements. Make sure that names of entities and attributes are all snake-cased.
New format:
Entity name of Spaceship: `spaceship`
Attributes:
Serial number: `serial_number` – `CHAR(17)`
Recommended price: `recommended_price` – `NUMERIC(8, 2)`
Primary key: `serial_number`

Here is a sample answer from ChatGPT to this request:

ChatGPT GPT-4o

Entity name of Brand: `brand`
Attributes:
Name: `name` – `VARCHAR(100)`
Primary key: `name`

Entity name of Model: `model`
Attributes:
Model number: `model_number` – `CHAR(4)`
Model name: `model_name` – `VARCHAR(100)`
Primary key: `model_number`

Entity name of Spaceship: `spaceship`
Attributes:
Serial number: `serial_number` – `CHAR(17)`
Recommended price: `recommended_price` – `NUMERIC(8, 2)`
Year: `year` – `year`
Primary key: `serial_number`

Entity name of Dealer: `dealer`
Attributes:
Name: `name` – `VARCHAR(100)`
Address: `address` – `VARCHAR(255)`
Phone number: `phone_number` – `CHAR(15)`
Email address: `email_address` – `VARCHAR(100)`
Website URL: `website_url` – `VARCHAR(255)`
Primary key: `name`

Entity name of Customer: `customer`

Attributes:

Name: `name` – `VARCHAR(100)`

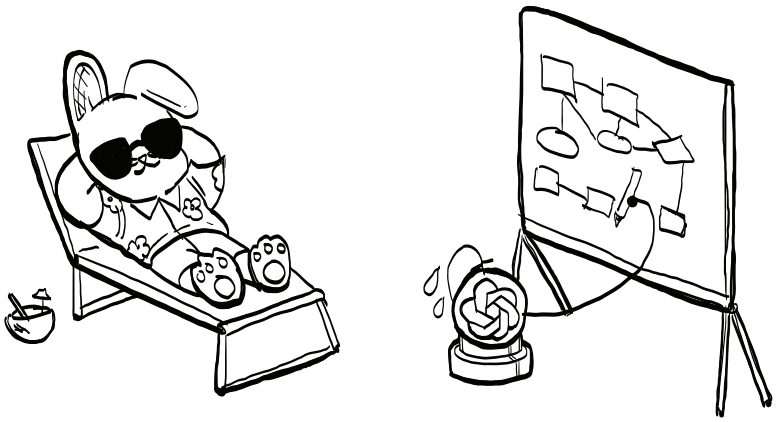
Address: `address` – `VARCHAR(255)`

Phone number: phone`_number` – `CHAR(15)`

Email address: `email_address` – `VARCHAR(100)`

Primary key: `email_address`

As you see, ChatGPT does many things right, such as using the desired format, following snake-case convention in naming entities and attributes, and making some reasonable guesses on data length when no requirements are specified.



Some design choices in the answer are questionable, however. Three notable problems stand out.

First, the selection of primary keys of certain entities may not be what you want. Because the requirement summaries don't provide enough information about how primary keys should be selected, ChatGPT took the liberty of sneaking in some assumptions. The `name` attribute was chosen as the primary key of the `dealer` entity, for example, and the `email_address` attribute was chosen as the primary key of the

customer entity. Both choices are problematic. Although a dealer's name is not likely to change often, it is possible for two dealers to have the same name, even in a single US state, let alone the world. A different attribute should be used as the primary key, but what would that be? After diving into the requirement analysis further, you find the following information:

- A customer is allowed to update their email address and phone number.
- A dealer is allowed to update their email address and phone number.

In other words, neither the email nor the phone number is stable. If you make either attribute the primary key of an entity, you will face a lot of trouble in updating phone number or email addresses. That said, no single attributes are readily available to be used as the primary key in dealer or customer. As you learned in chapter 4, a surrogate key that increments automatically can be used as the primary key in this situation. Such a primary key has no problems in identifying a data record uniquely but can't prevent data duplications by itself—that is, it can't prevent a person or a robot to register more than one account using the same information. Based on what you learned in chapter 6, you can add a unique constraint to the email_address attribute and another to the phone_number attribute to address concerns about data duplication. When you revise the design of the customer entity based on these considerations, it looks like this:

Entity name of Customer: customer

Attributes:

Customer ID: customer_id – INT

Name: name – VARCHAR(100)

……

Primary key: customer_id

Note 1: email_address requires a unique constraint; phone_number requires a unique constraint

The `dealer` entity has a similar update:

Entity name of Dealer: `dealer`

Attributes:

Dealer ID: `dealer_id` – `INT`

Name: `name` – `VARCHAR(100)`

……

Primary key: `dealer_id`

……

Second, the design of phone-number attributes failed to account for the fact that a dealer or customer can be anywhere on Earth. Both `customer` and `dealer` have a `phone_number` attribute. The data length `CHAR(15)` may not be sufficient for all international numbers. If you do some research on phone-number lengths, you find that the maximum length of a phone number is 15 digits, thanks to the international phone-numbering plan. This length assumes that the country code is included and that the number is a simple numeric string without spaces, dashes, or parentheses. Some international numbers, however, have non-numeric strings. In such a case, the total length may exceed 15 characters. To address this problem, you can separate the country code from the phone number and prepare two attributes to store each piece of data separately. When you revise the design of the `customer` entity based on these considerations, it looks like this:

Entity name of Customer: `customer`

Attributes:

Customer ID: `customer_id` – `INT`

Name: `name` – `VARCHAR(100)`

Phone number:

```
    country_code - CHAR(5)
    customer_phone_number - CHAR(15)
```

……

The `dealer` entity has a similar update:

Entity name of Dealer: `dealer`

Attributes:

Dealer ID: `dealer_id` – `INT`

Name: `name` – `VARCHAR(100)`

Phone number:

```
    country_code - CHAR(5)
    dealer_phone_number - CHAR(15)
```

……

The separation between country codes and phone numbers can help standardize phone numbers and ensure consistency in how phone numbers are stored. More important, `VARCHAR(15)` is sufficient for any phone numbers when the country code is not considered.

Third, the design of address attributes failed to account for the fact that the dealer and customer can be anywhere on Earth. A single attribute, `VARCHAR(255)`, may not be sufficient for storing some long addresses. What's worse, this design will lead to many addresses with unpredictable formats because no structure is enforced. Unpredictable formats create further troubles in parsing, searching, and filtering, let alone data validation and error handling.

To make address storage more robust and flexible, consider breaking the address into multiple attributes. You might update the design of the `customer` entity like this:

Entity name of Customer: `customer`

Attributes:

……

Address:

```
    street_address - VARCHAR(255)
    suburb - VARCHAR(100)
    city - VARCHAR(100)
    state_province - VARCHAR(100)
    postal_code - VARCHAR(20)
         country - CHAR(2)
```

……

The design of the `dealer` entity is updated as follows:

Entity name of Dealer: `dealer`

Attributes:

……

Address:

```
   street_address - VARCHAR(255)
    suburb - VARCHAR(100)
    city - VARCHAR(100)
    state_province - VARCHAR(100)
    postal_code - VARCHAR(20)
    country - CHAR(2)
```

……

As soon as you break the address information into six attributes in both the `customer` and `dealer` entities, you may notice that the two entities suddenly have significant overlap. This situation may ring a bell if you remember chapter 7, which addressed a similar problem between a user address and a billing address. If you use the same solution, you can prepare a new table representing addresses and link that table to both the `dealer` and `customer` entities. The new table looks like this:

Entity name of Address: `address`

Attributes:

Address ID: `address_id` – `INT`

Street address: `street_address` – `VARCHAR(255)`

Suburb: `suburb` – `VARCHAR(100)`

City: `city` – `VARCHAR(100)`

State/Province: `state_province` – `VARCHAR(100)`

Postal code: `postal_code` – `VARCHAR(20)`

Country: `country` – `CHAR(2)`

Primary key: `address_id`

The postal codes are not universally applicable. Some countries use them, and others don't. Countries that use postal codes tend to have different opinions about how they should be formatted. As a result, you should make the `postal_code` attribute optional, which means that

it can be `null`. The same can be said of the `suburb` attribute. With the `address` entity representing addresses, you can remove the attributes about addresses from the `customer` and `dealer` entities.

It is important to keep a record of your revisions in text because you are not done designing and implementing the database, and ChatGPT is a primarily a text-based AI tool. You'll feed your design of entities and attributes to ChatGPT again for future design revisions and implementations. When you merge all your revisions with the answers from ChatGPT, you get the following results:

Entity name of Brand: `brand`
Attributes:
Name: `name` – `VARCHAR(100)`
Primary key: `name`

Entity name of Model: `model`
Attributes:
Model number: `model_number` – `CHAR(4)`
Model name: `model_name` – `VARCHAR(100)`
Primary key: `model_number`

Entity name of Spaceship: `spaceship`
Attributes:
Serial number: `serial_number` – `CHAR(17)`
Recommended price: `recommended_price` – `NUMERIC(8, 2)`
Year: `year` – `year`
Primary key: `serial_number`

Entity name of Dealer: `dealer`
Attributes:
Dealer ID: `dealer_id` – `INT`
Name: `name` – `VARCHAR(100)`

Email address: `email_address` – `VARCHAR(100)`

Website URL: `website_url` – `VARCHAR(255)`

Phone number:

```
country_code - CHAR(5)
dealer_phone_number - CHAR(15)
```

Primary key: `dealer_id`

Note 1: `email_address` requires a unique constraint; (`country_code`, `phone_number`) requires a unique constraint

Entity name of Customer: `customer`

Attributes:

Customer ID: `customer_id` – `INT`

Name: `name` – `VARCHAR(100)`

Email address: `email_address` – `VARCHAR(100)`

Phone number:

```
country_code - CHAR(5)
customer_phone_number - CHAR(15)
```

Primary key: `customer_id`

Note 1: `email_address` requires a unique constraint; (`country_code`, `phone_number`) requires a unique constraint

Entity name of Address: `address`

Attributes:

Address ID: `address_id` – `INT`

Street address: `street_address` – `VARCHAR(255)`

Suburb: `suburb` – `VARCHAR(100)`

City: `city` – `VARCHAR(100)`

State/Province: `state_province` – `VARCHAR(100)`

Postal code: `postal_code` – `VARCHAR(20)`

Country: `country` – `CHAR(2)`

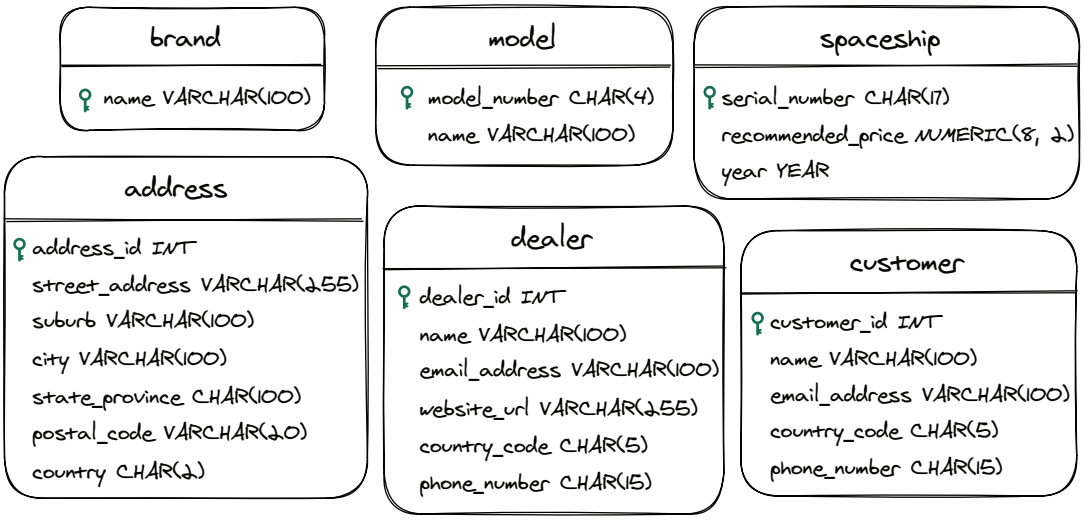
Primary key: `address_id`

Note 1: `suburb` and `postal_code` are optional

Communicate your design

You have completed the design of entities and attributes based on the requirement analysis and ChatGPT. Everything is stored in a long, formatted piece of text. The text is essential for communicating with ChatGPT but not great for sharing your design with other people. To communicate your design effectively, you have to visualize it.

If you have only a few entities, you can use any diagram tool to draw them. You can draw all the entities of SHIPS R US as follows:



If you are working on a much bigger project that involves a dozen or more entities, however, you may want to turn to ChatGPT for help again. Although ChatGPT can't generate the diagram you need in the target format, you can ask it to generate the corresponding SQL code and import the code into a tool that visualizes database design based on SQL code, such as dbdiagram.io (https://dbdiagram.io). To ask ChatGPT for SQL code, try the following example request:

I am designing a database. Help me generate MySQL code based on the following description of entities and attributes. Keep the entities independent from one another.

Description: """

Entity name of Brand: `brand`

Attributes:

Name: `name` – `VARCHAR(100)`

Primary key: `name`

.....

Country: `country` – `CHAR(2)`

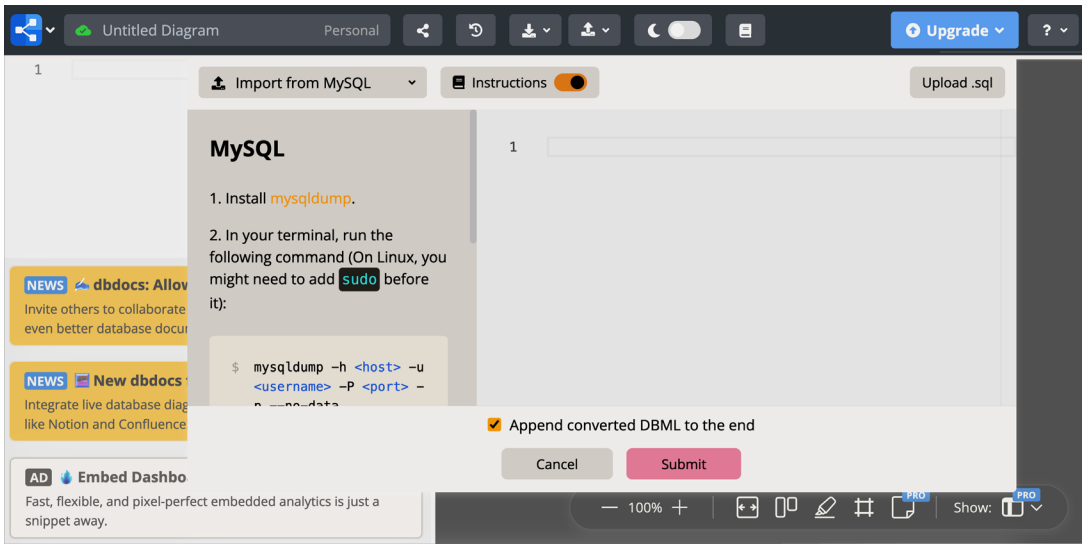
Primary key: `address_id`

Note 1: `suburb` and `postal_code` are optional

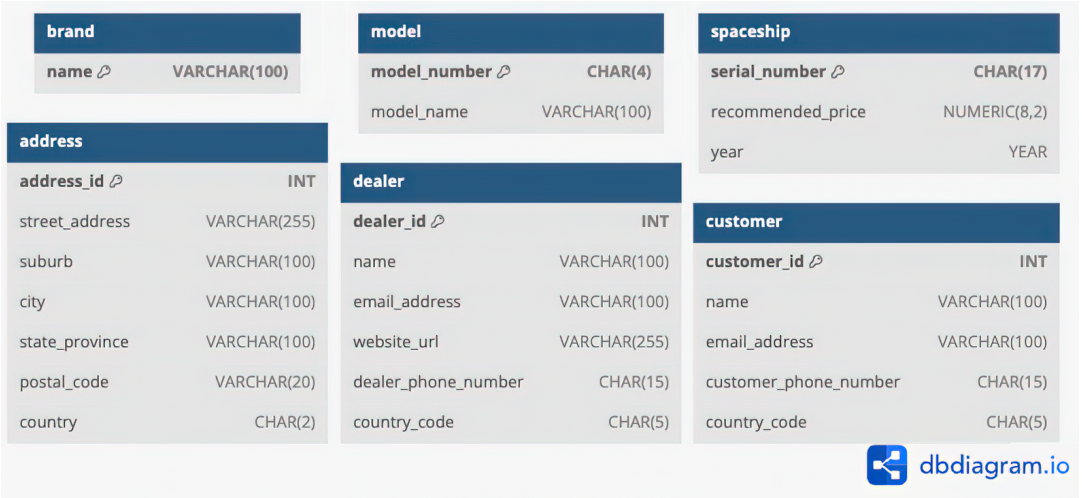
(This is an excerpt. Full text can be found at https://bit.ly/grdb.)

"""

After getting the code, you can copy and paste it by using the import function of dbdiagram.io:



The generated visualization is much easier to share with others and appropriate for communication:



As you can see, ChatGPT can speed the process of database design. Although you can't use the answers directly, they are useful as the starting point in your problem-solving journey. When you get some answers from ChatGPT, it is up to you to identify the problems in those answers and fix them based on your requirement analysis and knowledge of database design. When you need to communicate your design to others, ChatGPT can convert the text-based design to SQL code, which in turn can be turned into a diagram.

Relationships among entities

Now that you have identified and designed all the entities, their attributes, and corresponding data types for the database, you are ready to work on the relationships among entities. The following list may refresh your memory of the requirements for relationships:

- The database needs to maintain records of all spaceship brands and models, manufactured spaceships, dealers, customers, and relationships between dealers and customers.

- A spaceship brand is associated with multiple models, and a model is associated with multiple spacecraft.
- A dealer is distributed one or more spaceships and serves one or more customers.
- A customer is served by one or more dealer, and is associated with one or more spaceships that they bought from the dealer.
- A spaceship is associated with one model and can be associated with no more than one owner and no more than one dealer.
- A customer is associated with one address. Multiple customers may share an address. A dealer is associated with one address.

You added the last item when you decided to use an independent entity to represent addresses. You need to address the relationship between customers and addresses as you build the relationships among entities; the same is true of dealers and addresses.

As what you did in the preceding section, you can start by asking ChatGPT to generate a design draft. To achieve a more useful response, follow the rules of thumb to structure your request. Try the following request:

> I am designing a database for a manufacturer of spaceships. I have identified six entities: brand, model, spaceship, dealer, customer, and address. Help me identify the relationships between each pair of entities based on the given requirements. Generate output using the desired format.
>
> **Desired format:**
>
> **brand | model**
>
> **Relationship:** one to many
>
> **Note:** The side pointing to model is many
>
> **Requirements:** """
>
> The database needs to maintain records of all spaceship brands and models …… A dealer is associated with one address.
>
> (This is an excerpt. Full text can be found at https://bit.ly/grdb.)
>
> """

Here is a sample response from ChatGPT:

ChatGPT GPT-4o

brand | model
Relationship: one to many
Note: the side pointing to model is many

model | spaceship
Relationship: one to many
Note: The side pointing to spaceship is many

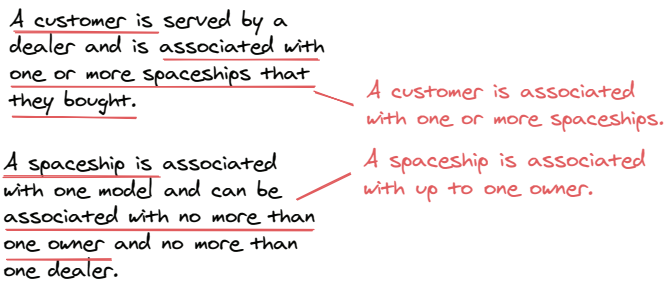
spaceship | dealer
Relationship: many to one
Note: The side pointing to dealer is one

spaceship | customer
Relationship: many to one
Note: A customer can own multiple spaceships, and a spaceship can be owned by multiple customers (e.g., shared ownership)

dealer | address
Relationship: one to one
Note: Each dealer is associated with exactly one address

customer | address
Relationship: many to one
Note: The side pointing to address is one

As in the design of entities and attributes, ChatGPT does many things right. In the preceding answer, most relationships were developed properly with the exception of the relationship between `customer` and `spaceship`. The requirements you gathered were quite explicit:



It is possible that when the information on a relationship is scattered across multiple sentences, ChatGPT has difficulty parsing it properly. It is also possible that ChatGPT somehow starts to hallucinate when answering this question. Regardless, you have a robust mastery of database design after reading the first seven chapters of this book, and the error is not hard to fix:

spaceship | customer
Relationship: many to one
Note: the side pointing to customer is one

You may also notice that the relationship between `customer` and `dealer` is missing. The requirement you gathered is clear:

- A dealer serves one or more customers.
- A customer is served by a dealer.

You can add this relationship yourself to the text of relationships:

customer | dealer
Relationship: many to many
Note: both sides are many

Minimum cardinalities and foreign keys

ChatGPT didn't provide much information about the min cardinalities or foreign keys of relationships between pairs of entities. You didn't ask for this information to begin with because you needed to make sure that each relationship (max cardinalities) was identified properly first. Now

is a good time to make a new request to ChatGPT. Here is an example of how you might structure this request:

> I am designing a database for a manufacturer of spaceships. I have identified some of the characteristics of six entities: brand, model, spaceship, dealer, customer, and address. Help me identify the min cardinality and where the foreign key should be placed between each pair of entities based on the given requirements and the known relationship characteristics. Generate output using the desired format.
>
> **Desired format:**
>
> **brand | model**
>
> **Relationship:** one to many
>
> **Note 1:** The side pointing to model is many
>
> **Note 2:** The foreign is in model
>
> **Note 3:** The min cardinality close to model is relaxed from 1 to 0
>
> **Requirements:** """
>
> The database needs to maintain records of all spaceship brands and models …… A dealer is associated with one address.
>
> (This is an excerpt. Full text can be found at https://bit.ly/grdb.)
>
> """
>
> **Characteristics:** """
>
> **brand | model**
>
> **Relationship:** one to many
>
> **Note:** The side pointing to model is many
>
> ……
>
> **customer | address**
>
> **Relationship:** many to one
>
> **Note:** The side pointing to address is one
>
> (This is an excerpt. Full text can be found at https://bit.ly/grdb.)
>
> """

Following is a sample answer from ChatGPT to this request:

ChatGPT GPT-4o

brand | model

Relationship: one to many

Note 1: The side pointing to model is many

Note 2: The foreign key is in model

Note 3: The minimum cardinality close to model is relaxed from 1 to 0

model | spaceship

Relationship: one to many

Note 1: The side pointing to spaceship is many

Note 2: The foreign key is in spaceship

Note 3: The minimum cardinality close to spaceship is relaxed from 1 to 0

customer | dealer

Relationship: many to many

Note 1: A junction table `customer_dealer` is needed

Note 2: The foreign keys are in `customer_dealer`

Note 3: The minimum cardinalities close to `customer_dealer` are relaxed from 1 to 0

spaceship | dealer

Relationship: many to one

Note 1: The side pointing to dealer is one

Note 2: The foreign key is in spaceship

Note 3: The minimum cardinality close to dealer is relaxed from 1 to 0

spaceship | customer

Relationship: many to one

Note 1: The side pointing to customer is one

Note 2: The foreign key is in spaceship

Note 3: The minimum cardinality close to spaceship is relaxed from 1 to 0

(continued)

dealer | address

Relationship: one to one

Note 1: Each dealer is associated with exactly one address

Note 2: The foreign key is in dealer

Note 3: The minimum cardinality close to address is 1

customer | address

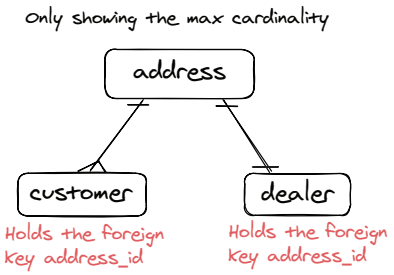
Relationship: many to one

Note 1: The side pointing to address is one

Note 2: The foreign key is in customer

Note 3: The minimum cardinality close to address is relaxed from 1 to 0

ChatGPT handled the placement of foreign keys quite well, especially when three entities are related. The relationships among `customer`, `dealer`, and `address` are similar to a challenge you tackled in earlier chapters: how to handle the relationships among address, user, and payment methods for The Sci-Fi Collective. The relationships among `customer`, `dealer`, and `address` are summarized as follows:



Beyond foreign keys, ChatGPT is rather confused about the min cardinalities, especially when a min cardinality needs to be relaxed from one to zero. It is up to you to fix those mistakes, using what you learned from chapter 5. If your memory is blurred, here are some rules of thumb about when to relax a min cardinality:

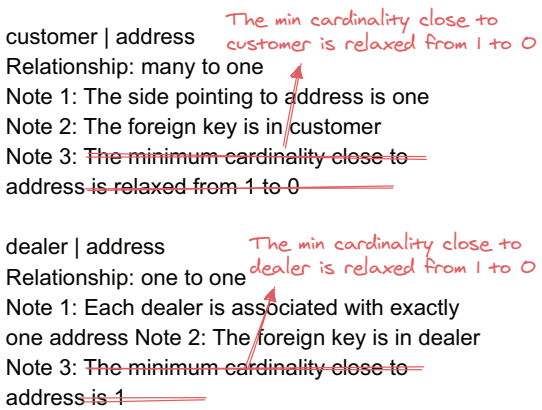
- In a one-to-one relationship, if one entity is typically created before the other, you may set the min cardinality of the second entity to zero to allow the first entity to exist without an immediate counterpart.

- In a one-to-many relationship, the cardinality that's closer to the many side needs to be relaxed. This relaxation allows for cases in which the entity on the one side might exist without any corresponding entries on the many side. A customer might exist without any addresses, for example.
- In a many-to-many relationship, the min cardinalities are often set to zero on both sides if either entity can exist without the other. If there are strict business rules about the existence of the entities, however, you might keep one cardinality set to one.

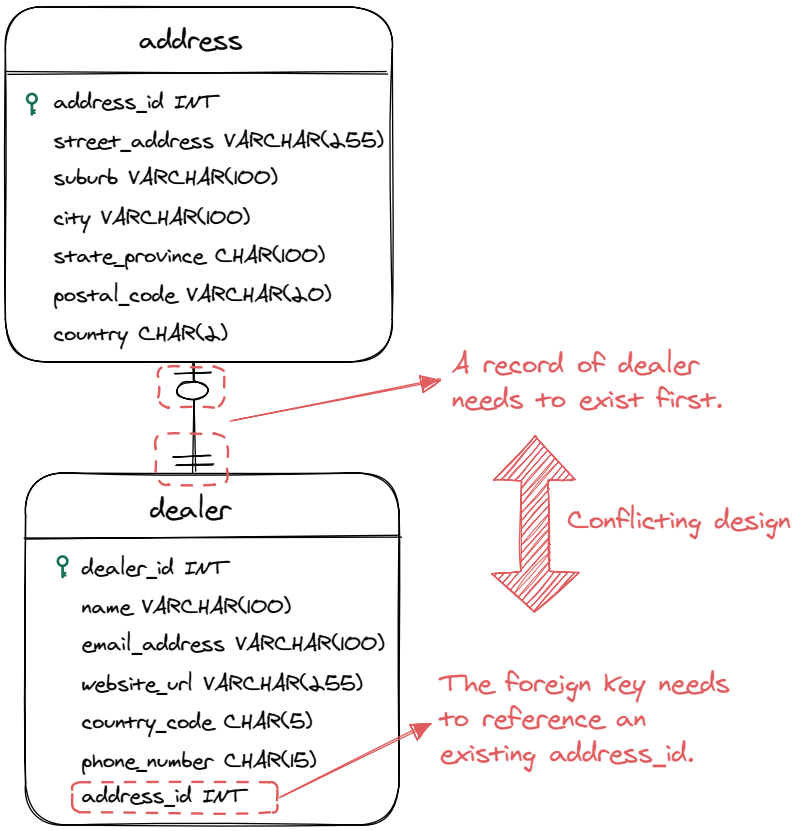
You can start with the relationships between `dealer` and `address` and between `customer` and `address`. The gathered requirements don't say much about the min cardinalities of their relationships. After following up with the software developers, you gained some new understanding:

- *A dealer can't have an account without an address.* SHIPS R US's web app doesn't allow dealers to register accounts without providing address information. In other words, it is not possible for a dealer record to exist without a corresponding address.
- *SHIPS R US maintains user records for purposes such as outreach and recalls but doesn't allow customers to register accounts directly.* Instead, dealers enter the customer information plus the address information into the system. In other words, it is not possible for a customer record to exist without a corresponding address.

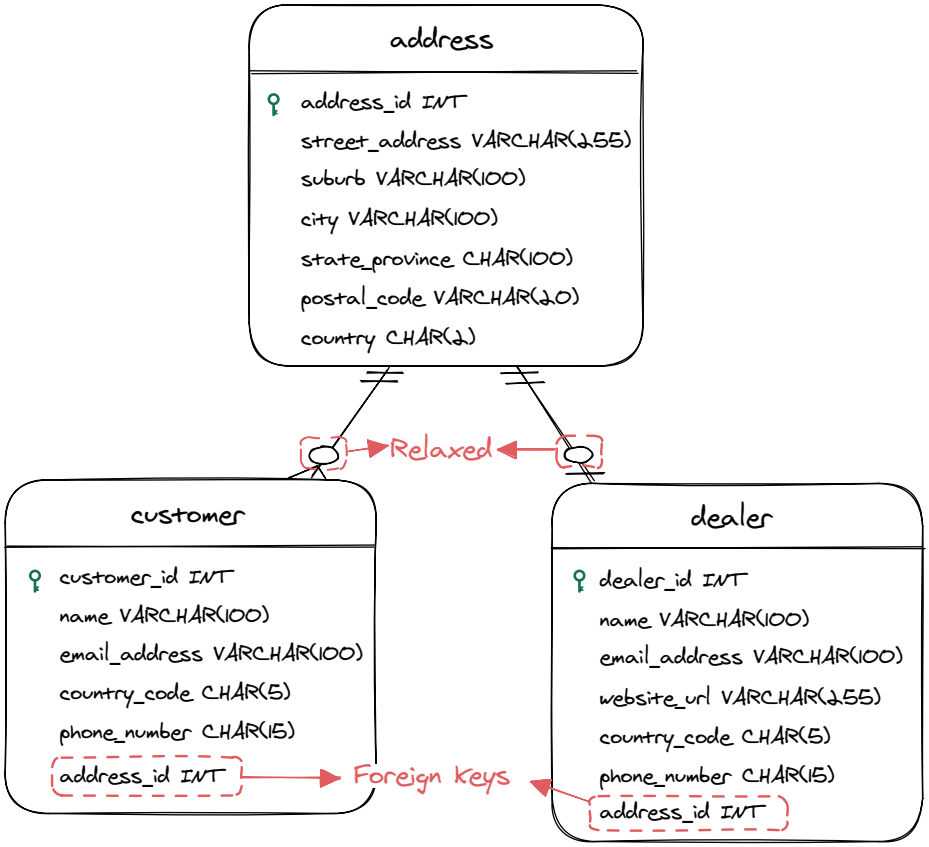
Combining such information and rules of thumb about when to relax a min cardinality, you can make the following revisions:



The revision to the relationship between `customer` and `address` is straightforward considering that the relationship is one-to-many. The min cardinality of the many side needs to be relaxed to make data entry possible. By contrast, the revision to the relationship between `dealer` and `address` is more nuanced. You may wonder whether you can relax the side closer to `address` instead of `dealer`. You can, but doing so will make things messier and less consistent, as demonstrated in the following figure:



Because `dealer` is the child table in this relationship, it contains the foreign key `address_id`. In other words, a new record in the `dealer` table needs to reference an existing `address_id` value. As the result, relaxing the cardinality closer to `dealer` is a better choice. The revised relationships between `customer` and `address` and between `dealer` and `address` can be summarized as follows:



Beyond the two relationships, you can find one more min cardinalities that need to be relaxed in the relationship between `spaceship` and `customer`. The requirements for this relationship are

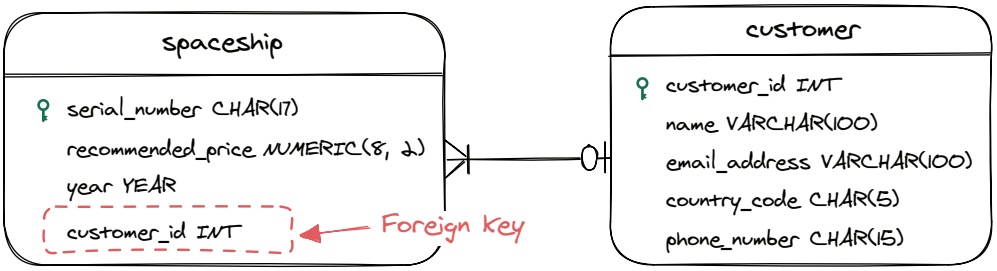
- A spaceship is associated with one model and can be associated with no more than one owner and no more than one dealer.
- A customer is associated with one or more spaceships that they bought.

In other words, when a spaceship is manufactured, its record is entered into the system immediately without being associated with a customer. Based on this information, the min cardinality close to `customer` rather than `spaceship` needs to be relaxed to zero:

spaceship | customer
Relationship: many to one
Note 1: The side pointing to customer is one
Note 2: The foreign key is in spaceship
Note 3: ~~The minimum cardinality close to spaceship is relaxed from 1 to 0~~

The min cardinality close to customer is relaxed from 1 to 0

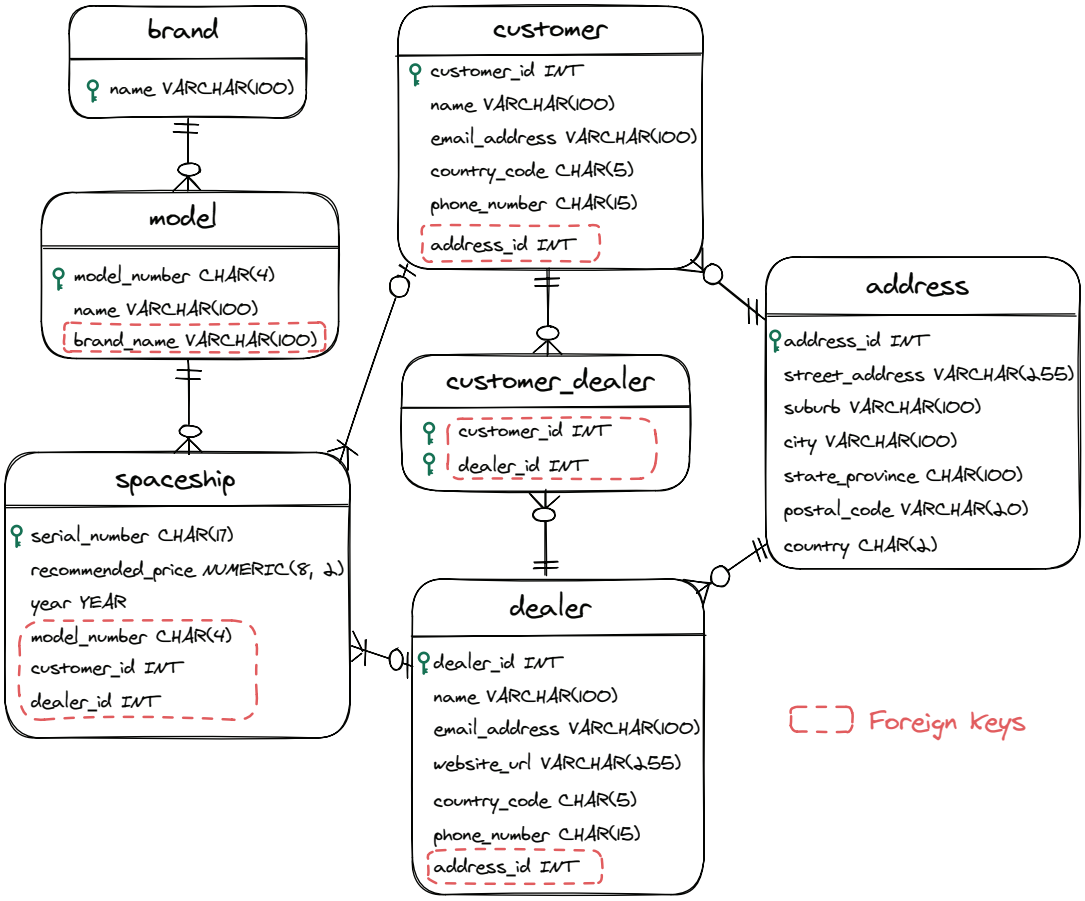
The revised relationship can be summarized as follows:



You may notice that the foreign key of this relationship (`customer_id`) is in `spaceship`. By default, `customer_id` needs to reference a record in `customer`. To make sure that the foreign key is aligned with the requirements, make it optional. In other words, it should allow `NULL` values.

Communicate your design

As discussed earlier in this chapter in the "Entities and attributes" section, you are likely to need visualization to communicate your design better. If you choose to draw your design by hand, you'll get something like the following figure.



If you need ChatGPT and dbdigram.io for a much bigger project, you must retrieve the SQL code first. The following request aims to get MySQL code from ChatGPT:

I am designing a database. Help me generate MySQL code based on the following description of entities and relationships.

Description: """

description of entities starts here

Entity name of Brand: `brand`

Attributes:

Name: name – `VARCHAR(100)`

Primary key: name

……

description of relationships starts here

brand | model

Relationship: one to many

Note 1: The side pointing to model is many

……

Country: `country` – `CHAR(2)`

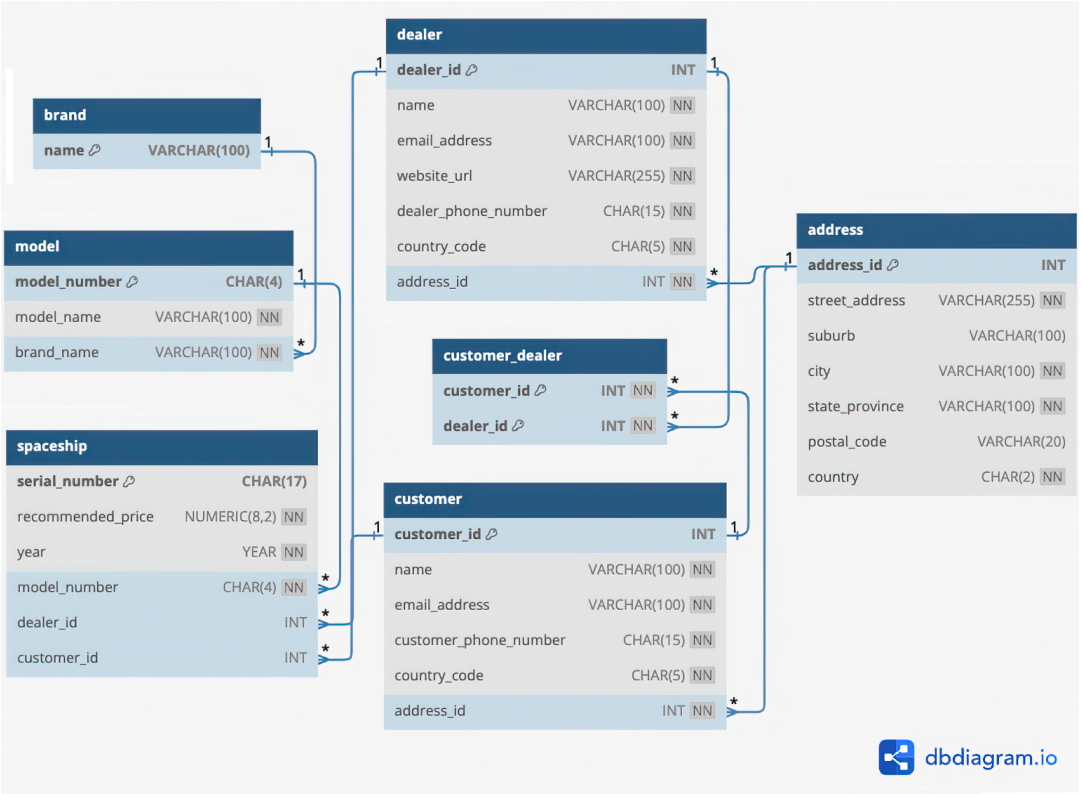
Primary key: `address_id`

Note 1: `suburb` and `postal_code` are optional

(This is an excerpt. Full text can be found at https://bit.ly/grdb.)

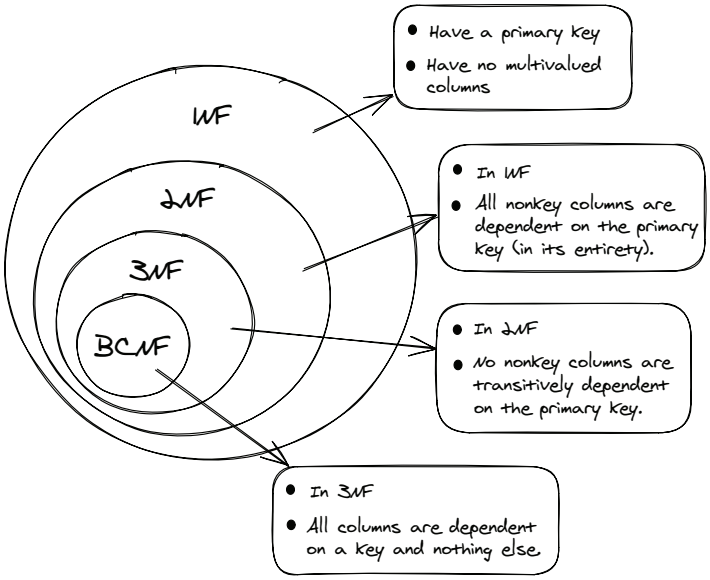
"""

After getting the code, you can copy and paste it into dbdiagram.io to get the following diagram:



Implementation

Now that you designed the entities, attributes, and relationships for the database of SHIPS R US, you are ready to implement the database. First, however, you have to make sure that your database design is normalized. When you check your database design against the requirements for normal forms, you see that your database design is in Boyce-Codd Normal Form (BCNF). You can also ask ChatGPT to help with the checking ("Is my database design normalized?") and verify the answers later.



Constraints

Applying constraints based on requirement analysis is a necessary step in implementing your database design. In chapter 6, you learned about a variety of constraints, such as primary key, foreign key, `NOT NULL`, and unique constraints. Review your analysis to identify the attributes that require constraints and the type of constraints that are appropriate. ChatGPT can offer help for this task as well. Copy and paste all the relevant requirements and the revised output from the preceding two sections and then formulate a new request:

> I am designing a database. Help me identify the attributes that need constraints and the type of constraints using the desired format based on the given requirements, entities and attributes, and relationships among entities.
>
> **Desired formats:**
>
> **Entity name:** `brand`
>
> **Attribute in focus:**
>
> * `model_number`: primary key
> * `model_name`: `NOT NULL`
> * `brand_name`: foreign key

Requirements: """
The database needs to maintain records of all spaceship brands and models, manufactured ……
(This is an excerpt. Full text can be found at https://bit.ly/grdb.)
"""

Entity and attributes: """
Entity name of Brand: `brand`
Attributes:
Name: `name` – `VARCHAR(100)` .....
(This is an excerpt. Full text can be found at https://bit.ly/grdb.)
"""

Relationships: """
brand | model
Relationship: one to many
Note 1: The side pointing to model is many……
(This is an excerpt. Full text can be found at https://bit.ly/grdb.)
"""

Here is a sample answer to this request:

ChatGPT GPT-4o

Entity: `Brand`
Attribute in focus:
name: primary key

Entity: `Model`
Attributes in focus:
`model_number`: primary key constraint
`model_name`: `NOT NULL`
`brand_name`: foreign key constraint (references `brand.name`)

(continued)

Entity: `Spaceship`
Attributes in focus:
`serial_number`: primary key
`recommended_price`: `CHECK` (e.g., `CHECK (recommended_price >= 0)`)
`year`: `CHECK` (e.g., `CHECK (year >= 1900)`)
`model_number`: foreign key (references `model.model_number`)
`dealer_id`: foreign key (references `dealer.dealer_id`)
`customer_id`: foreign key (references `customer.customer_id`)

Entity: `Dealer`
Attributes in focus:
`dealer_id`: primary key
`name`: `NOT NULL`
`email_address`: `UNIQUE`
`website_url`: `NOT NULL`
(`country_code`, `dealer_phone_number`): `UNIQUE`
`address_id`: foreign key (references `address.address_id`)

Entity: `Customer`
Attributes in focus:
`customer_id`: primary key
`name`: `NOT NULL`
`email_address`: `UNIQUE`
(`country_code`, `customer_phone_number`): `UNIQUE`
`address_id`: foreign key (references `address.address_id`)

Entity: `Customer_Dealer`
Attributes in focus:
(`customer_id`, `dealer_id`): primary key
`customer_id`: foreign key (references `customer. customer_id`)
`dealer_id`: foreign key (references `dealer.dealer_id`)

Entity: `Address`

Attributes in focus:

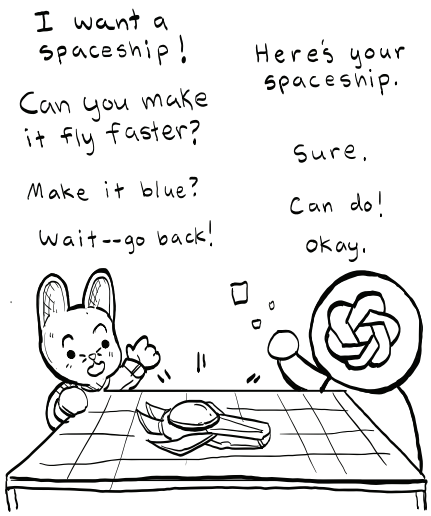
`address_id`: primary key

`country`: CHECK (ensures valid country codes, e.g., CHECK (country IN ('US', 'CA', ...)))

As you see, ChatGPT properly identified most of the constraints, but it missed a few attributes that require `NOT NULL` constraints. Also, it neglected the scenarios in which an attribute requires more than one constraint. `email_address` in `dealer`, for example, requires both `NOT NULL` and `UNIQUE` constraints. You need to fix these problems yourself and store revised text of the constraints so that you can use them for the next step.

Optimization

In optimization, you aim to reduce redundancy and increase the consistency and efficiency of your database design before converting it to code. You can pose some open-ended general questions to ChatGPT (such as "How do I optimize my database design?"). But asking specific questions about certain things that you notice always results in better responses.



You may notice, for example, that the ChatGPT suggested using a `CHECK` constraint on the `country` attribute in `address`. Considering that there are about 200 countries in the world, however, this constraint will be painful to code and messy. You can raise a question about this aspect:

> Q1. Applying a `CHECK` constraint to the `country` attribute in the `address` table is painful because there are about 200 countries in the world. What's a common and recommended approach to store such information without this constraint but still retain the data validity check?

You can raise the same question about the `country_code` attribute in `customer` and `dealer`:

> Q2. To ensure the validity of the country code of phone numbers in the `customer`/`address` tables, what's a common and recommended approach to store such information?

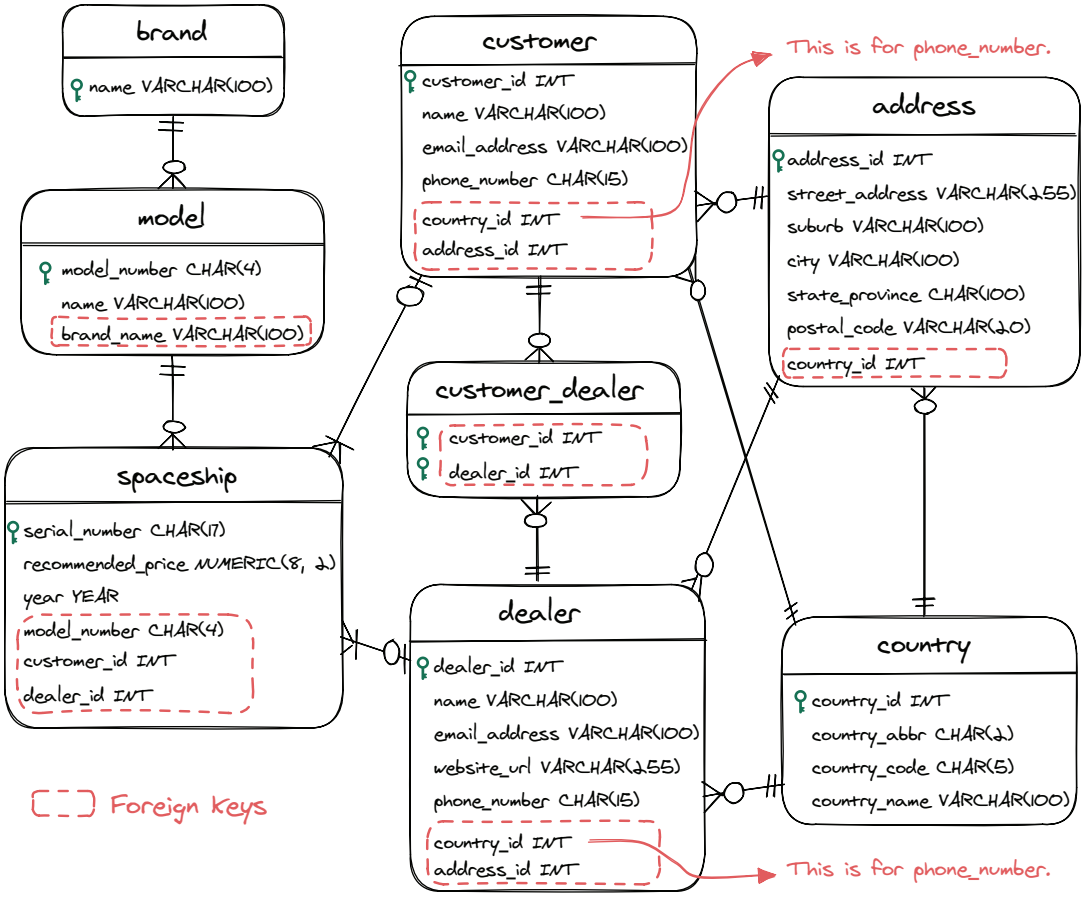
ChatGPT's responses are quite useful:

ChatGPT GPT-4o

Q1 Answer: Instead of using a `CHECK` constraint for the country code, a better approach is to use a reference to a separate `Country` table. This table would contain a list of valid country codes and their corresponding names …

Q2 Answer: To ensure the data validity, you can follow a similar approach to use a separate table …

To synthesize the two suggestions, you can create a single table to store information about each country, such as country abbreviation, country code, and name. The table will be prepopulated with valid information, so it can be linked to `customer`, `dealer`, and `address` to ensure data validity. The updated database design is as follows:



You need to update the information you stored about entities, attributes, relationships, and constraints. Then you will be ready to convert the updated design to SQL code with the help of ChatGPT. To achieve the best result, be specific in your request. Here is an example request that asks for MySQL code:

> I am designing a database. Generate MySQL code based on the given entity, attributes, and relationships, and constraints.
>
> **Entity and attributes:** """
>
> **Entity name of Brand:** `brand` ……
>
> (This is an excerpt. Full text can be found at https://bit.ly/grdb.)
>
> """

(continued)

Relationships: """

brand | model ……

(This is an excerpt. Full text can be found at https://bit.ly/grdb.)

"""

Constraints: """

Entity: `brand` ……

(This is an excerpt. Full text can be found at https://bit.ly/grdb.)

"""

Depending on the RDBMS you are using, you may want the code to be in a different SQL dialect. As long as the provided information is detailed enough, it is hard for ChatGPT to make serious mistakes at this step. You can find the full SQL code in `chapter_08` folder in our GitHub repository. If your requirements have sufficient information about which columns require indexing, chapter 7 shows you how to take preemptive measures to index those columns. Otherwise, you can wait for the data and the use of database tell you more.

Finally, ChatGPT can be very useful for generating sample data that you can use to test your database and for identifying problems preemptively. Following is a sample request for testing data:

I am designing a database. Generate sample data that can be used for testing based on the given SQL code that implements the database.

SQL Code:

```
'''
-- Create brand table
CREATE TABLE brand (name VARCHAR(100) PRIMARY KEY);
……
-- Create customer_dealer table
CREATE TABLE customer_dealer (
  customer_id INT NOT NULL,
  dealer_id INT NOT NULL,
  CONSTRAINT pk_customer_dealer ……
  CONSTRAINT fk_customer_dealer_customer ……
  CONSTRAINT fk_customer_dealer_dealer ……
);
```

(This is a snippet. Full code can be found at https://bit.ly/grdb.)

"""

Now you've gone through the full process of database design with the help of ChatGPT. Good job!

Recap

- To use ChatGPT for database design effectively, follow some rules of thumb: be specific, descriptive, and as detailed as possible; separate instructions from context; and articulate the desired output format through examples.
- You should take a stepwise approach to using ChatGPT to help with database design and make verifications and revisions to ChatGPT's output at each step.
- Store the design at each milestone in text for smooth communication with ChatGPT in the future.
- If you need to communicate your design draft with coworkers, you can convert your design to SQL code with the help of ChatGPT and visualize it by using tools such as dbdiagram.io.

index

Numbers

1NF (First Normal Form) 139
2NF (Second Normal Form) 139
3DES (Triple DES) 185
3NF (Third Normal Form) 139
4NF (Fourth Normal Form) 141
5NF (Fifth Normal Form) 141
6NF (Sixth Normal Form) 141

A

access control 178–182
accuracy, defined 96
address entity 189, 228
address_id attribute 190
address table 190–193, 250
AES (Advanced Encryption Standard) 185
aggregation 14–19
ALTER TABLE command 23
analysis and design 57
AND operator 13
AS clause 15, 32
attributes 74–78
  spaceship manufacturer database 218–232
author entity 123
author table 125
AVG() function 14

B

BCNF (Boyce-Codd Normal Form) 139, 145, 207, 245
BIGINT data type 91
billing_address table 189, 191
Blowfish 185
book entity 123
book_id attribute 123, 125
B-trees 195

C

caching 54
candidate key 79, 80, 140
cardinality 109–128
  direction and representation 110
  many-to-many relationships 121–128
  one-to-many relationships 117–121
  one-to-one relationships 112–116
card_number attribute 94
CASCADE actions 165
categorical data 192
character sets 90
CHAR data type 86, 183
ChatGPT 47, 77
  GPT-4o 214–216
check constraint 172–174, 250
child table 37, 162
code attribute 94
composite primary keys 124
confidentiality 178–187
  access control 178–182
  encryption 182–187
CONSTRAINT…FOREIGN KEY…REFERENCES… clause 37
constraints
  implementation 246
  naming 160
cost attribute 97
COUNT() function 14
country attribute 250
Country table 250
CREATE TABLE command 20, 21
cross join 35
CURRENT_TIMESTAMP function 170, 171
customer entity 224–228
customer table 250

D

data
  adding to tables 19–24
  creating tables 19–24
  managing 35–42
database design 49, 138, 213
  entity-relationship models 104
  relational database design, overview 104

database design goals 50–54
  data consistency and integrity 51
  data security 53
  maintainability and ease of use 51
  performance and optimization 52
  scalability and flexibility 54
databases
  design phases 60–69
  existing databases 61
  goals of 60
  relational 9–11, 26–35
data manipulation language 19
data schema 104
data types 13, 85–100. *See also* specific data types
  decimal 95
  integer 91–94
  string 85–91
  temporal 98–100
DATE data type 98
DATETIME data types 98, 101
DDL (data definition language) 19–24
dealer entity 224–228
dealer table 250
DECIMAL data type 95–96, 101
default constraint 169
delete anomaly 8, 51
denormalization 201–209
  database design 205–209
  overview 201–205
dot notation 31
DOUBLE data type 95
DROP TABLE command 24

E

email_address attribute 224
encryption 182–187
entities 6, 74–78. *See also* specific entities
  relationships among 232–244
  spaceship manufacturer database 218–232
  strong 129–136
  weak 129–136
E-R (entity-relationship)
  diagram 74, 93, 104, 142, 220
  models 104
expiry_date attribute 94

F

Fifth Normal Form (5NF) 141
filtering 12
First Normal Form (1NF) 139
fixed-point data types 95
FLOAT data type 95
floating-point data types 95
foreign keys 236–242
  constraint 161
  defined 27
Fourth Normal Form (4NF) 141
FROM clause 11, 34
full-text indexes 197
functional dependency 144–146

G

generative AI 214
GROUP BY clause 15, 16
GROUP keyword 77

H

hash function 183
hashing 182

I

imperative language 11
implementation 156–174, 245–253
  check constraint 172–174
  constraints 246
  default constraint 169
  foreign key constraint 161
  NOT NULL constraint 157
  optimization 249–253
  primary key constraint 159
  referential actions 164–167
  unique constraint 167
implementation/integration and testing phase 59
inconsistencies of data types across RDBMSs 101
indexing 52, 193–201
  for ER diagrams 198–200
  overview of full-text indexes 197
  overview of standard indexes 194–198
information management 61
INNER JOIN 46
INSERT INTO command 22
insertion anomaly 8, 51
INSERT keyword 77
INT data type 91
integer data types 91–94
integrity 177
interviews 63–69
  identifying subjects, characteristics, and relationships 65–69
  preparing for 63
IVs (initialization vectors) 186

J

JOIN clause 34, 46
JOIN keyword 77
JOIN…ON clause 30
joins, types of 32
JOINs vs. WHERE 34

K

keys 78–84

L

last_login_time attribute 100
LEFT JOIN 46
LLMs (large language models) 213
logarithmic time 195

M

MAC (mandatory access control) 178
manufacturer attribute 90

many-to-many relationships 121–128
identifying in database 126
max cardinality 110
MAX() function 14
MEDIUMINT data type 91
MIN() function 14
minimum cardinalities 110, 236–242
multivalued columns 142

N

name attribute 90, 224
NO ACTION action 165
nonempty (non-nullability) requirement 81
normalization 137–155
database design 147
functional dependency 144–146
multivalued columns 142
primary keys 142
transitive dependency 150–155
NOT NULL constraint 157
NUMBER data type 101
NUMERIC data type 96
NVARCHAR(MAX) data type 101

O

one-to-many relationships 117–121
identifying in database 119
one-to-one relationships 112–116
identifying in database 114
OOP (object-oriented programming) 75
ORDER keyword 77
OR operator 13
OUTER JOIN 46

P

parent table 37, 162
partial keys 129
payment_id attribute 93
payment_method entity 93
payment_method table 189, 191
indexing for ER diagrams 200
phone_number attribute 224, 225
postal_code attribute 115, 228
power(x) function 144
precision 96
price attribute 95, 97
primary keys 6, 142
composite 124
constraint 159
defined 78
product codes 81
product entity 93
product_id column 27, 37
product_id value 40
product names 81
product_price attribute 97
product_review table 58
product table 9, 26
indexing for ER diagrams 199
purchase_time attribute 100

Q

queries 11–19
aggregation 14–19
filtering 12

R

RBAC (role-based access control) 178
RDBMSs (relational database management systems) 3, 8, 25, 73, 103, 137, 176, 214
README.md file 10, 137, 175
redundant tables 188–192
referential actions 164–167
referential integrity 38
related tables 25–39
joining data from 29
managing data in 39
number of shared columns 27
types of joins 32
WHERE vs. JOIN 34
relational database design 4–9, 50
connecting related entities 106
entities 6
entities and attributes 73
implementation 156–174
overview 74, 104
overview of design process 55–60
primary keys 6
RDBMS (relational database management systems) 8
related tables 25
security and optimization 176
SQL 8
tables 6
relational databases 5
data types 85–100
designing implementation 245–253
managing data and tables 35–42
overview 26
related tables 26–39
SQL, first query 9–11
relationships 103
among entities 232–244
requirement gathering, defined 56
requirements analysis 217
reserved keywords 77
RESTRICT action 165
review table 26
indexing for ER diagrams 200
review_time attribute 98, 100
RIGHT JOIN 46

S

scale 96
Second Normal Form (2NF) 139
security 176
and optimization 175
confidentiality 178–187
integrity 177
SELECT statement 11, 16, 17, 19, 77, 93
SET NULL action 166
SHIPS R US database, implementation 245–253
constraints 246
optimization 249–253

Sixth Normal Form (6NF) 141
SMALLINT data type 91
SMEs (subject-matter experts) 62
spaceship manufacturer database,
  entities and attributes 218–232
  attributes 219, 223–228
  communicating design 230
  entities 218
SQL (Structured Query Language)
  5, 8
  first query 9–11
  learning on your own 42–45
  queries 11–19
standard indexes 194–197
state attribute 115
state table 189, 192, 193
storage considerations 187–193
  categorical data 192
  redundant tables 188–192
street_address column 189
string data types 85–91
strong entities 129–136
  identifying in database 130–136
suburb attribute 228
sum aggregate function 43
sum function 44
SUM function 14, 45
superkey 140
surrogate key 83

T

tables 6. *See also* specific tables
  creating, altering and dropping
    19–24
  managing 35–42
temporal data types 98–100
testing 59
TEXT data type 86, 87, 90,
  101
Third Normal Form (3NF) 139
TIME data type 98
TIMESTAMP data type 98
TINYINT data type 91
total_price attribute 97
transitive dependency 150–155

U

unique constraint 167
UPC (Universal Product Code)
  82, 94
update anomaly 28, 51
user_address entity 115, 116
user table 189, 190, 191, 192
UTC (Coordinated Universal
  Time) 99

V

VARCHAR data type 86
VARCHAR2 data type 101
VARCHAR(15) 226
VARCHAR(255) 226

W

weak entities 129–136
  identifying in database 130–136
WHERE clause 11, 12, 34, 46

Achievement & Skills Map

Goal Achieved	Skill Gained	Page Number
Foundational knowledge of SQL and relational concepts	Proficiency in basic SQL queries, data manipulation, and table management	Page 4
Relational database design skills	Ability to extract entities and attributes from requirement analysis	Page 74
	Ability to identify the data types for attributes	Page 85
	Ability to identify relationships between entities and cardinalities of relationships	Page 104
Normalization of relational databases	An understanding of different normal forms; ability to normalize a given database design based on the understanding of normal forms	Page 138
Implementation of relational databases	Ability to translate a database design to SQL code, implementing constraints according to the design	Page 156
Database security	Ability to take measures to secure a database, such as encryption and access control	Page 176
Database optimization	Ability to take measures to optimize a database, such as indexing and denormalization	Page 193
Use generative AI to speed up database design	Ability to use accurate prompts to query generative AI tools to design, optimize, and implement databases iteratively	Page 214

DATABASES

grokking
Relational Database Design

Qiang Hao • Michail Tsikerdekis



Almost every business uses a relational database system. Whether you're a software developer, an analyst creating reports and dashboards, or a business user just trying to pull the latest numbers, it pays to understand how a relational database operates. This friendly, easy-to-follow book guides you from square one through the basics of relational database design.

Grokking Relational Database Design introduces the core skills you need to assemble and query tables using SQL. The clear explanations, intuitive illustrations, and hands-on projects make database theory come to life, even if you can't tell a primary key from an inner join. As you go, you'll design, implement, and optimize a database for an e-commerce application and explore how generative AI simplifies the mundane tasks of database designs.

What's Inside

- Define entities and their relationships
- Minimize anomalies and redundancy
- Use SQL to implement your designs
- Security, scalability, and performance

For self-taught programmers, software engineers, data scientists, and business data users. No previous experience with relational databases assumed.

Dr. Qiang Hao and *Dr. Michail Tsikerdekis* are both professors of Computer Science at Western Washington University.

The technical editor on this book was Louis Davidson.

Cody Pham created the illustrations for this book.

"If anyone is looking to improve their database design skills, they can't go wrong with this book."
—Ben Brumm, DatabaseStar

"Goes beyond SQL syntax and explores the core principles. An invaluable resource!"
—William Jamir Silva, Adjust

"Relational database design is best done right the first time. This book is a great help to achieve that!"
—Maxim Volgin, KLM

"Provides necessary notions to design and build databases that can stand the data challenges we face."
—Orlando Méndez, Experian

For print book owners, all digital formats are free:
https://www.manning.com/freebook

ISBN-13: 978-1-63343-741-8



